

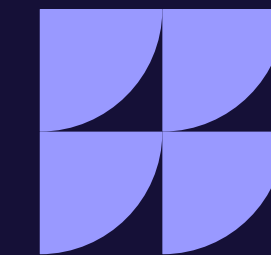
LÀM QUEN CÁC THUẬT TOÁN MACHINE LEARNING CƠ BẢN **SVM**

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

<https://cybersoft.edu.vn/>



Nội dung & kết quả



➤ Support Vector Machine

➤ Bài tập thực hành

➤ K-Means

➤ Dự án trên lớp

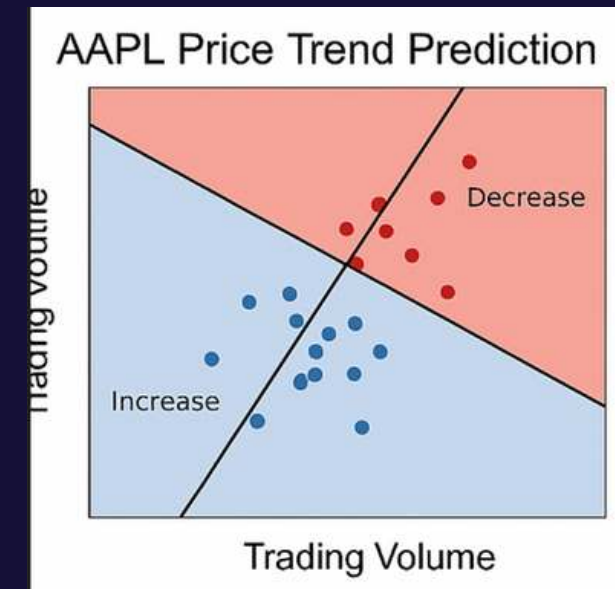
➤ Giảm chiều dữ liệu

➤ Bài tập về nhà

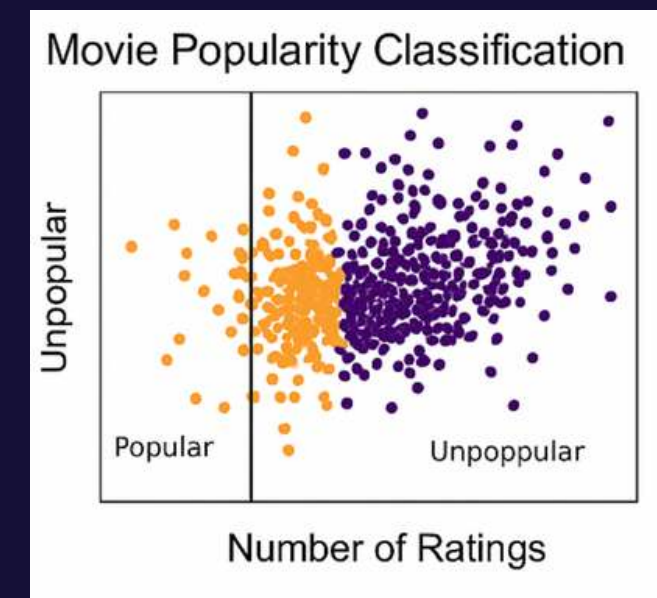
CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Kiến thức học được

Mục tiêu

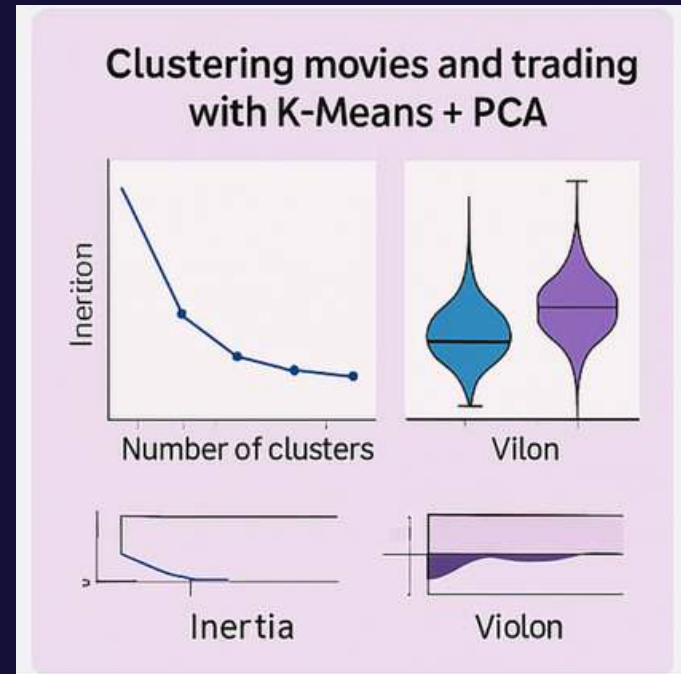


Dự đoán xu hướng giá cổ phiếu, cảm xúc văn bản, mức độ phổ biến phim bằng SVM.

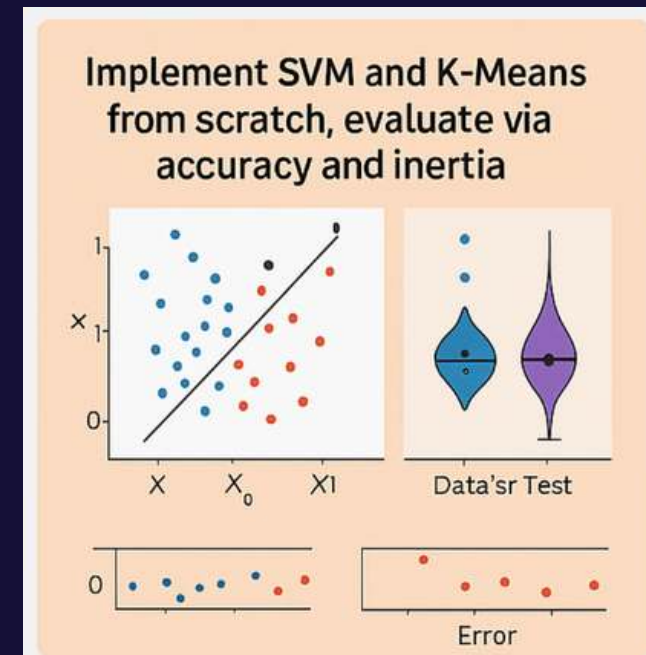


Trực quan hóa kết quả và hiểu rõ cơ chế hoạt động qua việc tự cài đặt thuật toán.

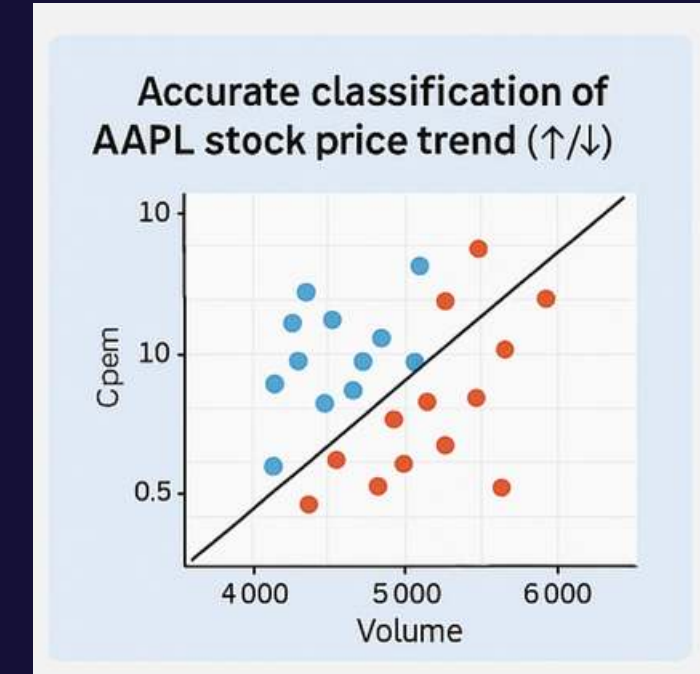
Kiến thức học được



Phân cụm phim và giao dịch bằng K-Means + PCA.



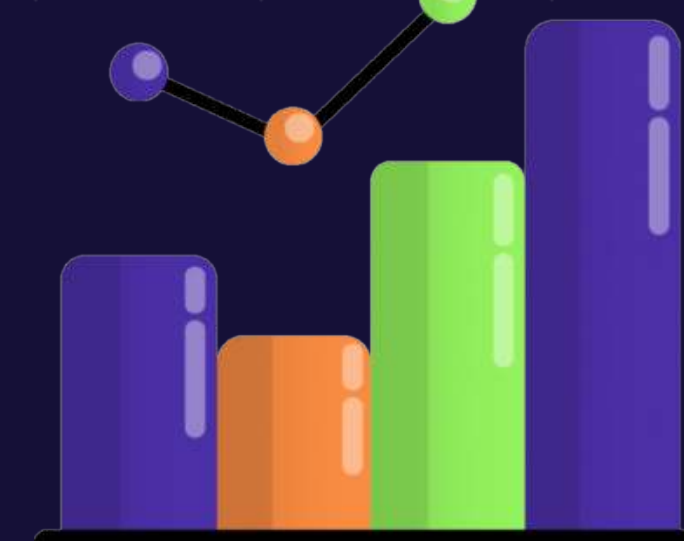
Triển khai SVM và K-Means từ đầu, đánh giá qua accuracy và inertia.



Vẽ scatter plot, confusion matrix, elbow & violin plots.

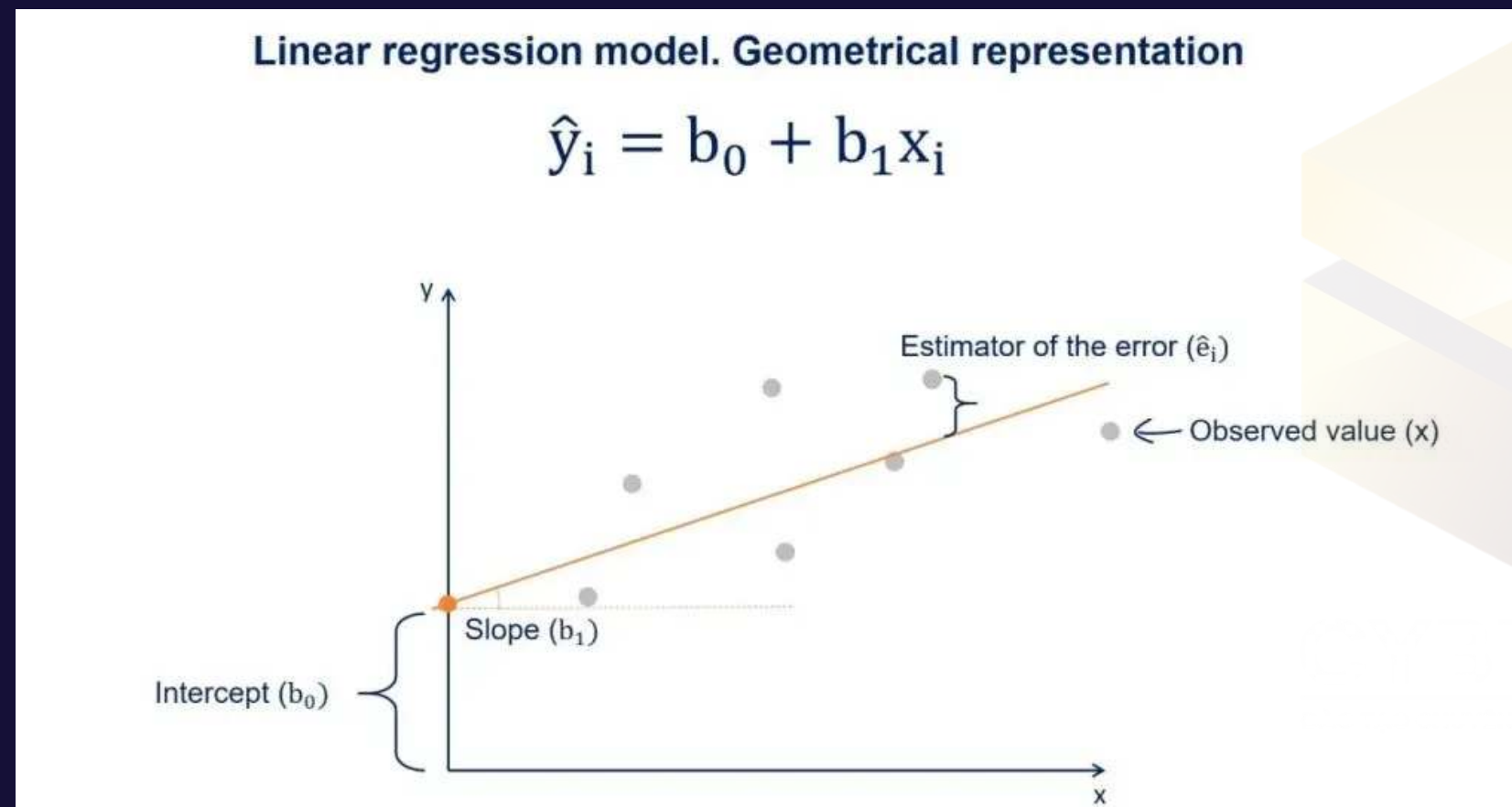
Mục tiêu

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



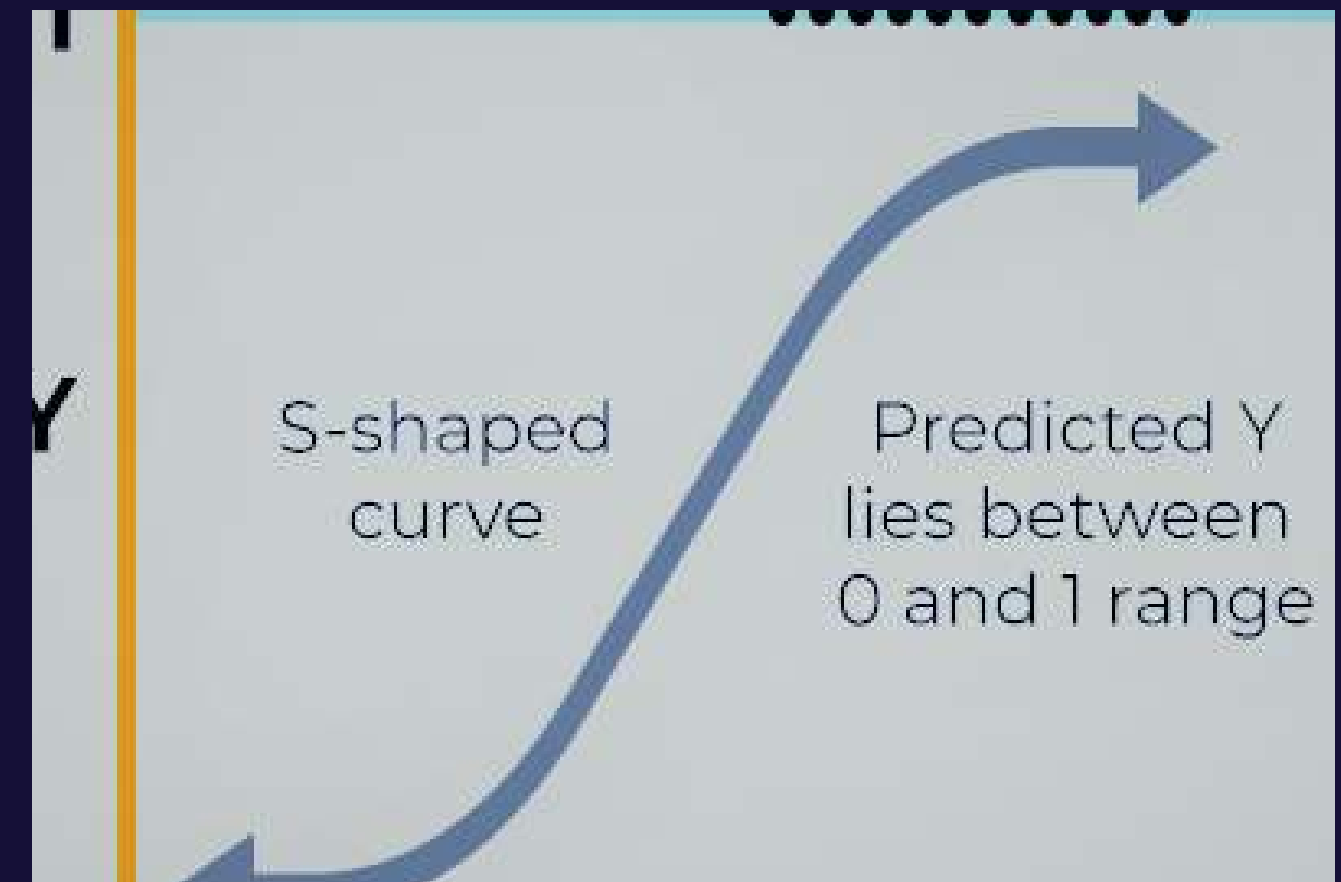
Machine Learning cơ bản

Linear Regression



Dự đoán giá trị liên tục (số thực) từ các đặc trưng đầu vào bằng cách tìm đường thẳng/hyperplane tốt nhất.

Logistic Regression

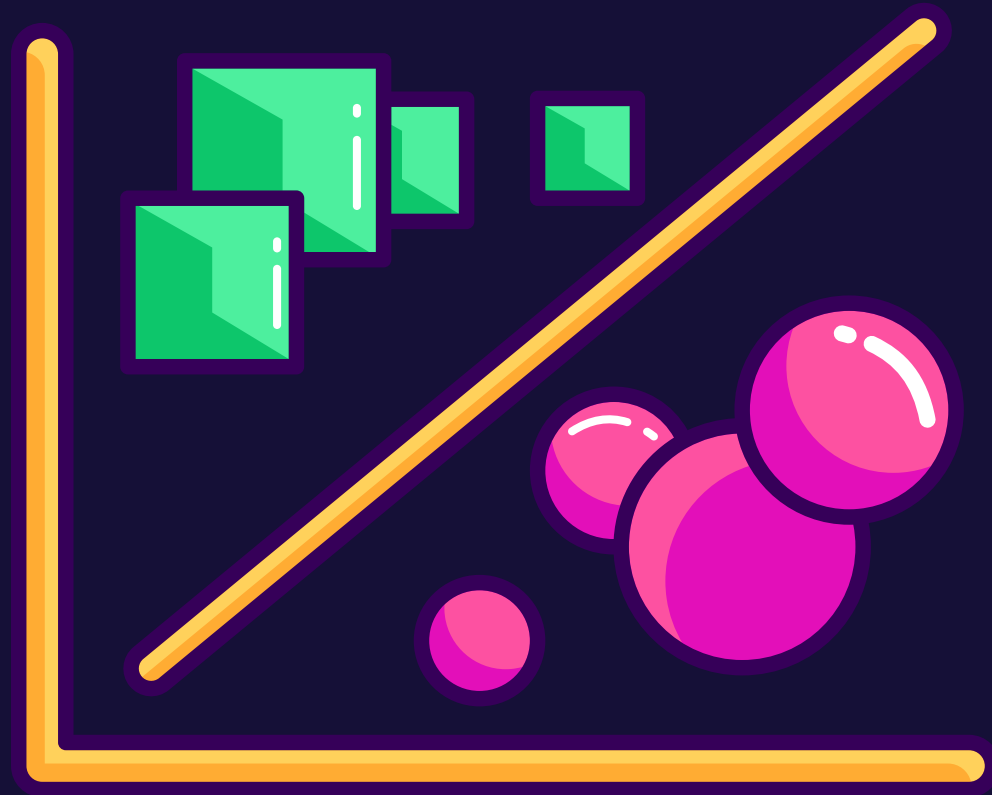


Dự đoán xác suất và phân loại đầu ra nhị phân (ví dụ: có bệnh/không bệnh) bằng hàm sigmoid.

Machine Learning nâng cấp

Phân lớp

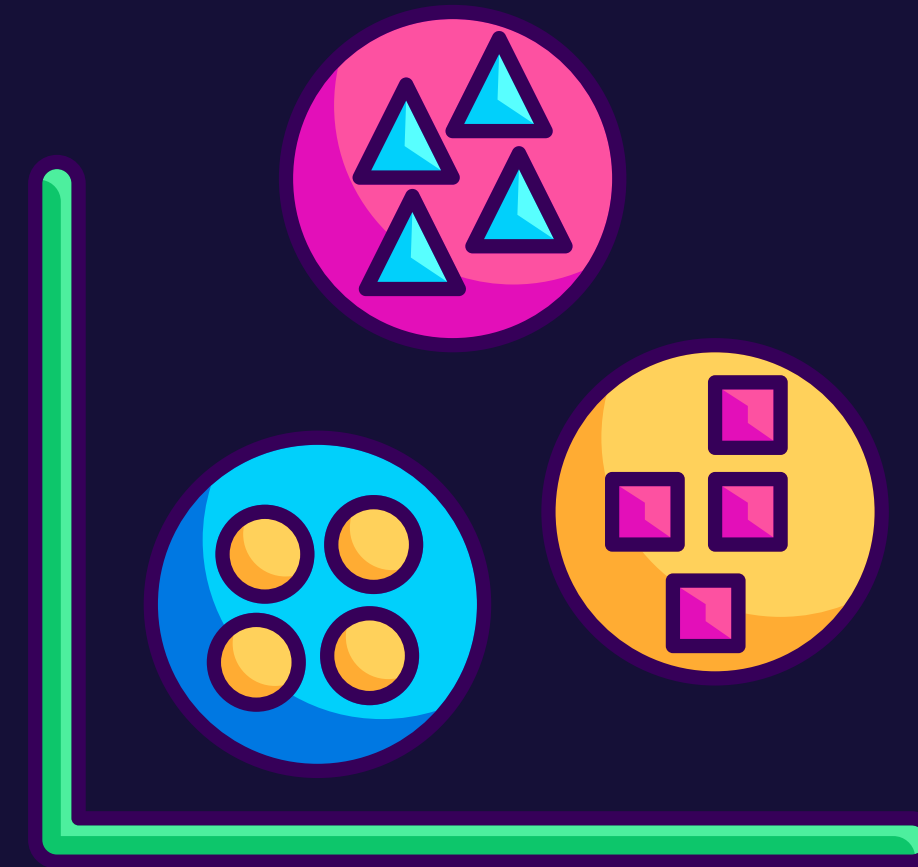
SVM



Phân loại dữ liệu bằng cách tìm siêu phẳng tối ưu (hyperplane) để tách biệt các lớp với khoảng cách (margin) lớn nhất.

Phân cụm

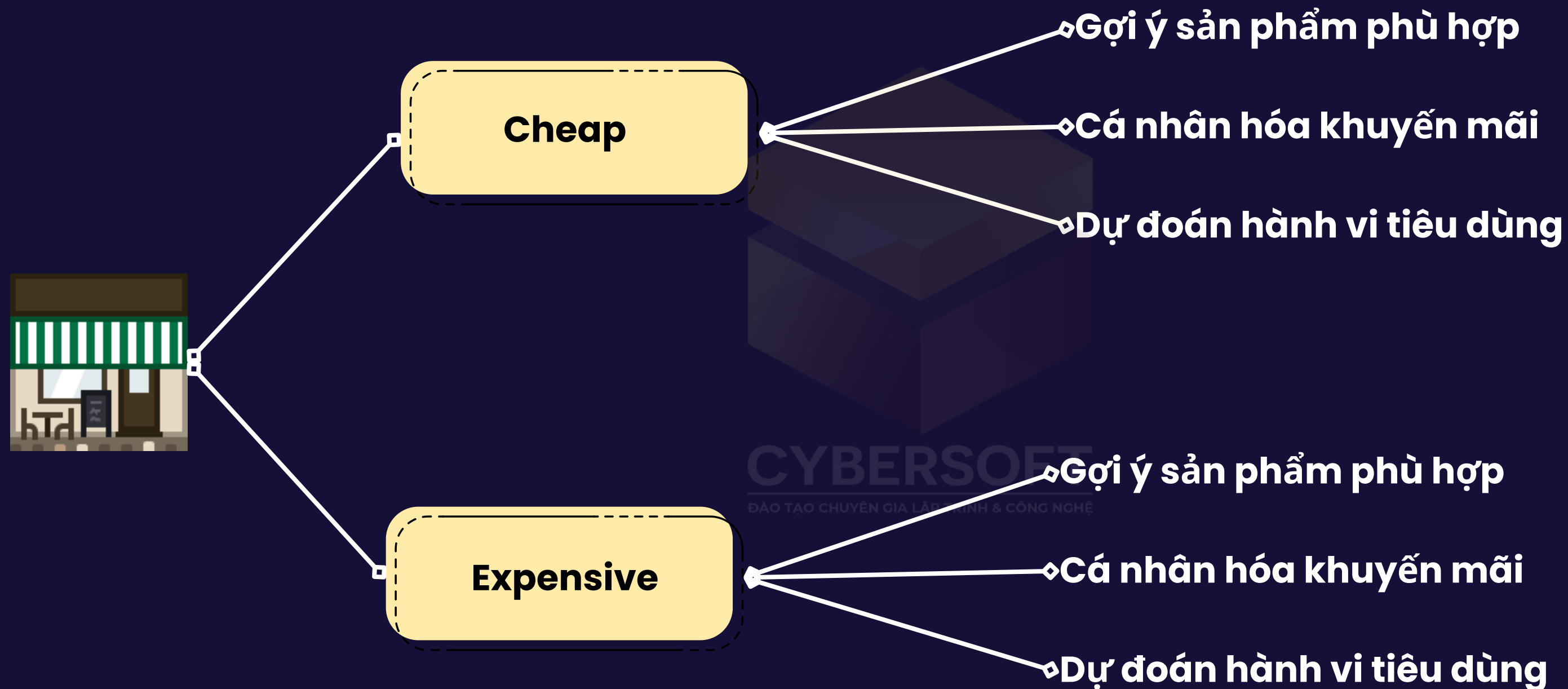
Clustering



Nhóm các điểm dữ liệu chưa gán nhãn thành từng cụm dựa trên khoảng cách và sự tương đồng mà không cần nhãn trước.

Tại sao lại phân lớp?

Phân lớp giúp hệ thống tự động nhận diện nhóm đối tượng và đưa ra quyết định phù hợp mà không cần can thiệp con người.



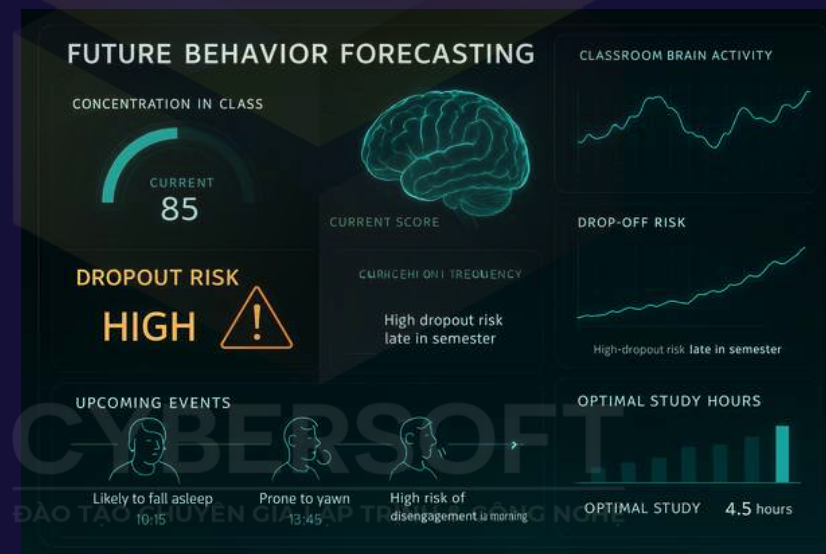
Tại sao lại phân lớp?

Thông qua phân lớp, doanh nghiệp có thể

Xác định nhóm khách hàng mục tiêu



Dự báo hành vi trong tương lai



Tối ưu chiến lược bán hàng, quảng cáo, chăm sóc khách hàng



=> Phân lớp là chìa khóa để hệ thống AI hiểu và phản ứng khác nhau với từng nhóm người dùng hoặc tình huống khác nhau, một cách thông minh và tự động.

Định nghĩa phân lớp

Phân lớp (Classification) là một kỹ thuật trong học máy (machine learning) dùng để chia dữ liệu vào các nhóm (lớp) xác định trước dựa trên đặc trưng đầu vào.



→ Cat



→ Email Spam



→ Cheap, Expensive

=> Phân lớp là quá trình dạy cho máy tính biết cách nhận diện và gán nhãn dữ liệu mới vào đúng nhóm, dựa trên những đặc điểm đã học từ dữ liệu cũ.

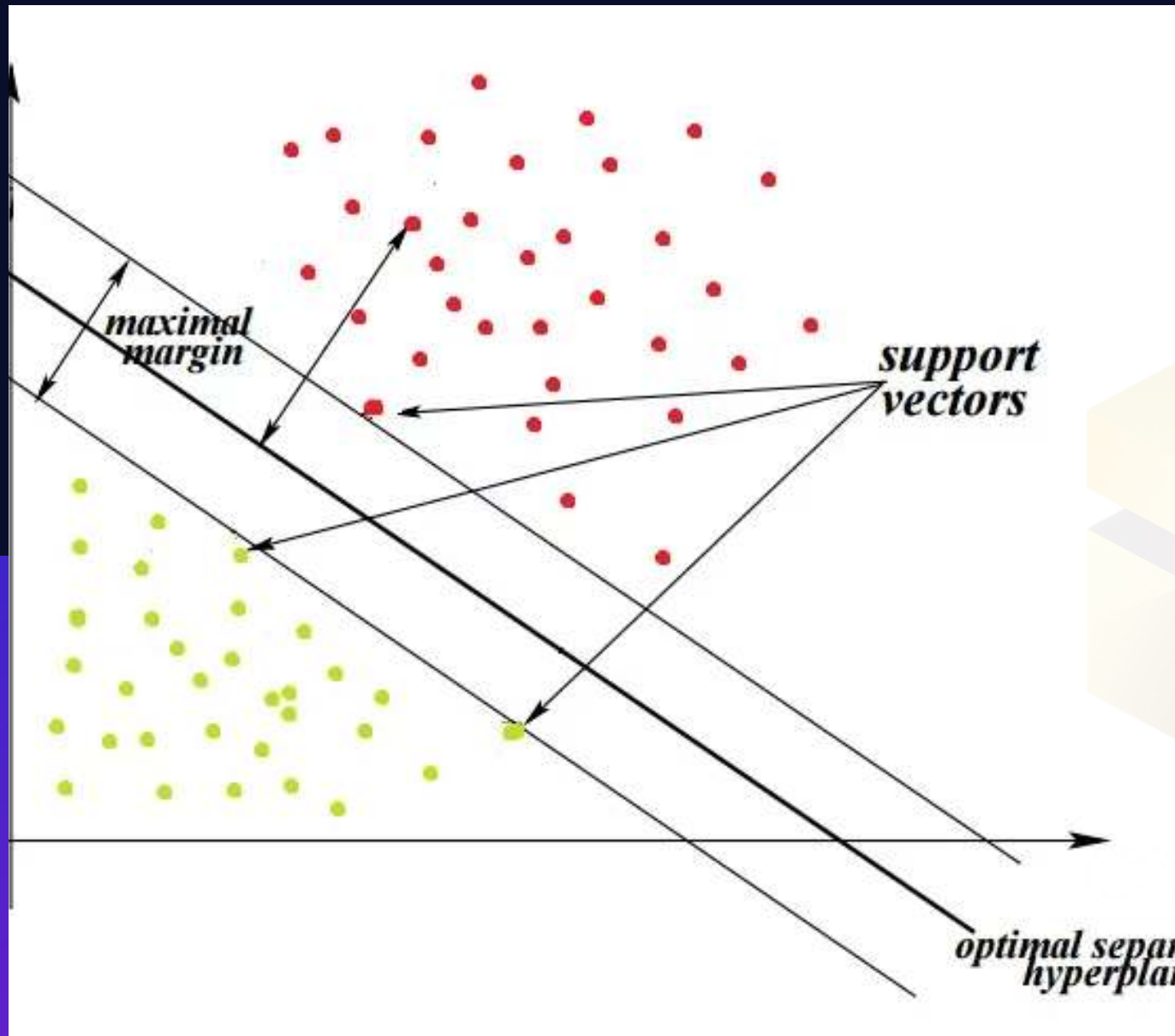
CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Support Vector Machine

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Support Vector Machine

Các nội dung tìm hiểu



01. Bài toán tiếp cận (Starbucks ở Singapore)

02. Support Vectors

03. Đường phân loại (Decision Boundary)

04. Hyperplane & Margin

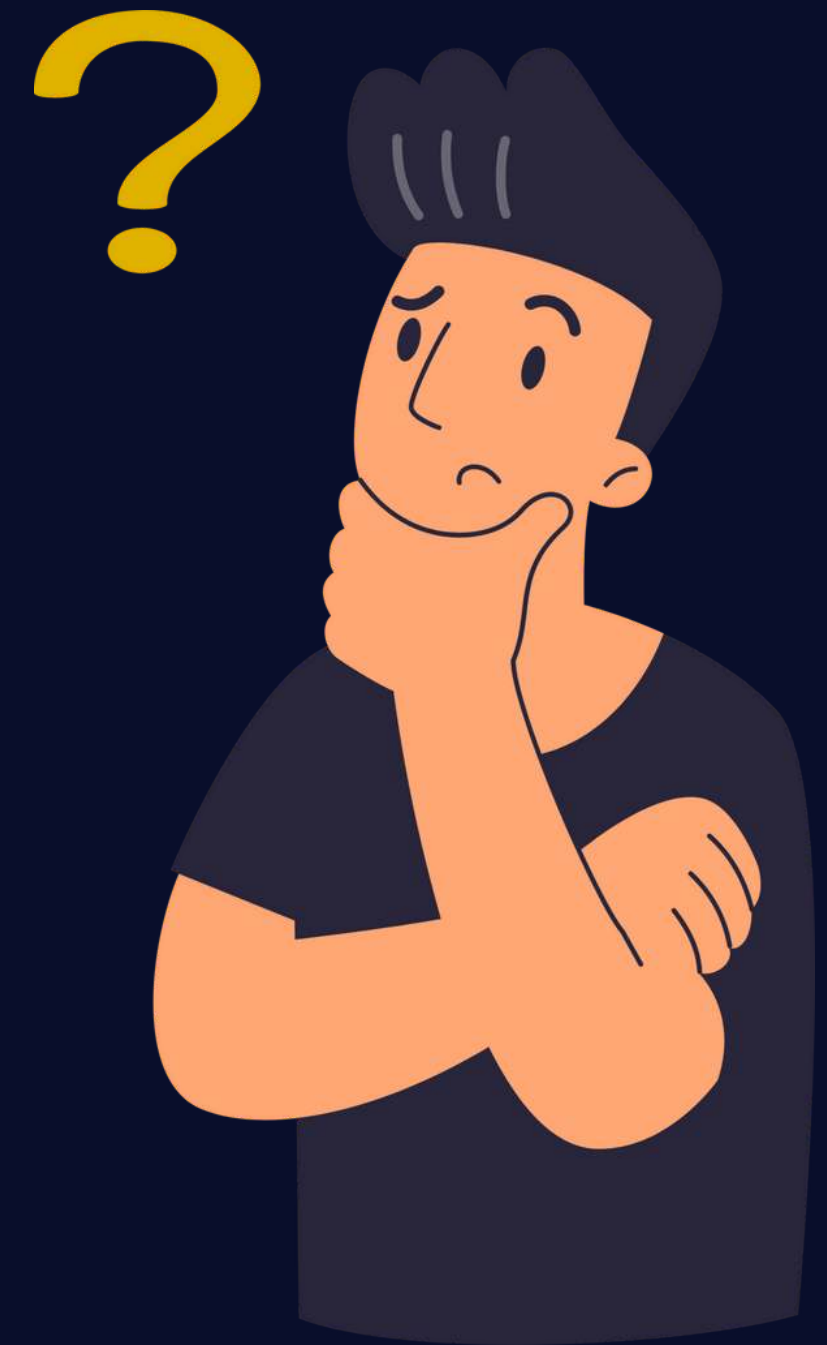
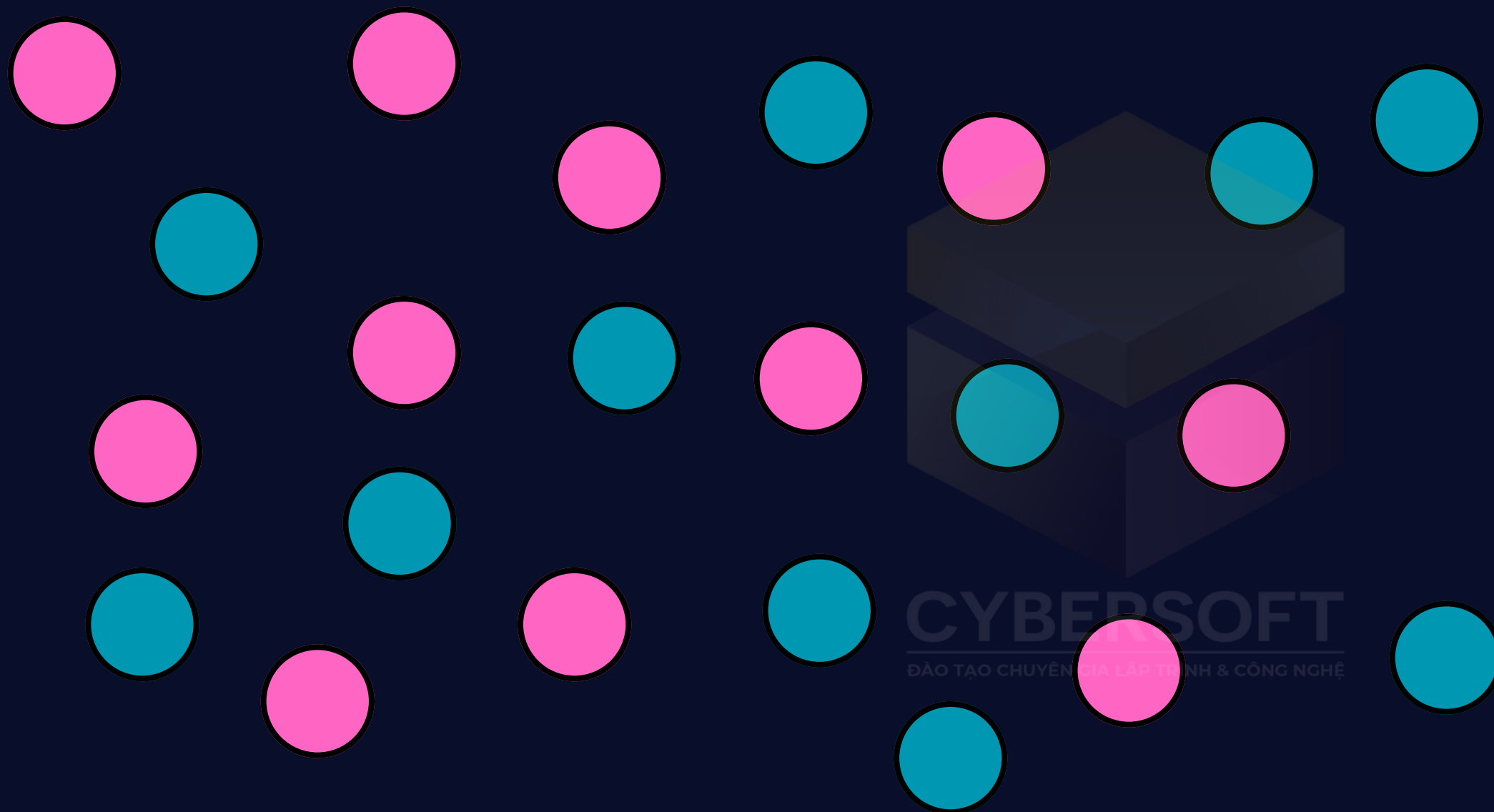
05. Accuracy (Chỉ số chính xác)

06. LOSS VALUE

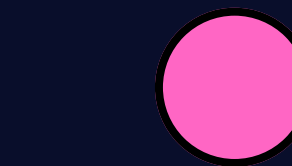
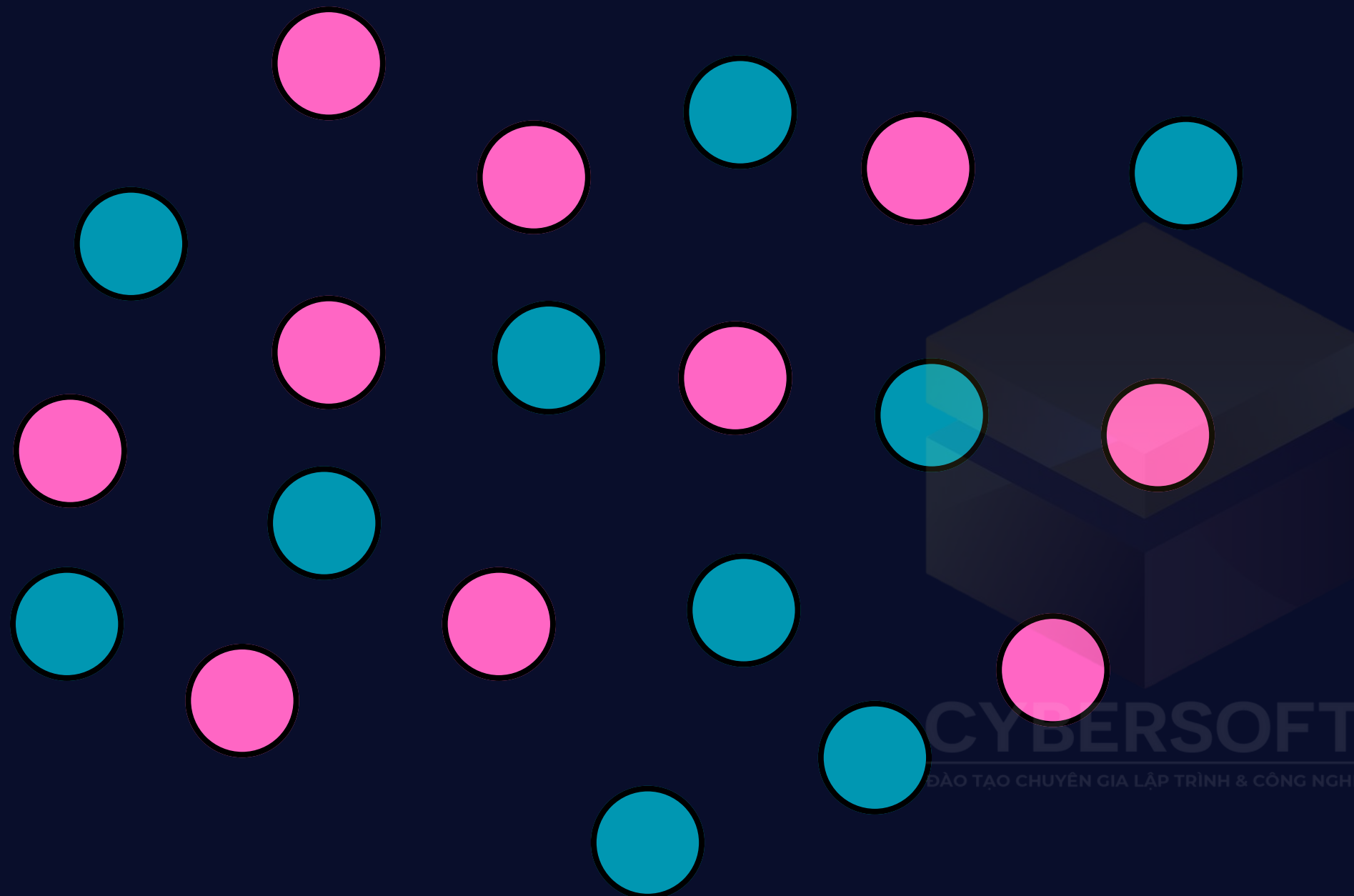
07. KERNEL Trick

08. Bài tập thực hành

Các bài toán chúng ta cần dự đoán dựa trên các labels đã có sẵn
và dựa trên các features có thể phân loại



Các bài toán chúng ta cần dự đoán dựa trên các labels đã có sẵn
và dựa trên các features có thể phân loại

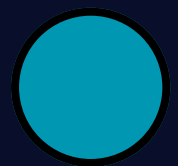


➤ Email spam

➤ Ảnh dog

➤ Khối u lành tính

➤ Giao dịch bình thường



➤ Email không spam

➤ Ảnh cat

➤ Khối u ác tính

➤ Giao dịch gian lận

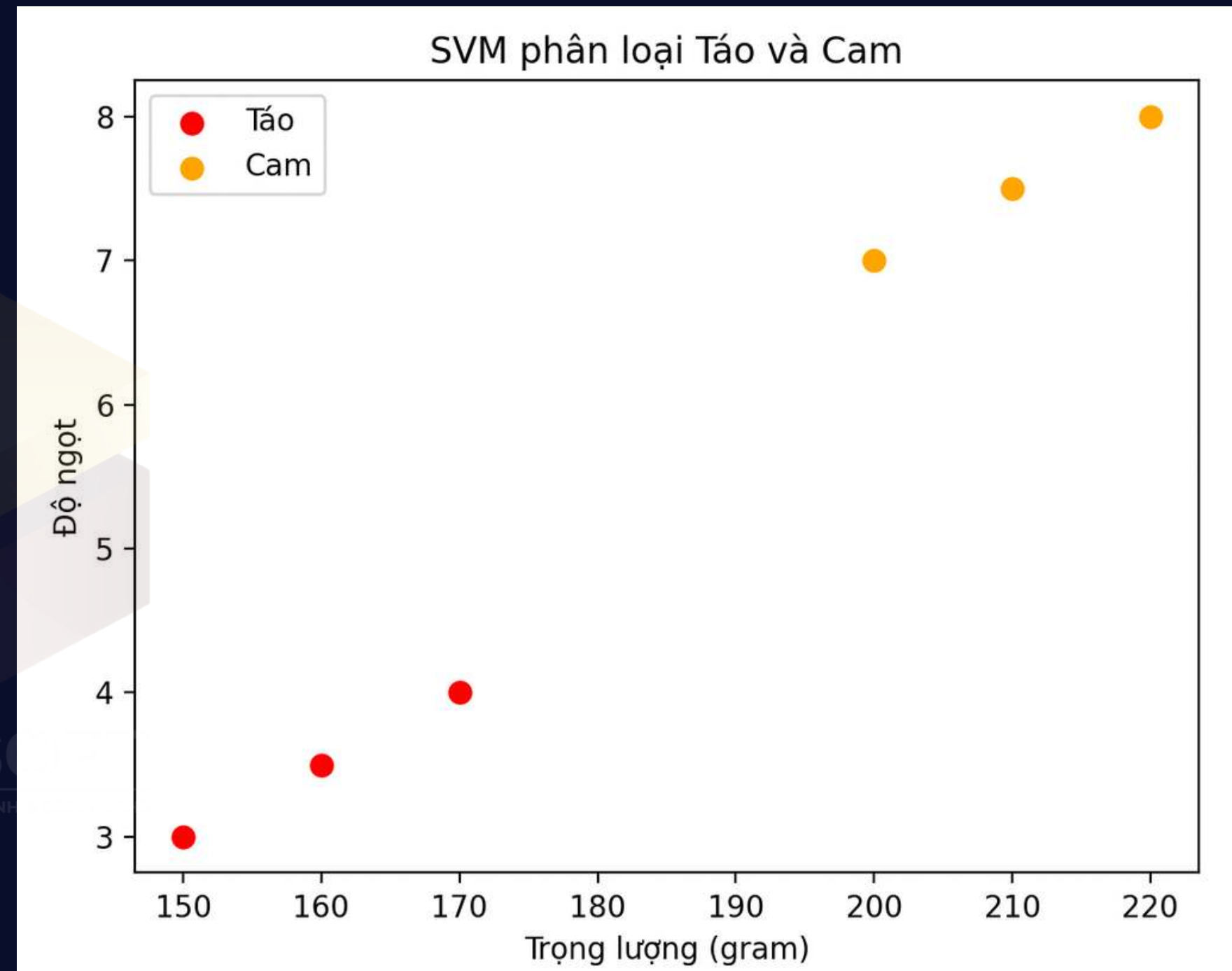
Ta có một tập dữ liệu sau dựa trên đặc 2 đặc trưng.
Và có nhãn 0: Táo, 1: Cam

#Trọng lượng
(gram)

#Độ ngọt

#Label

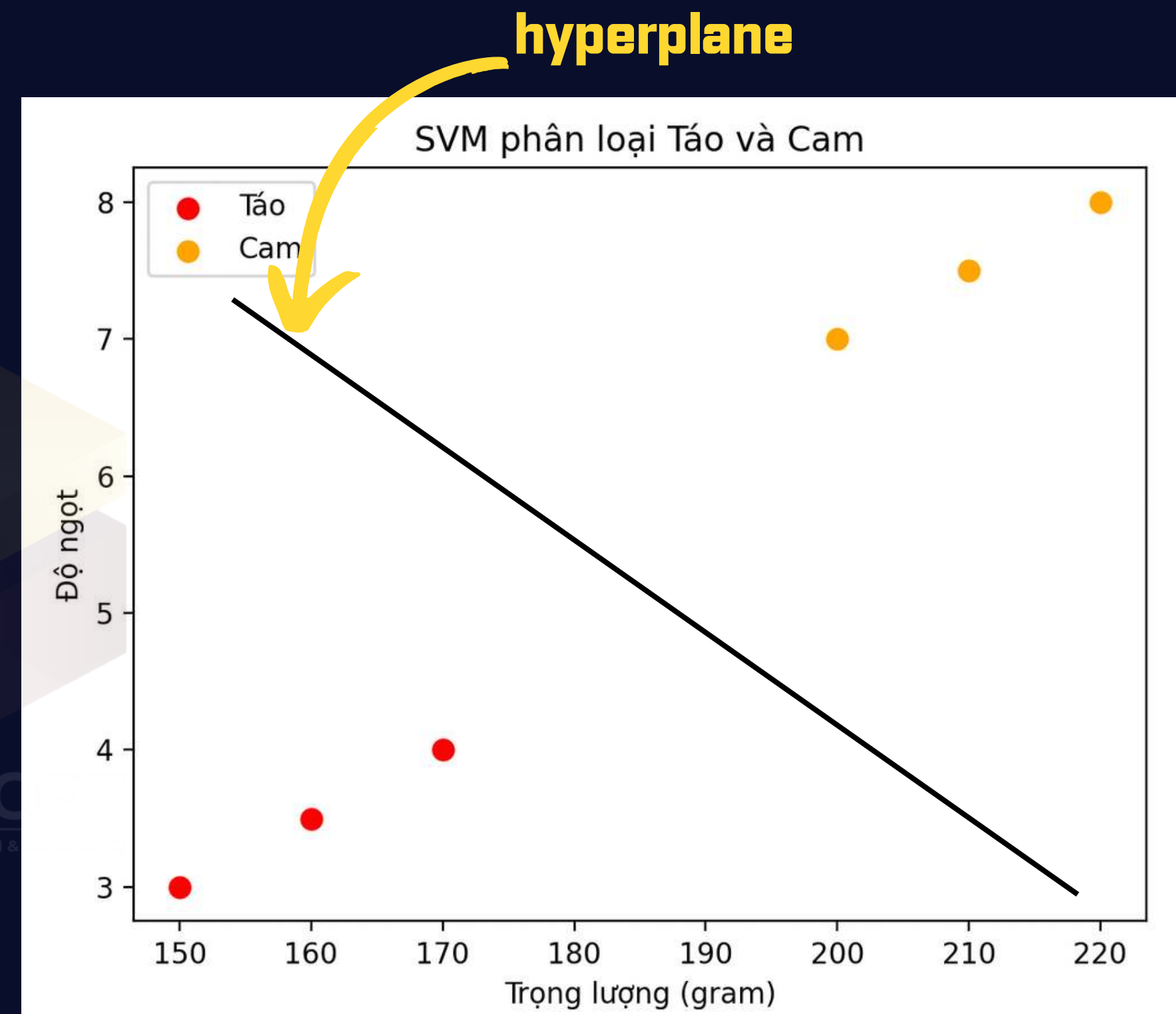
150	3	0
170	4	0
160	3.5	0
200	7	1
220	8	1
210	7.5	1



Ta có một tập dữ liệu sau dựa trên đặc 2 đặc trưng.
Và có nhãn 0: Táo, 1: Cam

#Trọng lượng
[gram] #Độ ngọt #Label

150	3	0
170	4	0
160	3.5	0
200	7	1
220	8	1
210	7.5	1



👉 Mục tiêu của SVM là tìm ra một ranh giới (gọi là hyperplane) tốt nhất để chia tách các lớp dữ liệu khác nhau.

Dataset Visualize



Starbucks ở Singapore. Muốn phân loại khách hàng thành hai nhóm:

- "Mua cà phê đắt" (>5 USD, nhãn +1)
- "Mua cà phê rẻ" (≤ 5 USD, nhãn -1)

=> Dựa trên Thời gian ở quán (Time Spent, phút) và Thu nhập hàng tháng (Monthly Income, USD). Dữ liệu từ 6 khách hàng:

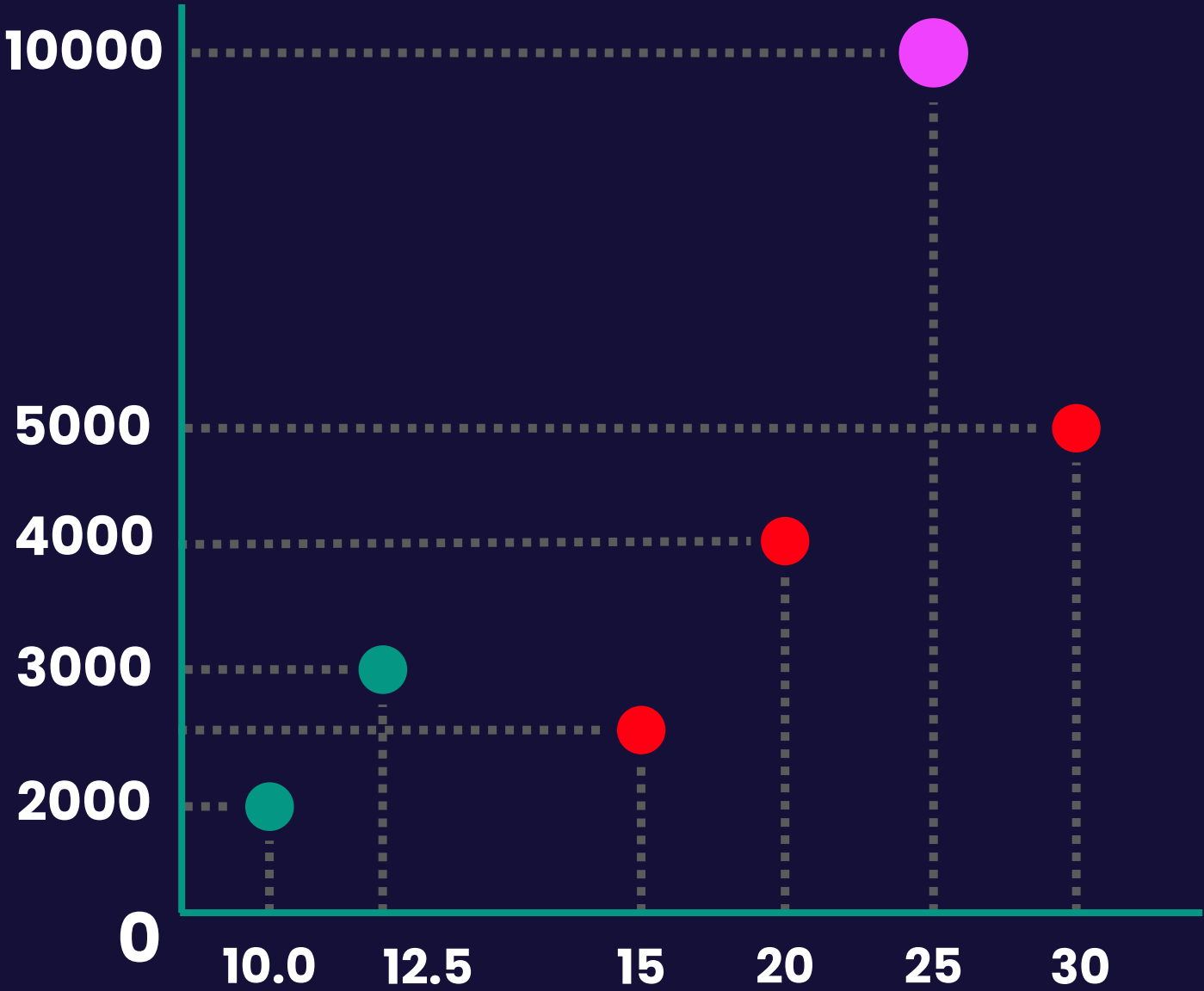
Monthly Income (USD)	Time Spent (min)	Nhóm khách hàng
2000	10	-1
4000	20	1
2500	15	-1
5000	30	1
3000	12	-1
10000	25	-1

Có thể tìm một đường phân lớp tối ưu giữa 2 nhóm khách hàng này không?

outlier

Monthly Income (USD)	Time Spent (min)	Nhóm khách hàng
2000	10	-1
4000	20	1
2500	15	-1
5000	30	1
3000	12	-1
10000	25	-1

Monthly income (USD)

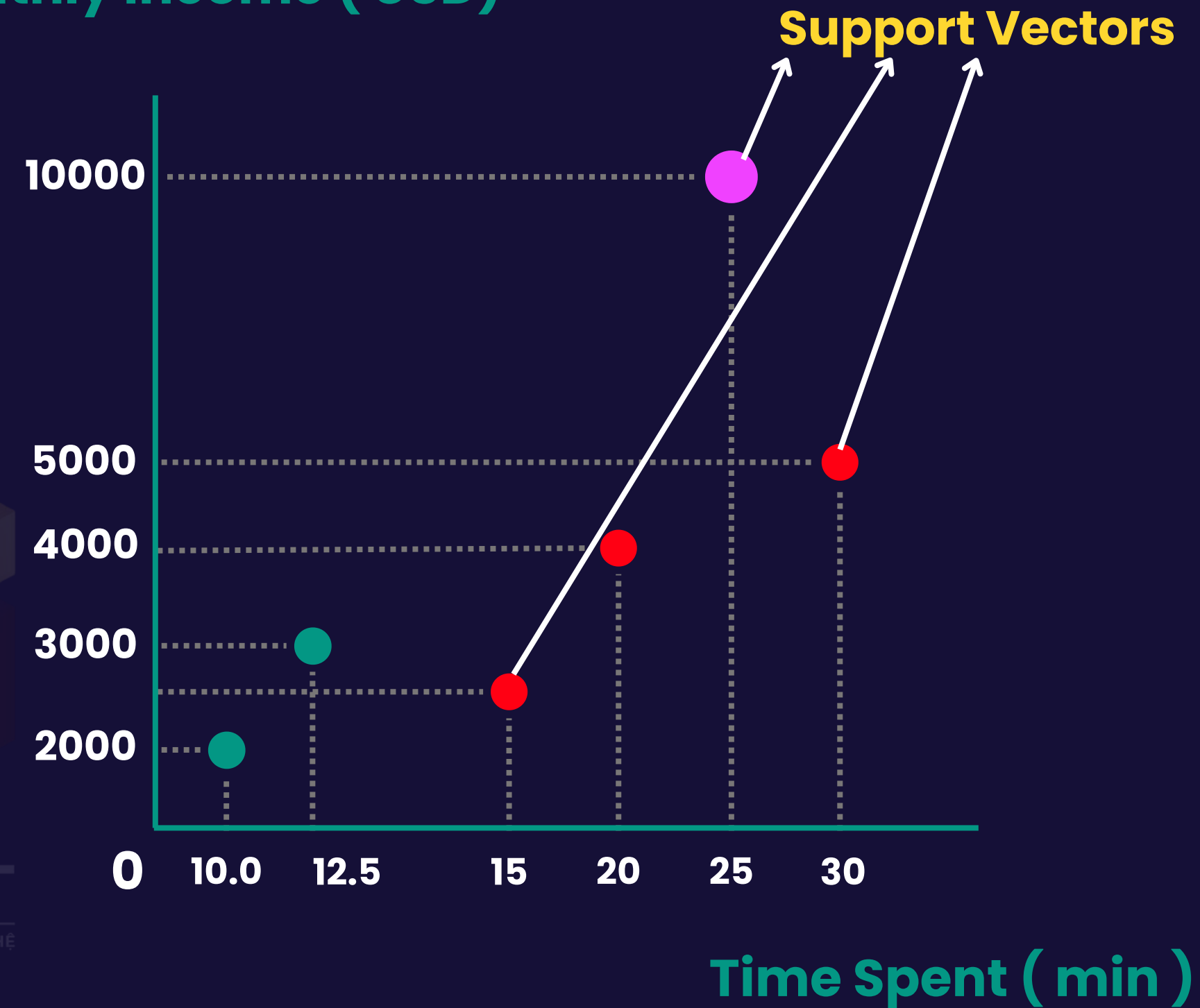


- Cheap (-1)
- Expensive (1)

Support Vectors

Monthly Income (USD)	Time Spent (min)	Nhóm khách hàng
2000	10	-1
4000	20	1
2500	15	-1
5000	30	1
3000	12	-1
10000	25	-1

Monthly income (USD)



- Không phải tất cả điểm đều là **support vector**. Chỉ những điểm "quan trọng" ở ranh giới quyết định mới là **support vector**.

Vẽ plot dataset

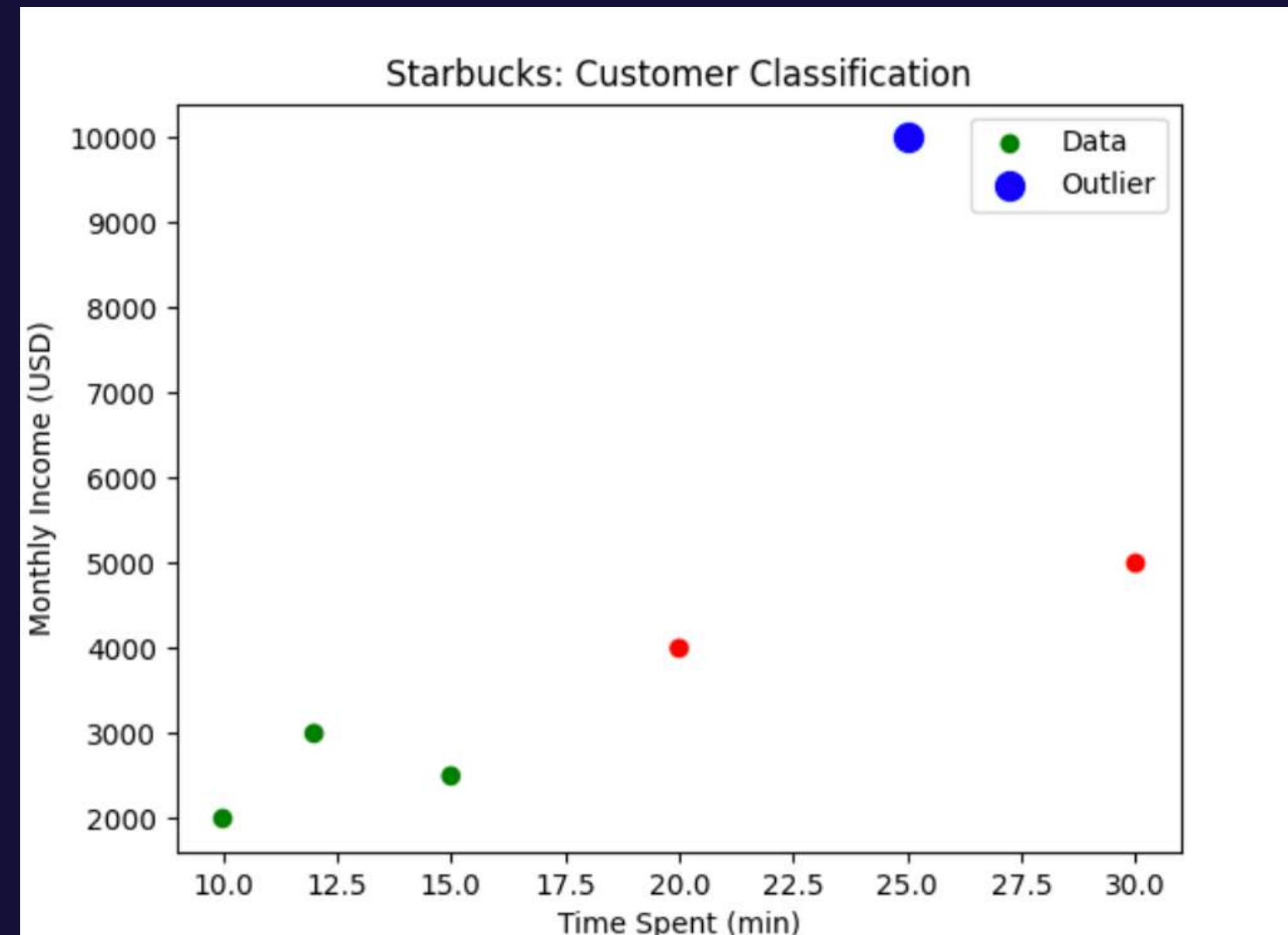
```
import matplotlib.pyplot as plt
import numpy as np

# Dữ liệu
X = np.array([[10, 2000], [20, 4000], [15, 2500], [30, 5000], [12, 3000], [25, 10000]])
y = np.array([-1, 1, -1, 1, -1, -1])
labels = ['Cheap' if label == -1 else 'Expensive' for label in y]

plt.scatter(X[:, 0], X[:, 1], c=['green' if y[i] == -1 else 'red' for i in range(len(y))],
            label='Data')
plt.scatter([25], [10000], c='blue', s=100, label='Outlier')
plt.xlabel('Time Spent (min)')
plt.ylabel('Monthly Income (USD)')
plt.title('Starbucks: Customer Classification')
plt.legend()
plt.savefig('starbucks_customer_scatter.png')
plt.close()
```

=> Mục tiêu là phân loại hành vi khách hàng và phát hiện bất thường trong mô hình tiêu dùng.

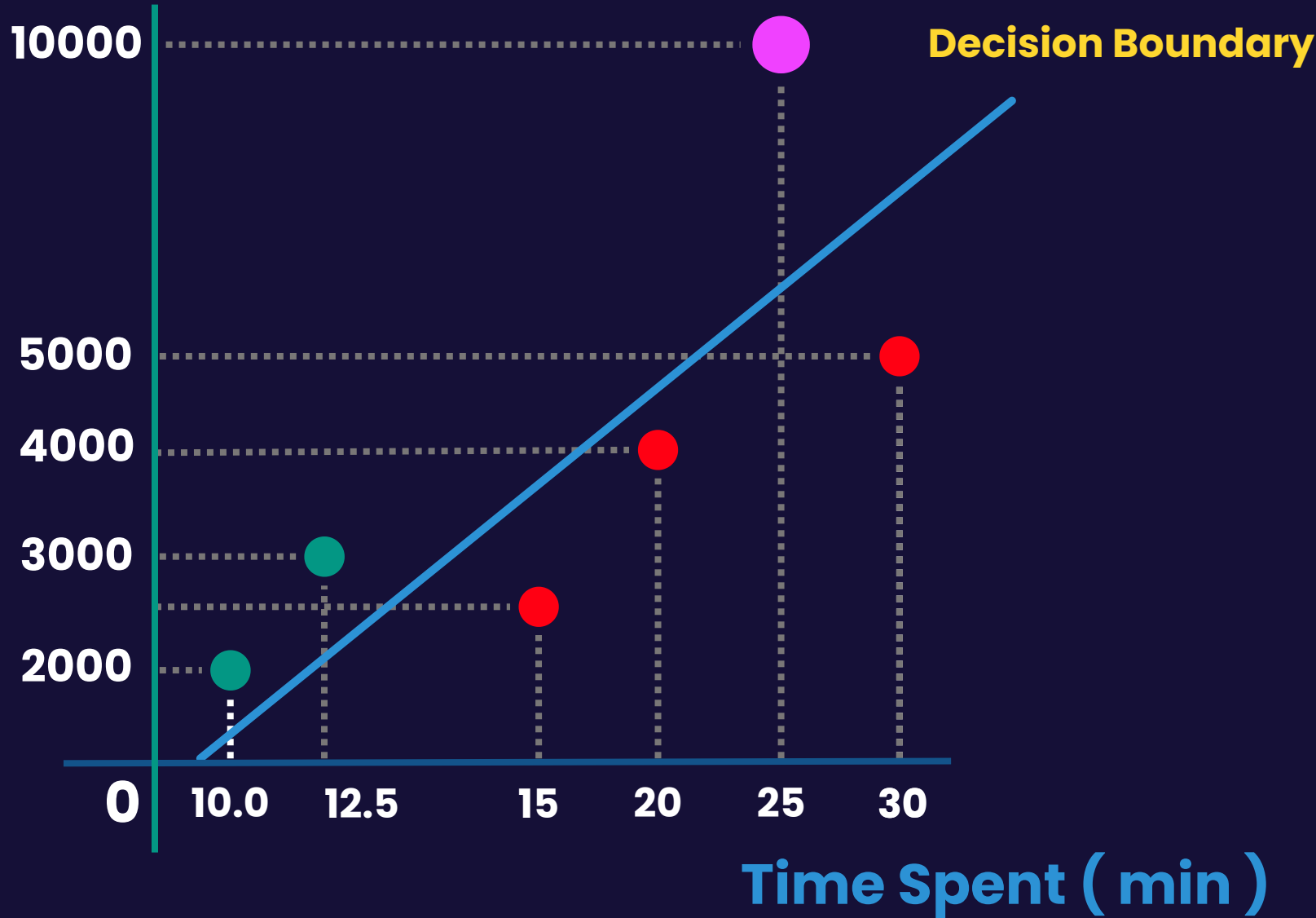
=> Làm sao để biết các điểm đã phân loại ?



Đường phân loại (Decision Boundary)

Monthly Income (USD)	Time Spent (min)	Nhóm khách hàng
2000	10	-1
4000	20	1
2500	15	-1
5000	30	1
3000	12	-1
10000	25	-1

Monthly income (USD)

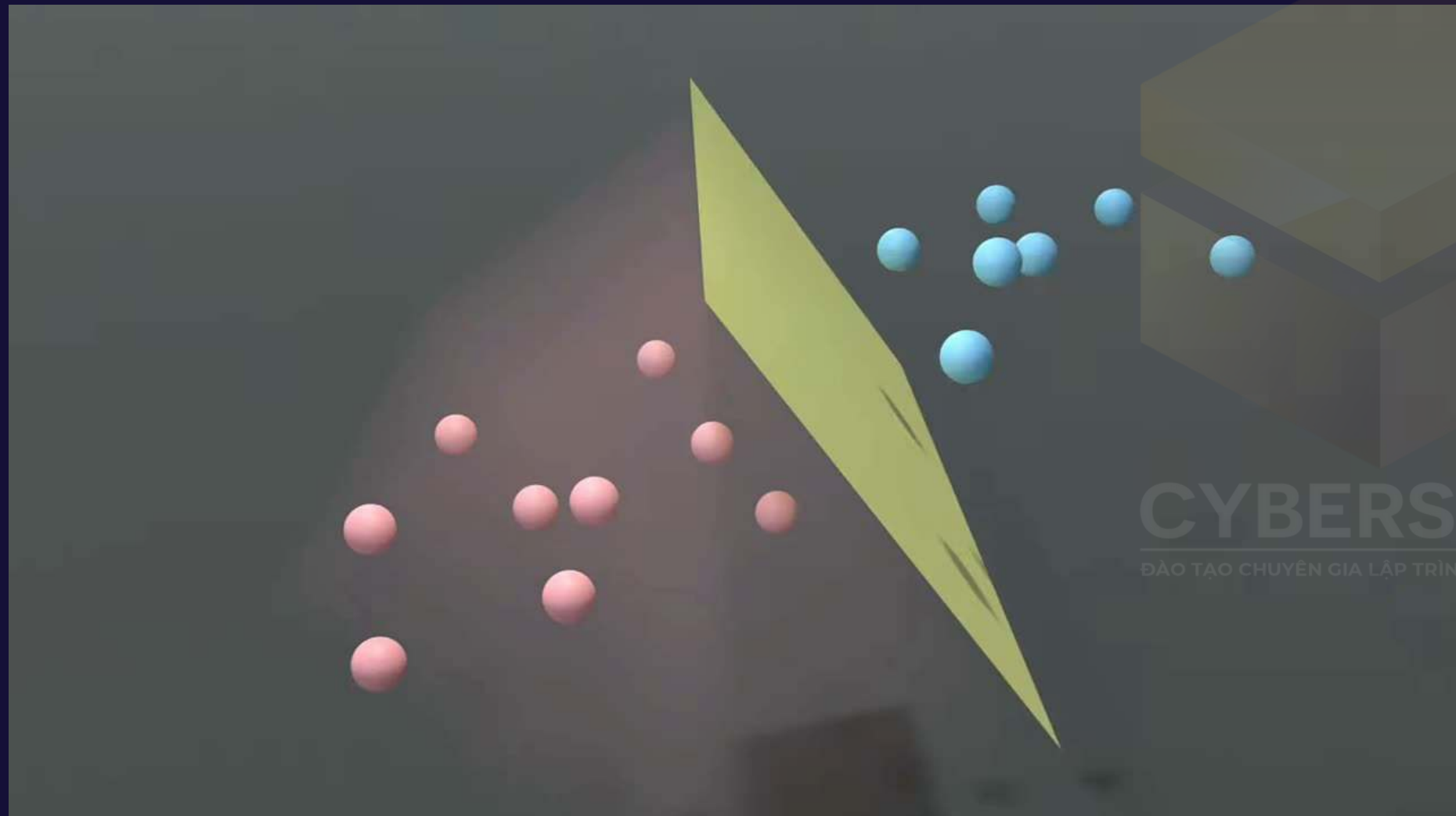


Decision Boundary:

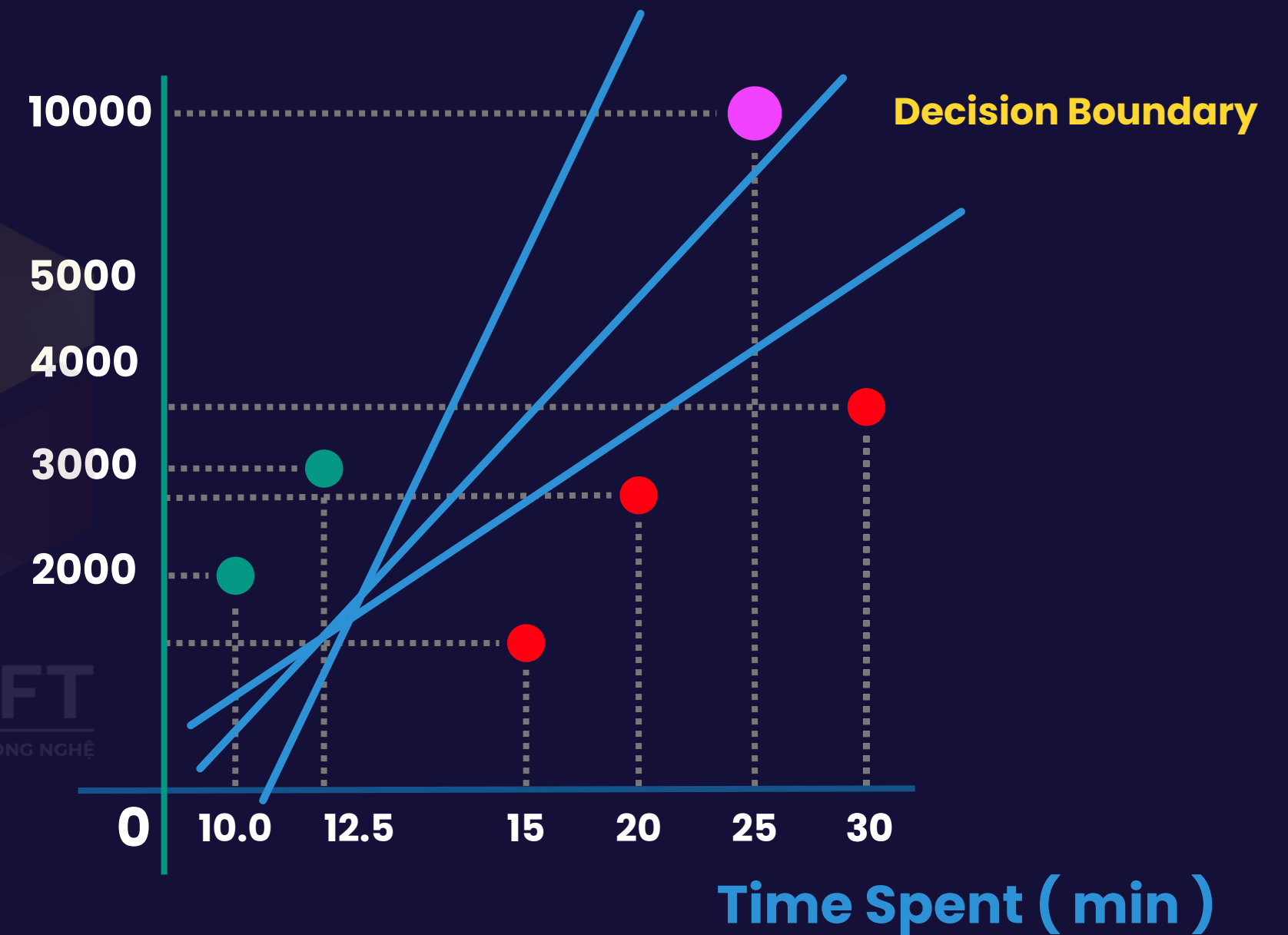
- Là siêu phẳng (đường thẳng trong không gian 2D) phân chia 2 lớp.
- Mục tiêu: chia dữ liệu thành hai nửa sao cho càng "rõ ràng" càng tốt.

Đường phân loại (Decision Boundary)

- Trong quá trình học, SVM luôn tìm đường phân loại một cách tốt nhất

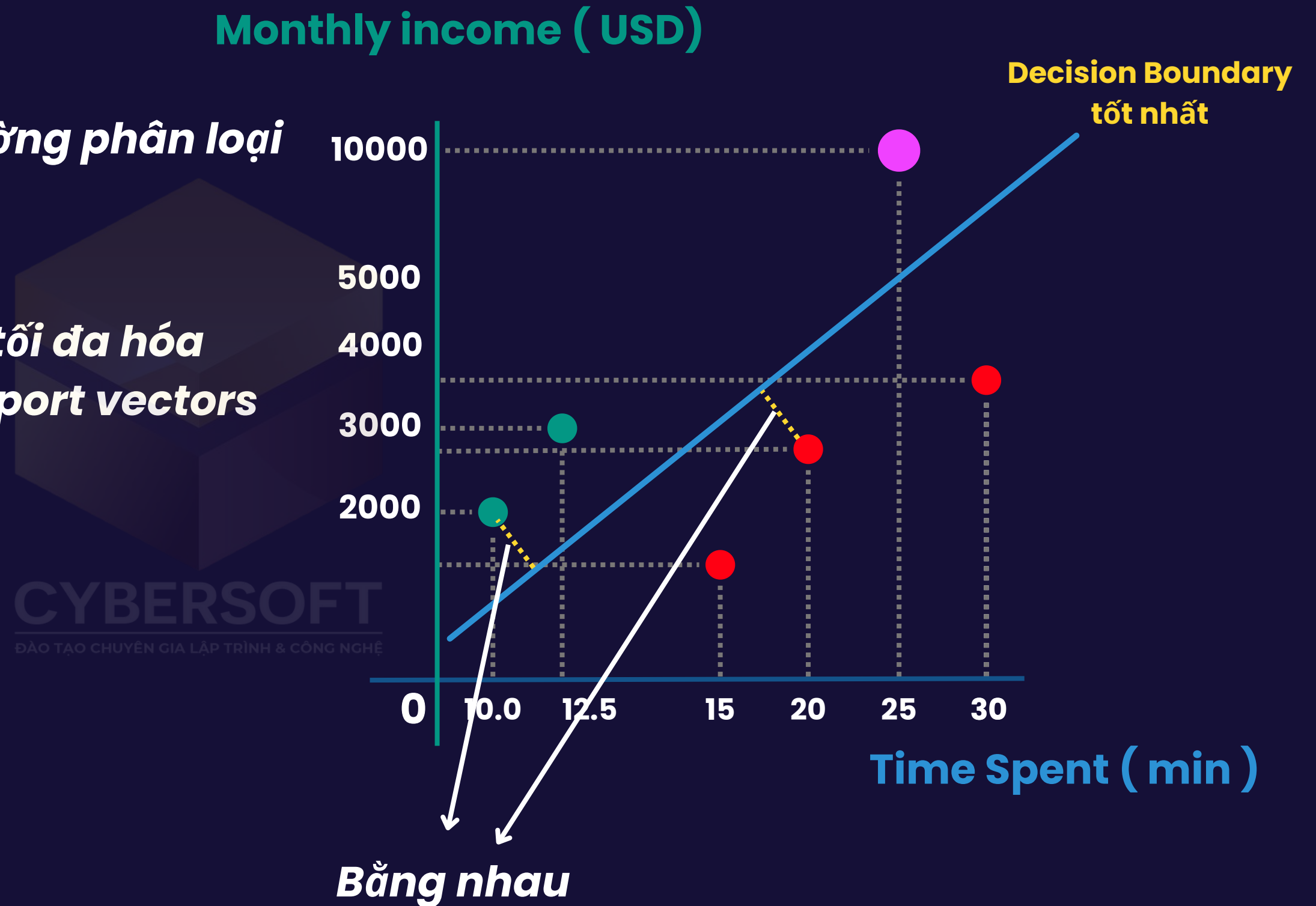


Monthly income (USD)



Đường phân loại (Decision Boundary)

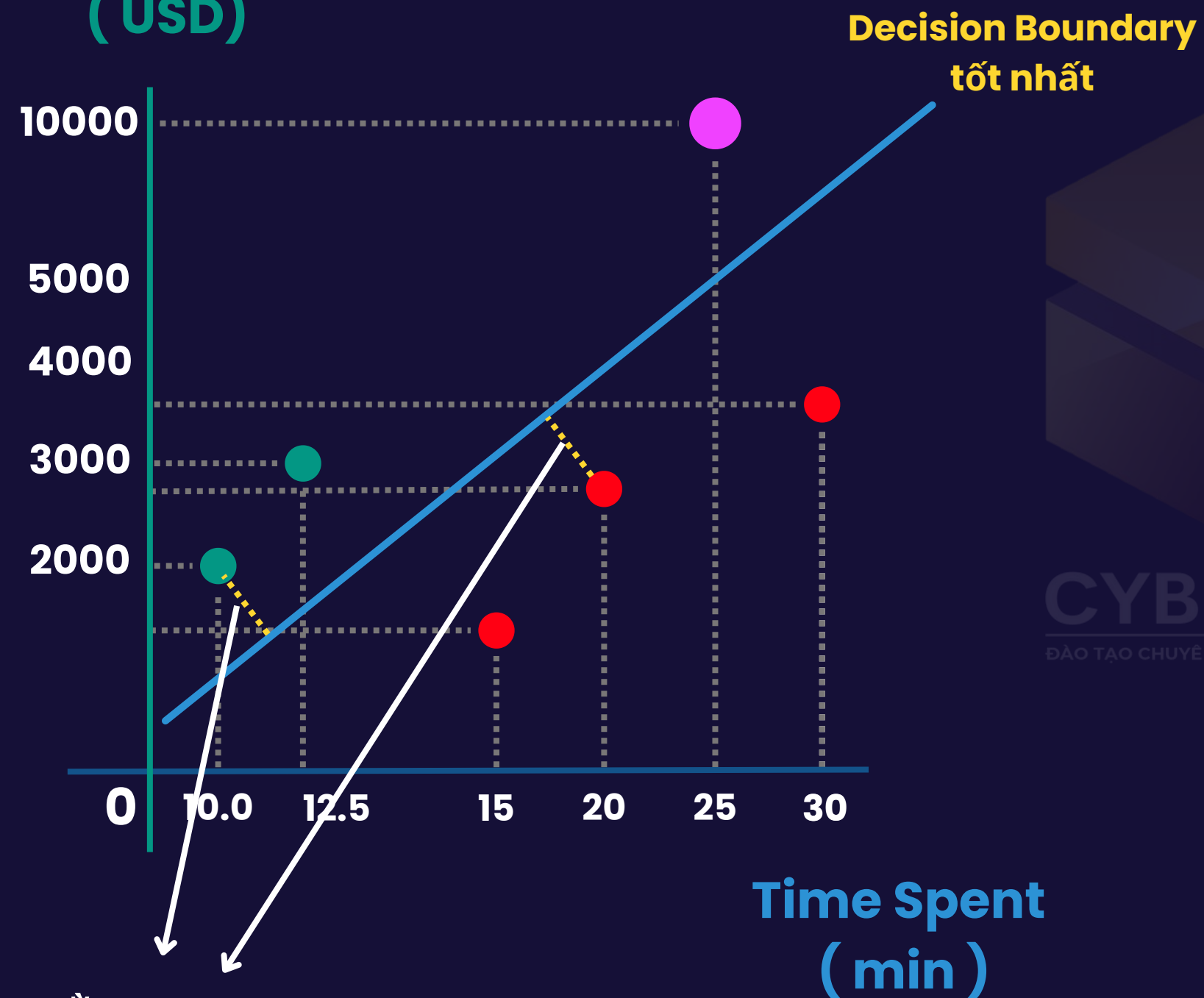
- Trong quá trình học, SVM luôn tìm đường phân loại một cách tốt nhất
- Và đường phân loại tốt nhất là đường tối đa hóa khoảng cách giữa nó và các điểm support vectors



2D

1D

Monthly income
(USD)



Bằng nhau

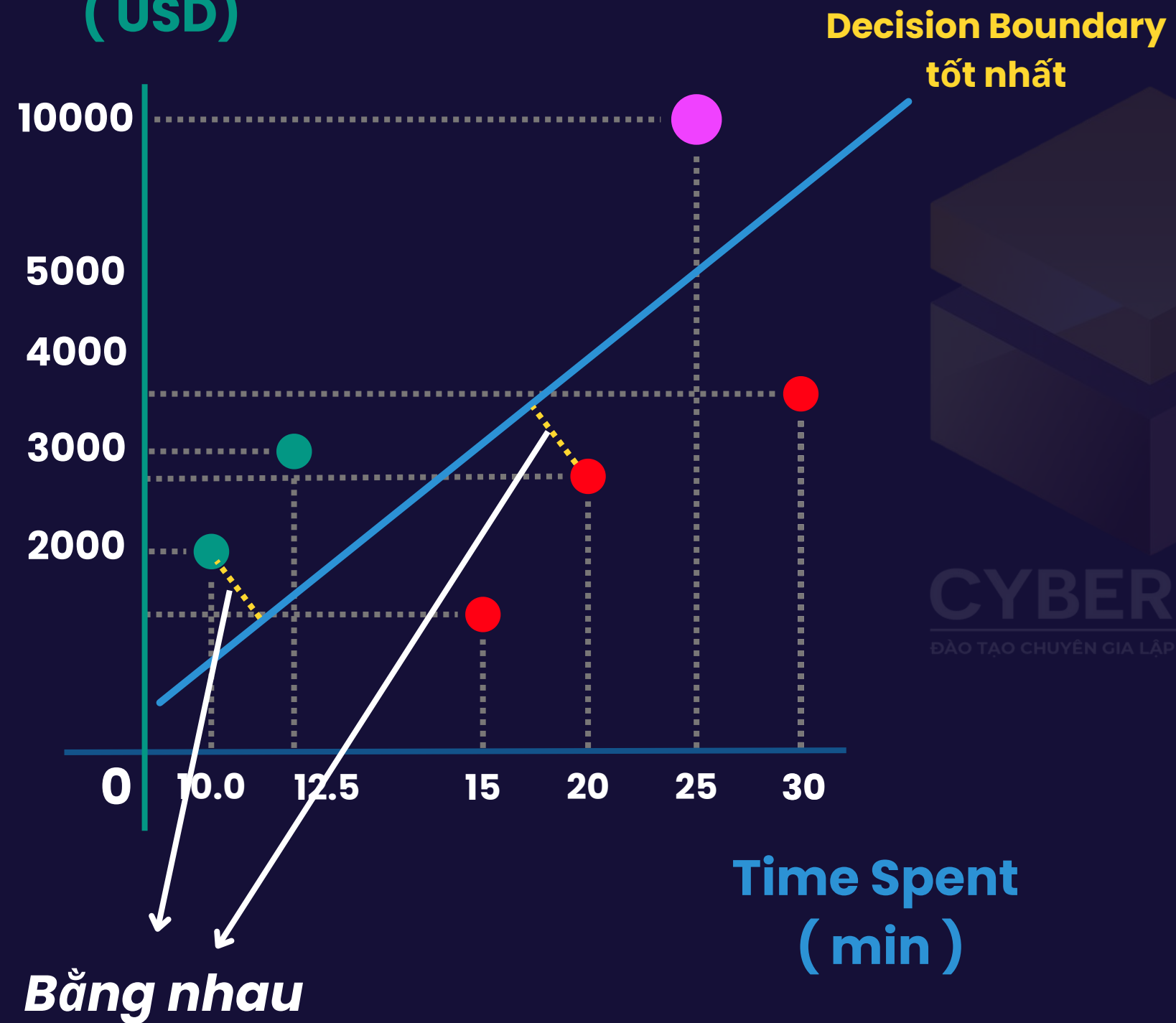
Decision Boundary

Bằng nhau

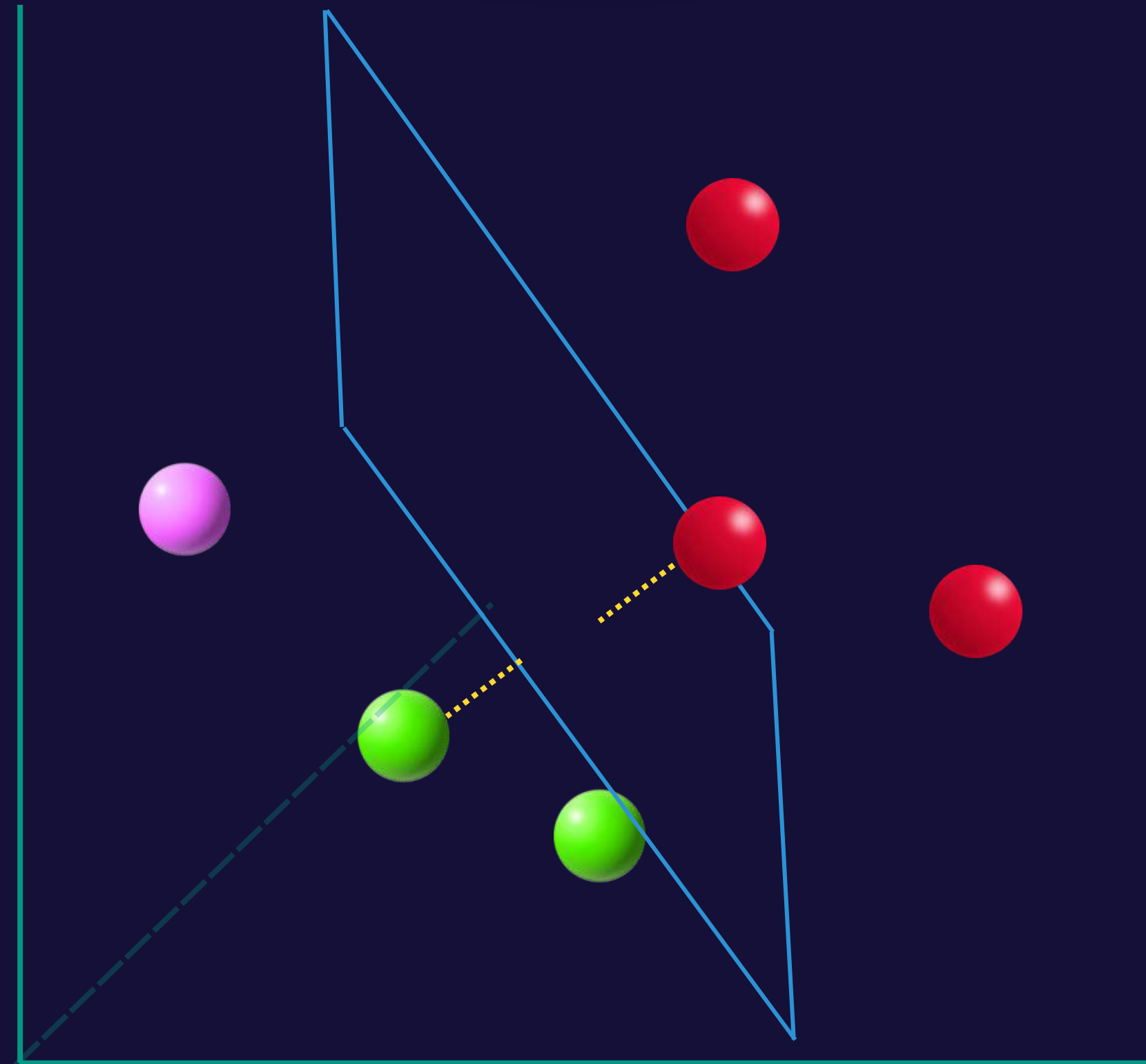
Support Vectors

2D

Monthly income
(USD)



3D





Starbucks ở Singapore. Muốn phân loại khách hàng thành hai nhóm:

- "Mua cà phê đắt" (>5 USD, nhãn +1)
- "Mua cà phê rẻ" (≤ 5 USD, nhãn -1)

=> Dựa trên Thời gian ở quán (Time Spent, phút) và Thu nhập hàng tháng (Monthly Income, USD). Dữ liệu từ 6 khách hàng:

Làm sao để mô hình biết chia dữ liệu ?



Đắt

Rẻ

Các chỉ số công thức

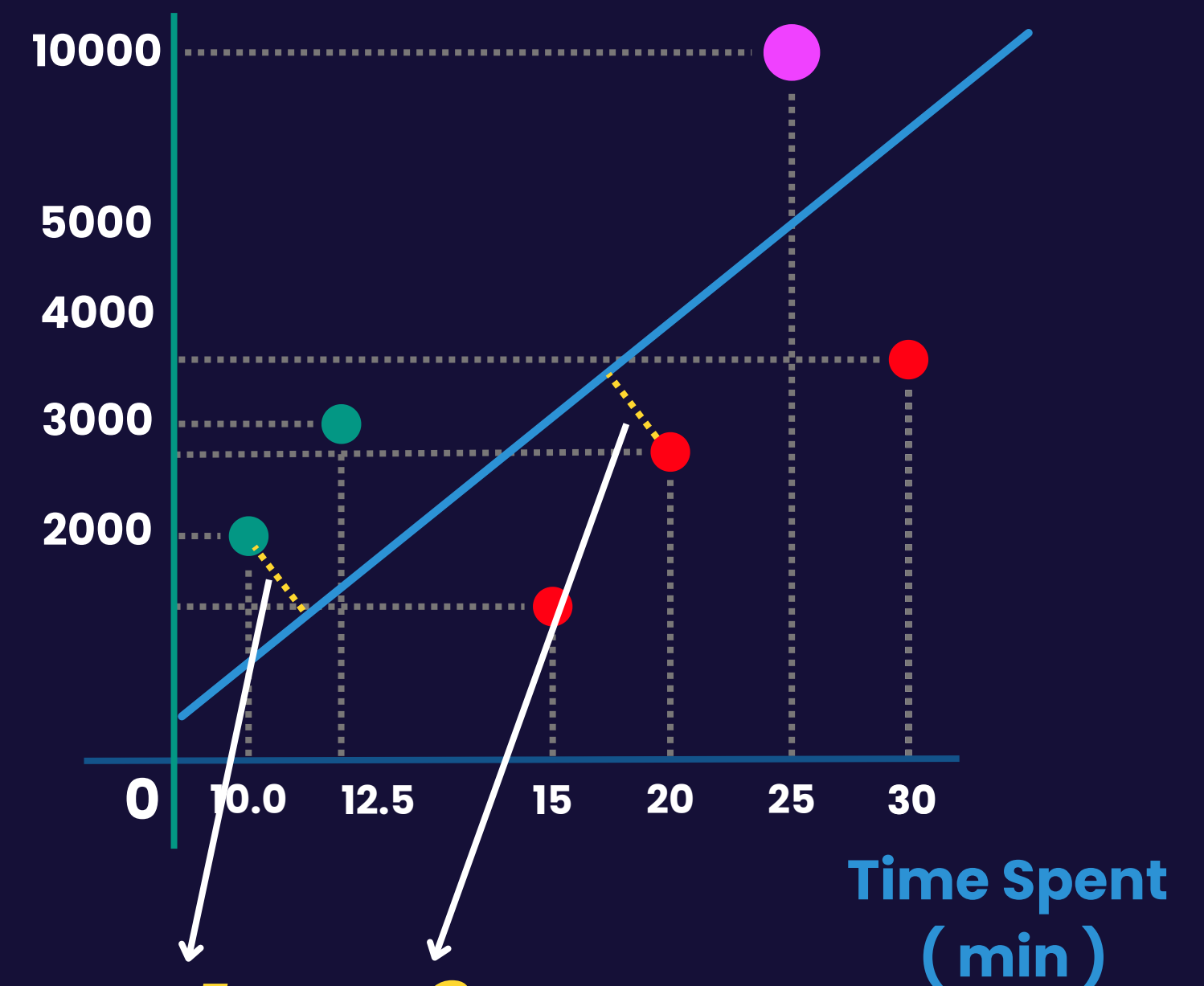
- Quyết định hướng nghiêng của đường phân cách hoặc còn gọi là **hyperplane**

$$w = [w1, w2]$$

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Monthly income
(USD)

Decision Boundary



$$w1 = w2$$

Các chỉ số công thức

$$w = [w1, w2]$$

$$Ys = label$$

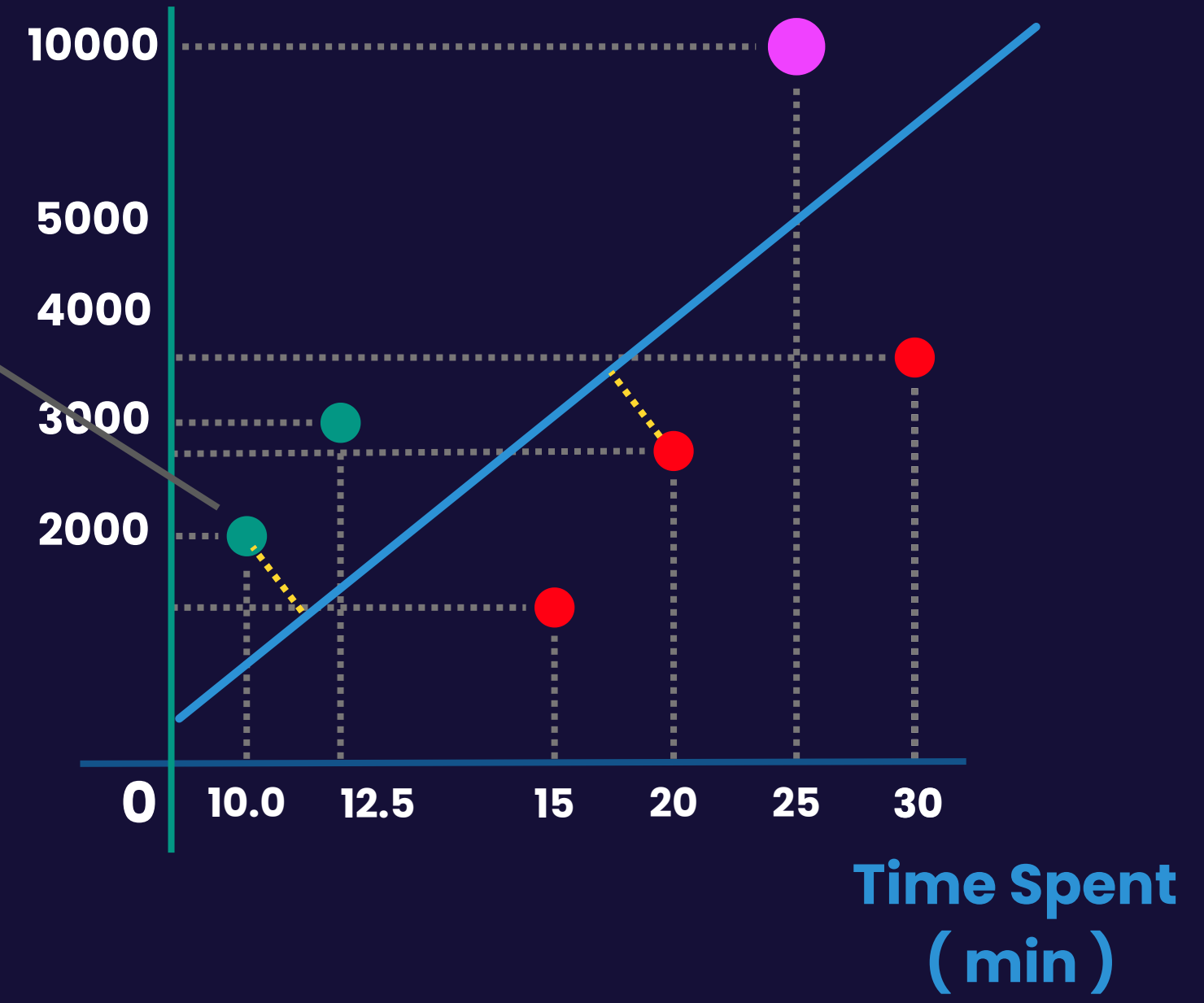
Support vector 1

$$Xs = [x, y]$$



Monthly income
(USD)

Decision Boundary



Các chỉ số công thức

$$\mathbf{w} = [w_1, w_2]$$

$$Y_s = \text{label}$$

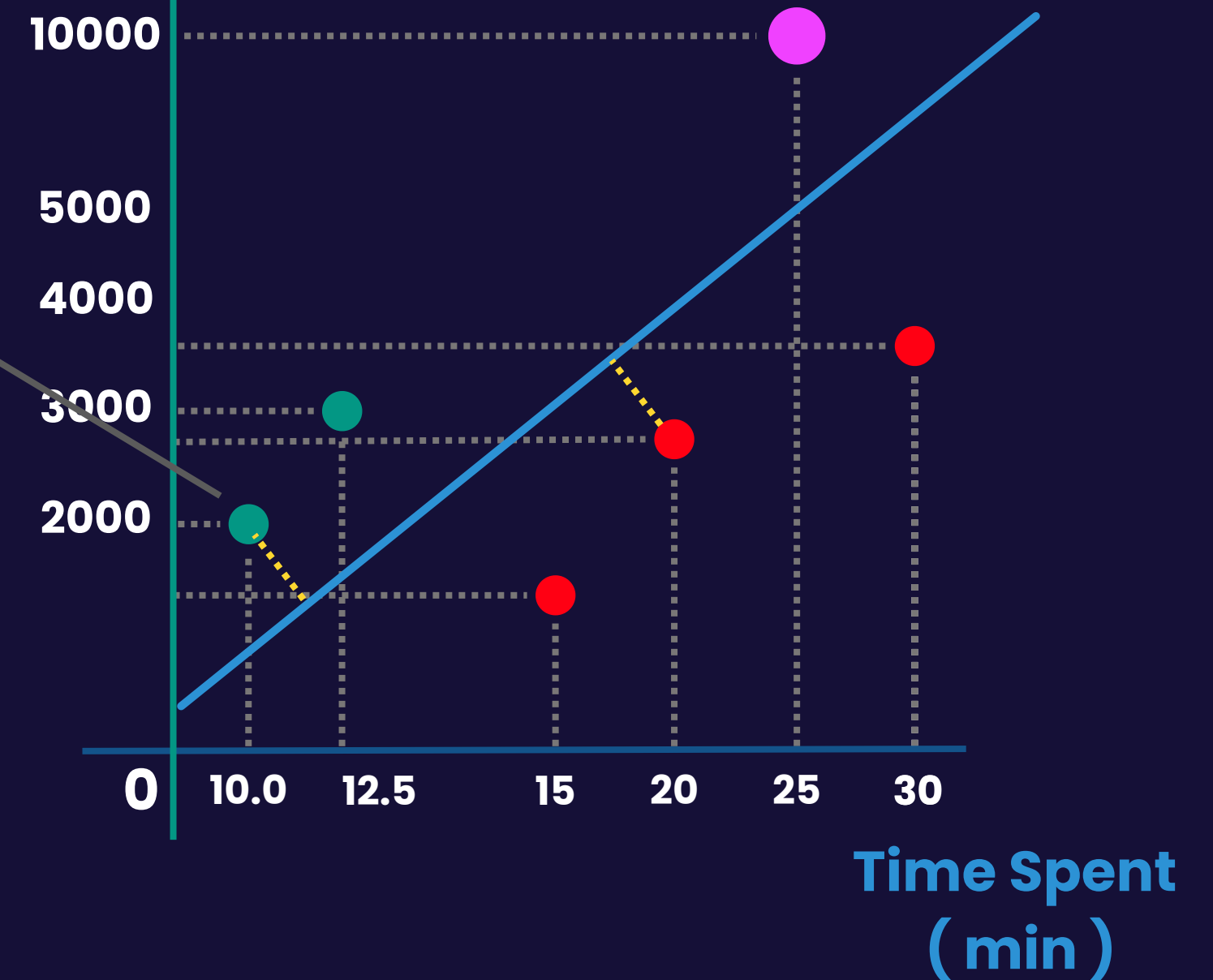
Support vector 1

$$X_s = [x, y]$$

$$b_1 = Y_s - \mathbf{w} * X_s$$

Monthly income
(USD)

Decision Boundary



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Các chỉ số công thức

$$\mathbf{w} = [\mathbf{w1}, \mathbf{w2}]$$

$$\mathbf{Ys} = \text{label}$$

Support vector 1

$$\mathbf{Xs} = [\mathbf{x}, \mathbf{y}]$$

$$\mathbf{b1} = \mathbf{Ys} - \mathbf{w} * \mathbf{Xs}$$

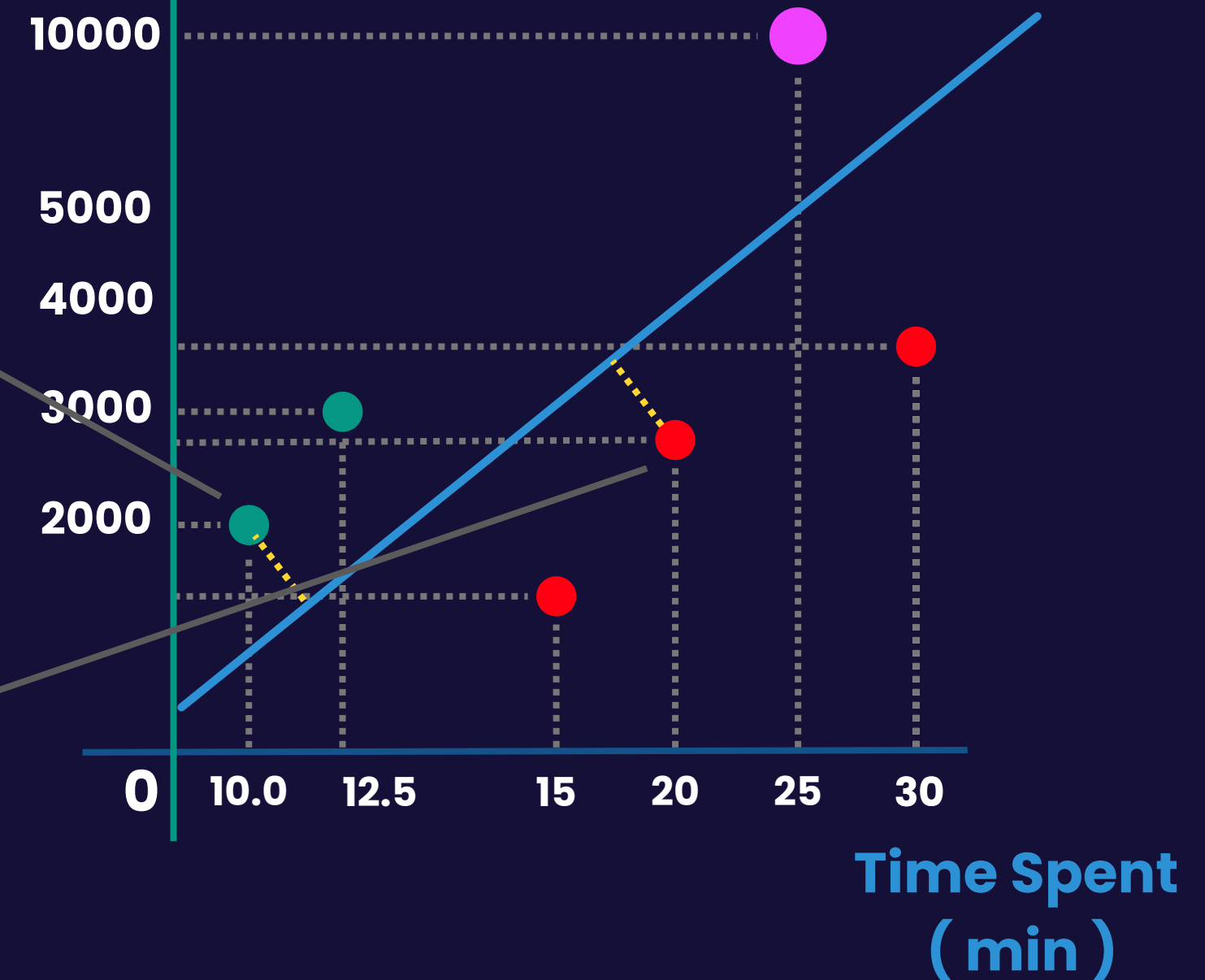
Support vector 2

$$\mathbf{b2} = \mathbf{Ys} - \mathbf{w} * \mathbf{Xs}$$

$$\mathbf{b} (\text{bias}) = \frac{\mathbf{b1} + \mathbf{b2}}{2}$$

Monthly income
(USD)

Decision Boundary



Công thức tổng quát

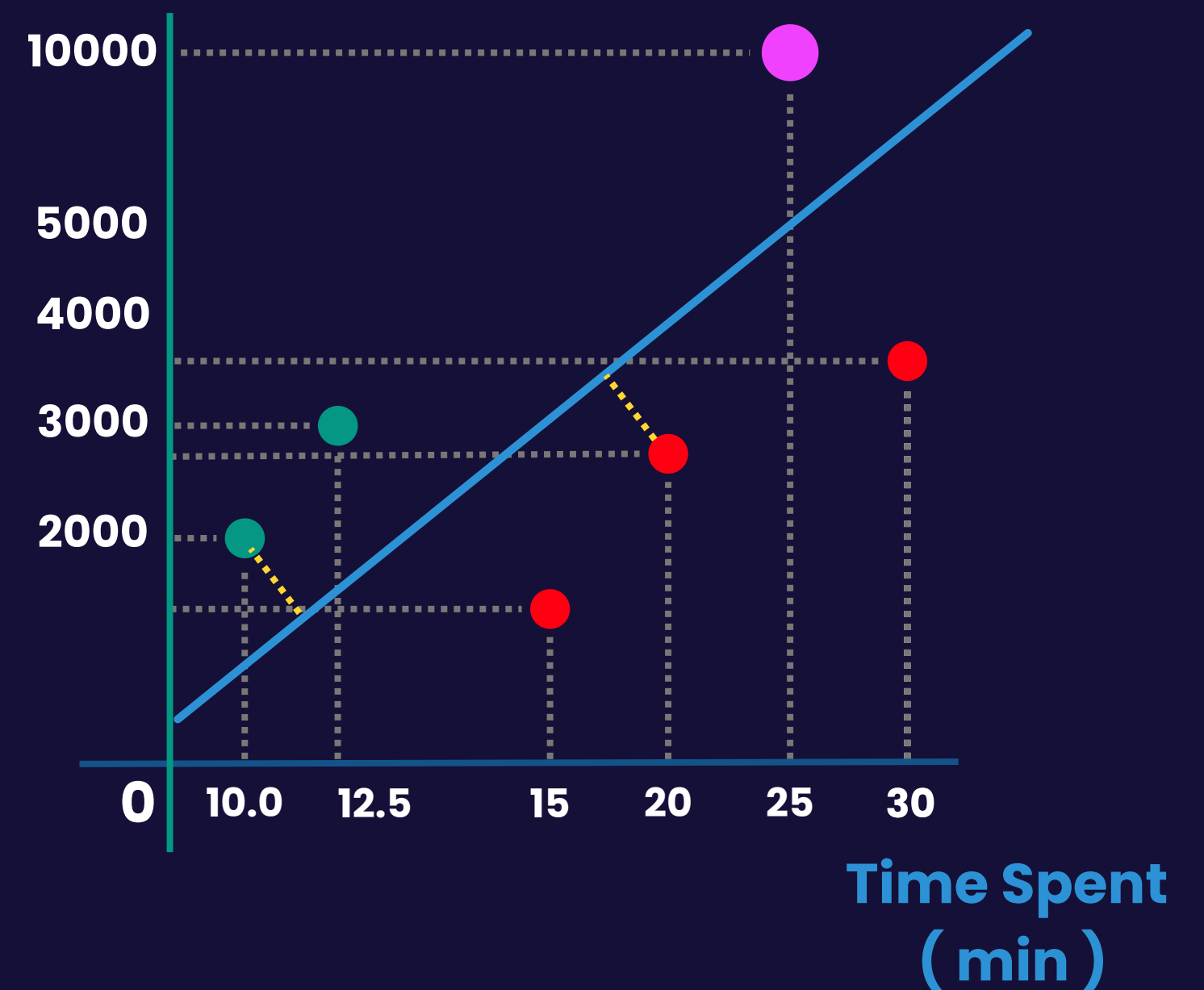
➤ Tìm đường **Decision Boundary**

$$w1 * x1 + w2 * x2 + b = 0$$

➤ Khoảng cách tới đường phân loại giữa 2 support vectors là bằng nhau và gần nhất

Monthly income
(USD)

Decision Boundary



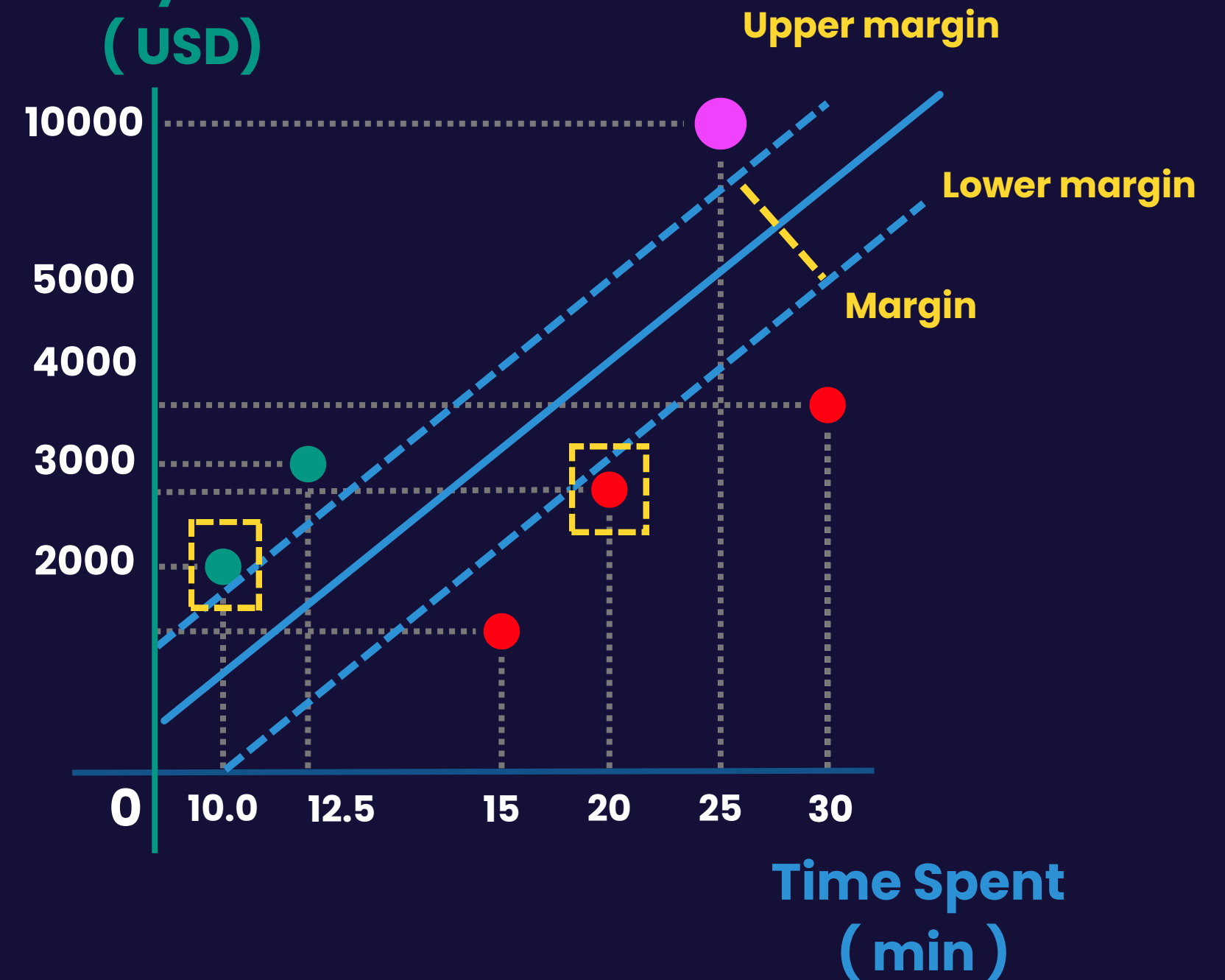
Margin

Đường margin là hai đường (hoặc mặt phẳng) song song với **hyperplane**, đi qua các support vectors.

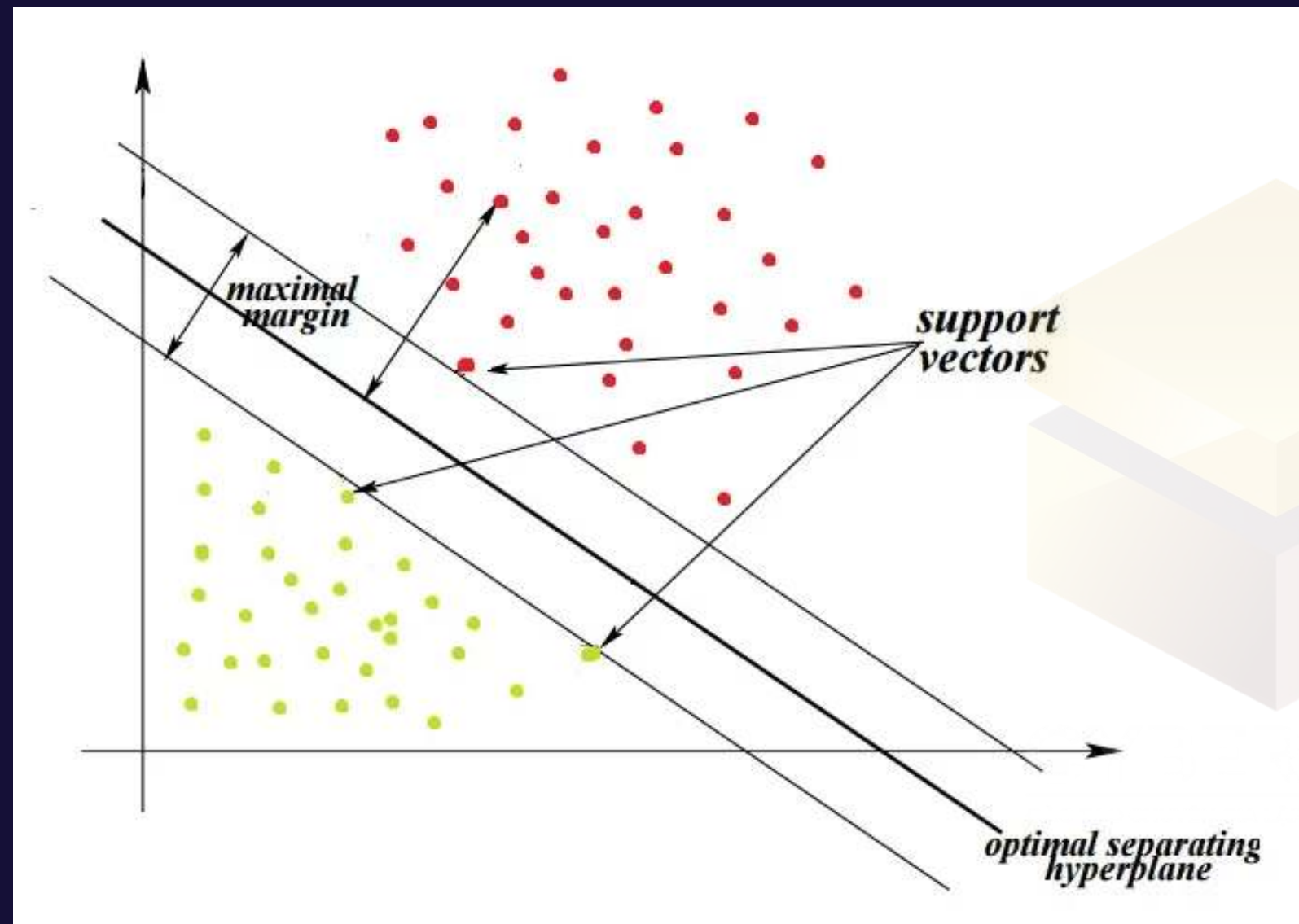
➤ Upper margin: $W * X + B = 1$

➤ Lower margin: $W * X + B = -1$

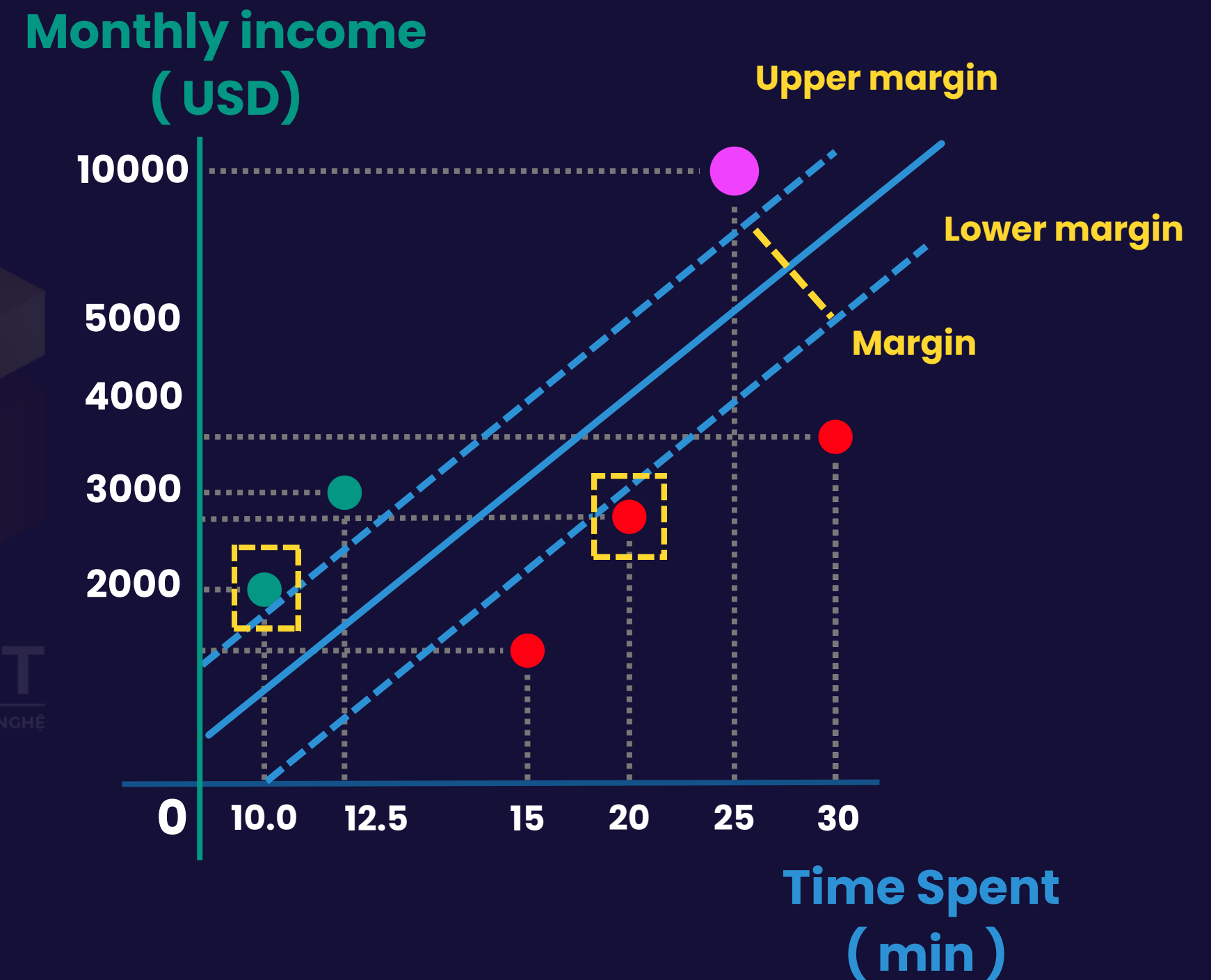
Monthly income
(USD)



Margin & hyperplane

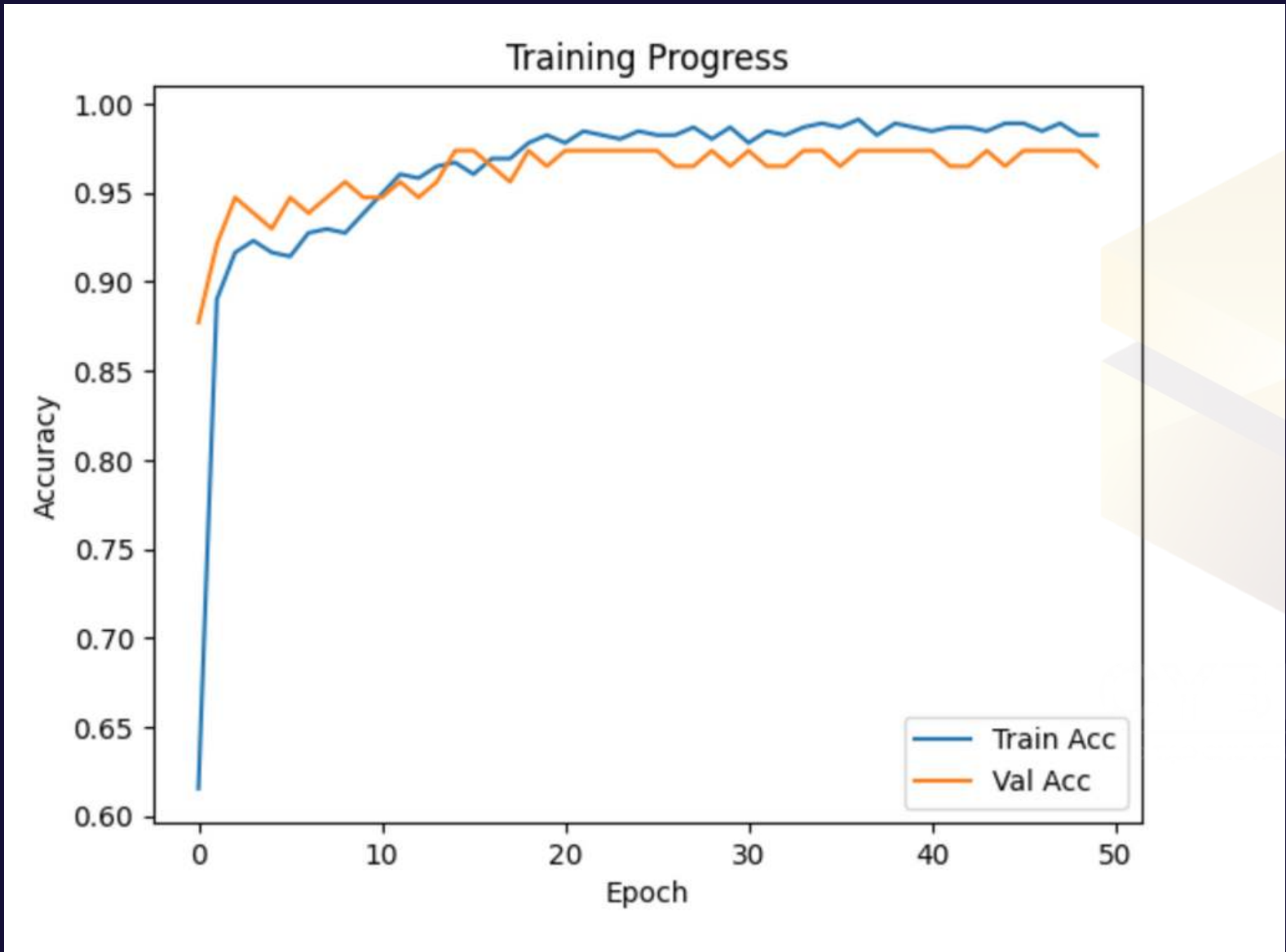


SVM tìm một siêu phẳng để tách biệt hai lớp dữ liệu sao cho khoảng cách (margin) giữa các điểm gần nhất và siêu phẳng là lớn nhất.



Accuracy

Accuracy là một chỉ số đánh giá hiệu suất mô hình phân loại, cho biết tỷ lệ dự đoán đúng trên tổng số dự đoán.

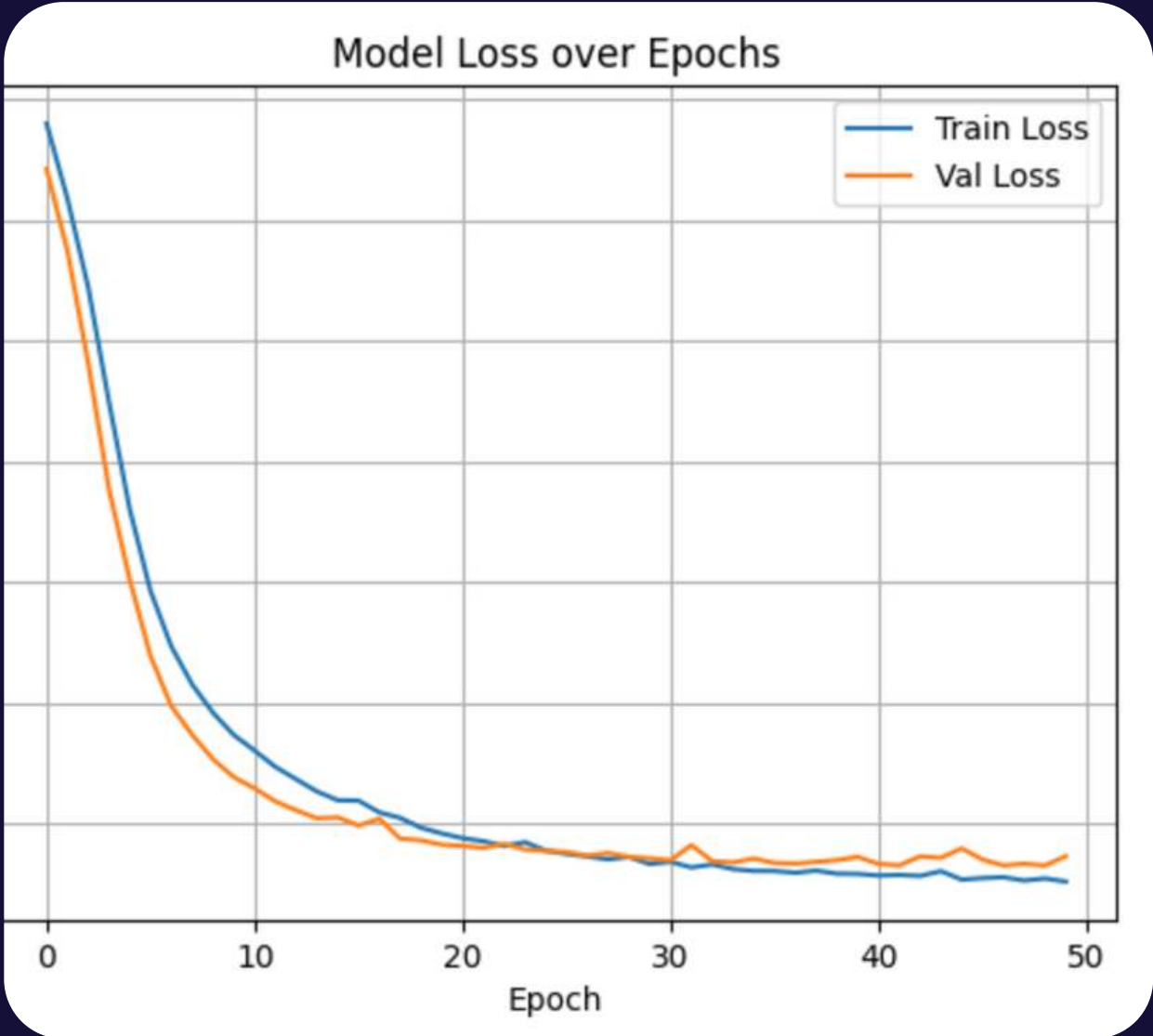


Số dự đoán đúng	Số lượng mẫu mà mô hình dự đoán chính xác nhân so với nhãn thật
Tổng số mẫu	Tổng số dữ liệu trong tập huấn luyện (training set)
Kết quả	Là tỉ lệ phần trăm (%) thể hiện mức độ chính xác của mô hình trên dữ liệu đã học

$$\text{Train Accuracy} = \frac{\text{Số dự đoán đúng trên tập huấn luyện}}{\text{Tổng số mẫu trong tập huấn luyện}} \times 100\%$$

LOSS VALUE

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$



Train Loss

Mức sai số (error) mô hình tạo ra trên tập huấn luyện, Đo mức độ “học được gì” từ dữ liệu đã thấy

Val Loss

Mức sai số mô hình tạo ra trên tập validation, đánh giá khả năng tổng quát hóa trên dữ liệu mới

Thành phần	Ý nghĩa
N	Số lượng mẫu trong tập train hoặc validation
y_i	Nhãn thật của mẫu thứ ii (0 hoặc 1)
$\hat{y}^i$	Xác suất mô hình dự đoán là “có ung thư” (giá trị từ 0 → 1)
log	Logarithm tự nhiên (base e), giúp “phạt nặng” nếu dự đoán sai nghiêm trọng

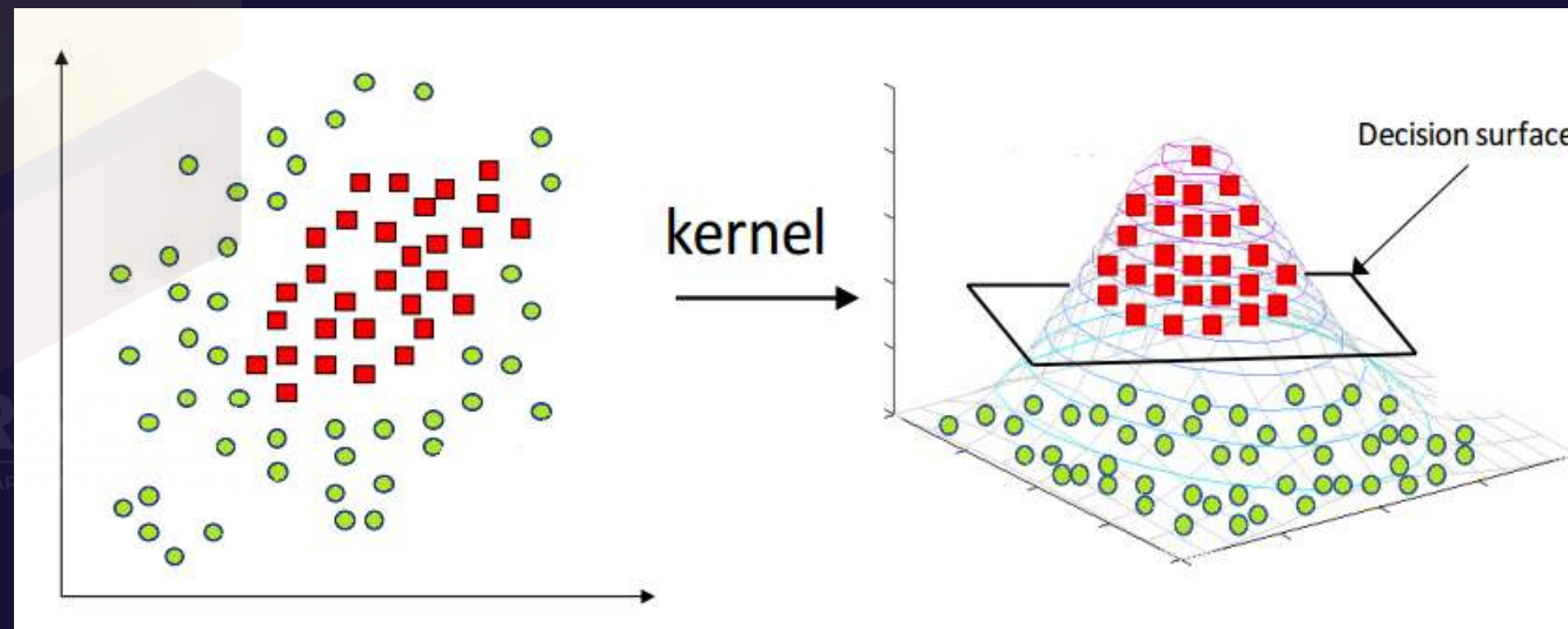
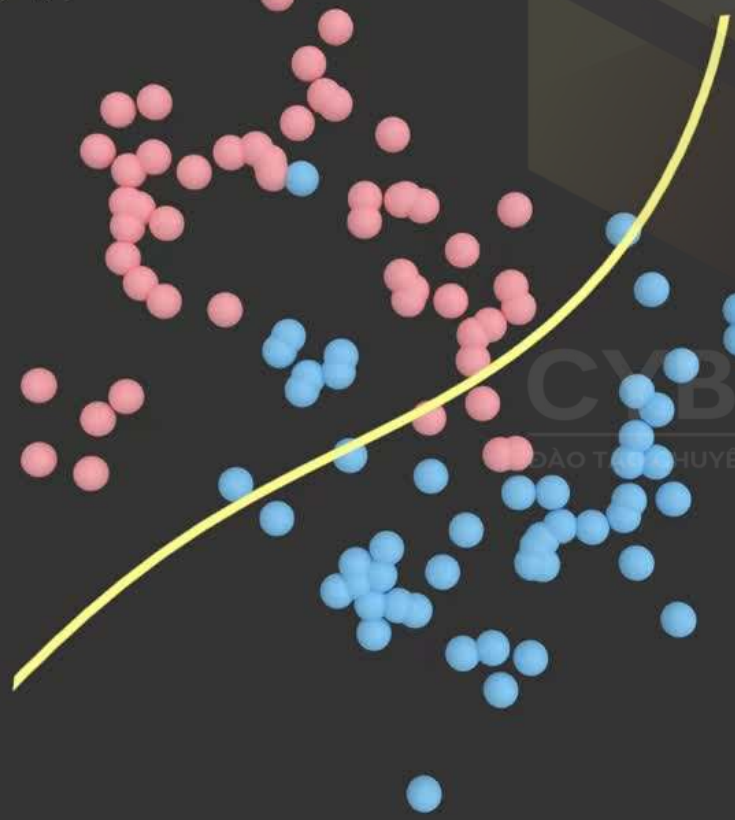
KERNEL Trick

Dữ liệu tuyến tính

Nhiều mô hình (ví dụ SVM) hoạt động tốt với dữ liệu tuyến tính. Nhưng với dữ liệu phi tuyến, ta cần ánh xạ dữ liệu từ không gian gốc sang không gian đặc trưng (feature space) có số chiều cao hơn, nơi dữ liệu có thể phân tách tuyến tính.

```
SVC(kernel='rbf', gamma=0.01).fit(X, y)
```

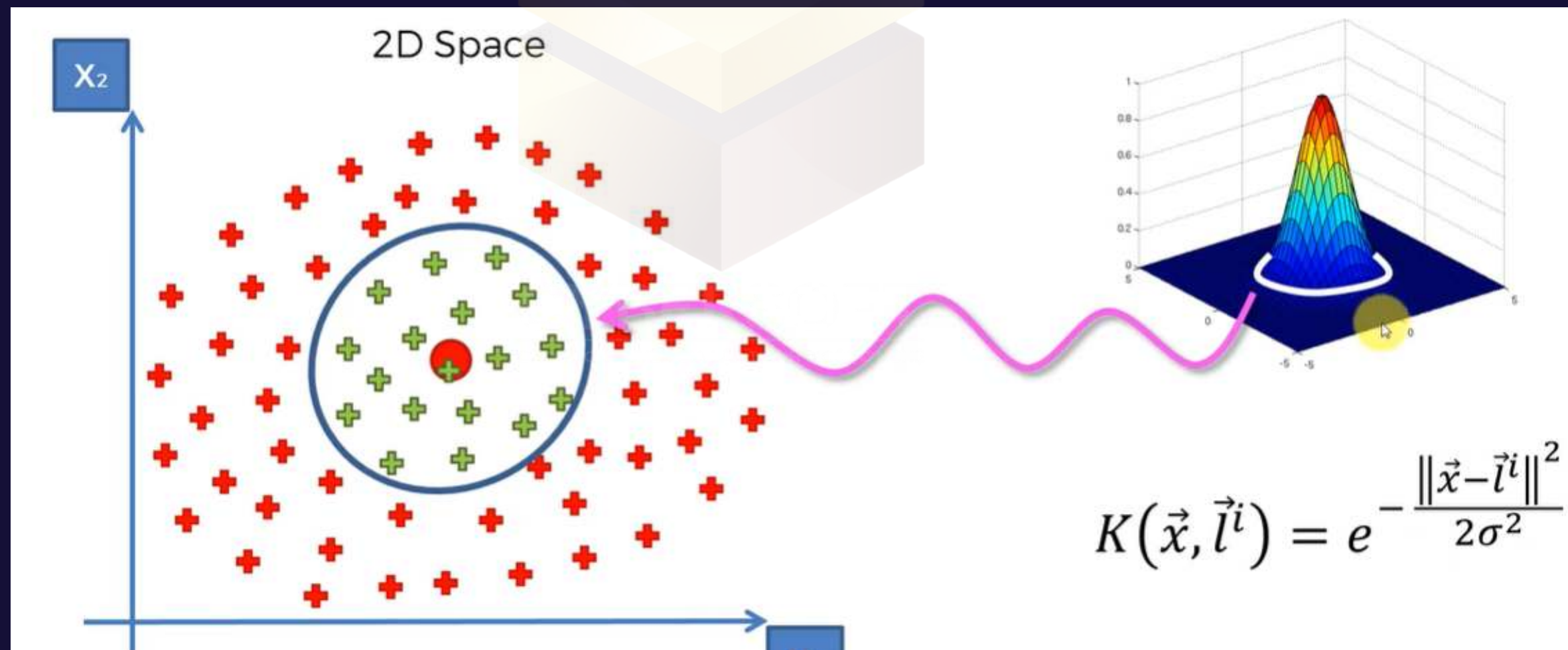
γ
0.01



KERNEL Trick

RBF Kernel (Radial Basis Function)

RBF Kernel giúp ánh xạ dữ liệu vào không gian đặc trưng vô hạn chiều mà không cần tính toán trực tiếp thông qua kernel trick.



KERNEL Trick

Công thức cần biết (Dữ liệu tuyến tính)

Hai vector đặc trưng

$$K(x, x') = x^T x' = \sum_{i=1}^n x_i x'_i$$

The diagram illustrates the Kernel Trick formula. It shows the kernel function $K(x, x')$ is equal to the dot product $x^T x'$, which is also equal to the sum of the products of corresponding elements $x_i x'_i$ from $i=1$ to n . The terms (x, x') and $x^T x'$ are highlighted with red boxes. An arrow points from the text "Hai vector đặc trưng" (Two characteristic vectors) to the first box, and another arrow points from the text "Tích vô hướng (dot product) giữa 2 vector" (Dot product between 2 vectors) to the second box.

Tích vô hướng (dot product) giữa 2 vector

KERNEL Trick

Công thức cần biết RBF Kernel

Hai vector đặc trưng


$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

KERNEL Trick

Output của từng kernel

- Là một con số: càng lớn $\rightarrow$ 2 điểm càng giống nhau trong không gian đặc trưng.
- Nhưng ý nghĩa số đó khác nhau với từng kernel, vì mỗi kernel ánh xạ theo cách khác nhau.

Linear

Output là Tích vô hướng $x^T * x'$

$\Rightarrow$ Chỉ so sánh góc giữa vector, tuyến tính

RBF

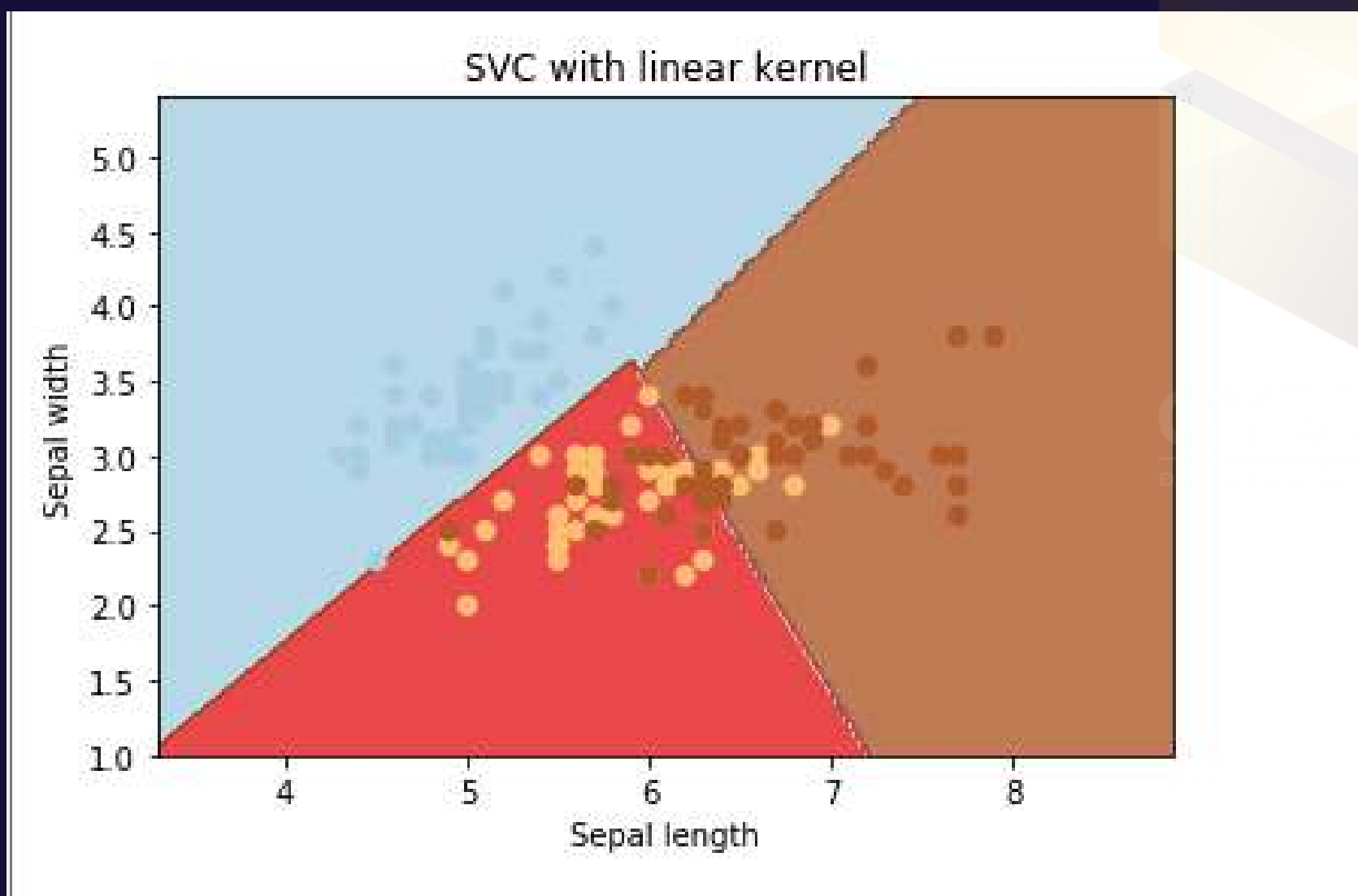
Output là $\exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$

$\Rightarrow$ So sánh khoảng cách $\rightarrow$ càng gần, giá trị càng gần 1

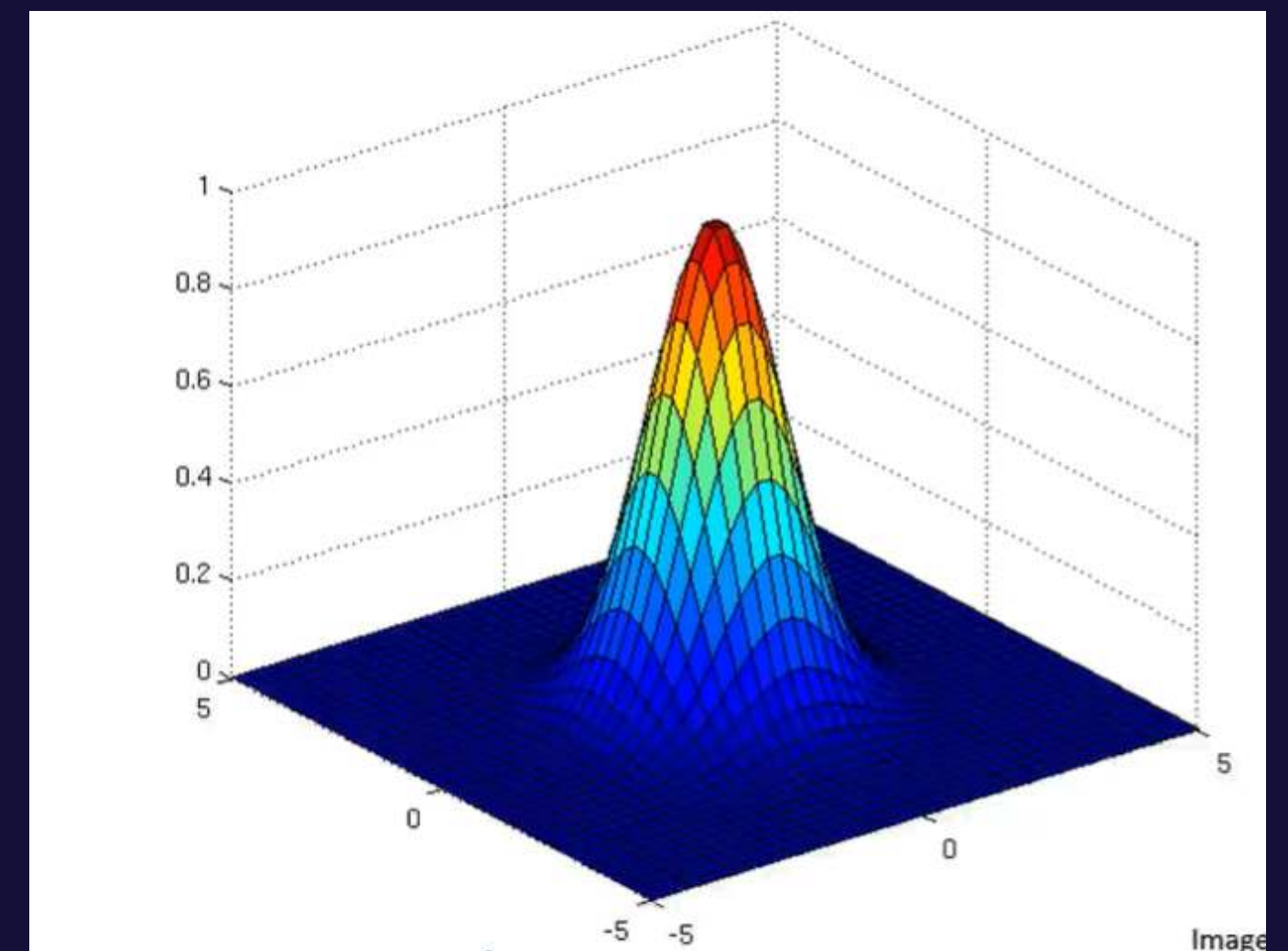
KERNEL Trick

Vì sao output khác nhau?

- Linear đo bằng góc giữa 2 vector
- Linear: không ánh xạ gì → dùng trực tiếp

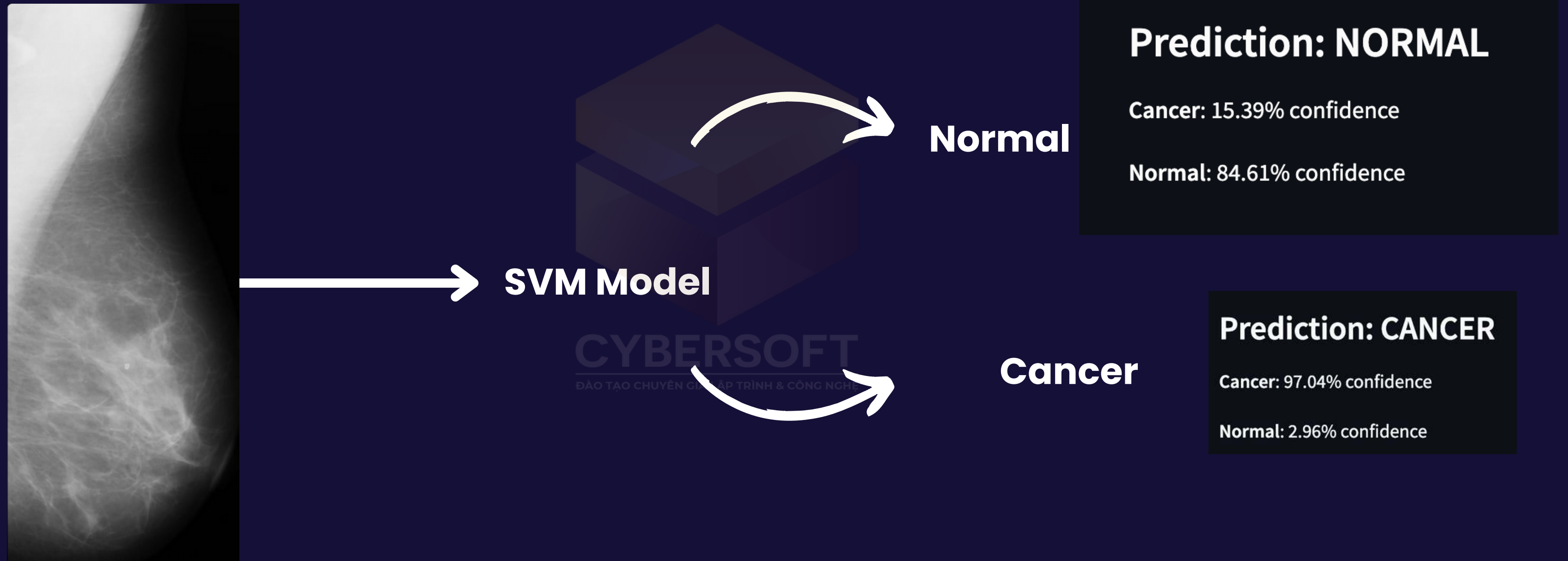


- RBF đo bằng khoảng cách, biến nó thành giá trị gần 1 nếu gần, gần 0 nếu xa.
- RBF: ánh xạ sang vô hạn chiều → có khả năng phân tách rất mạnh.



Bài tập thực hành

Breast Cancer Detector

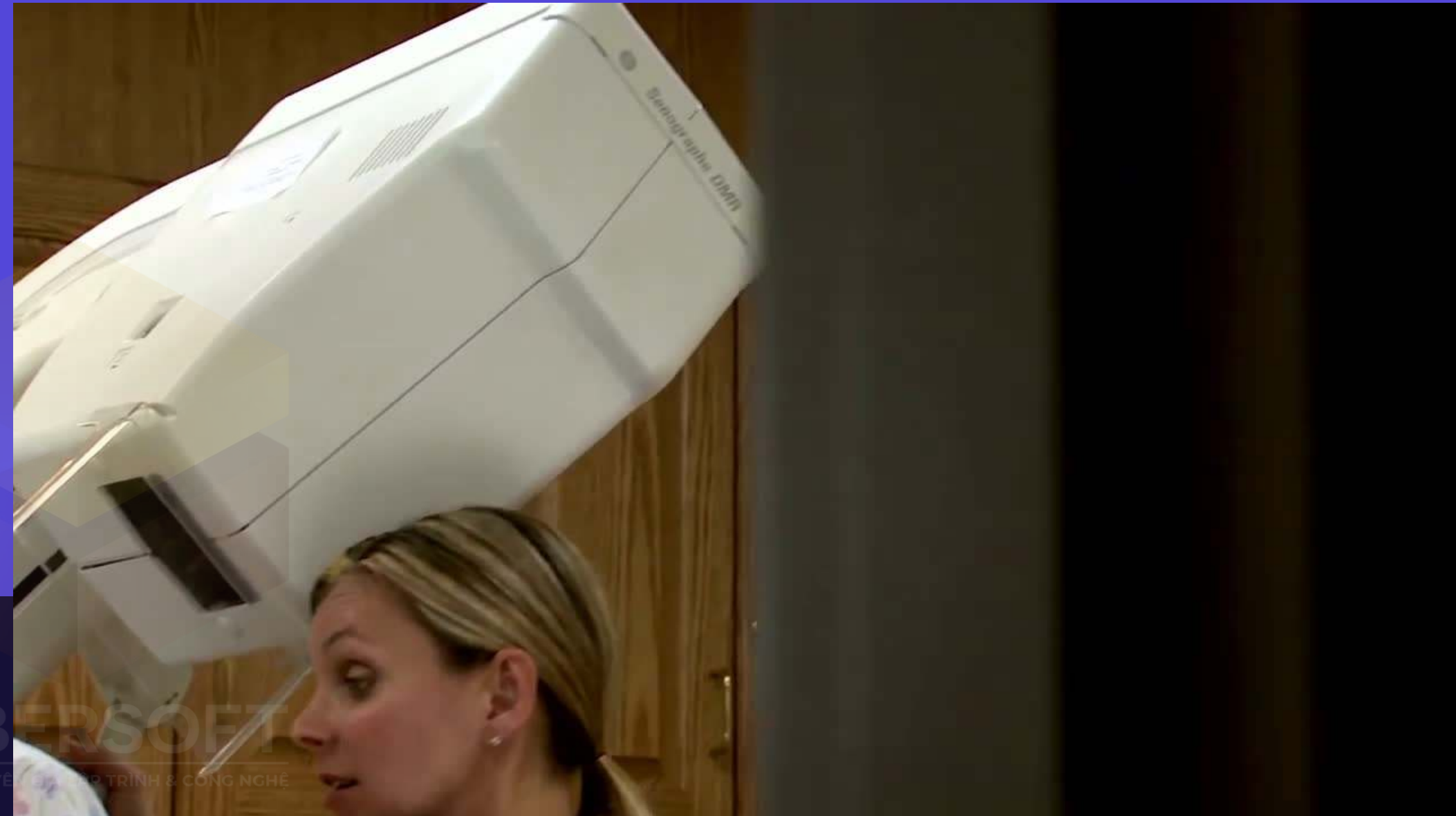


Case Study

Breast Cancer Detector

Vấn đề xã hội:

- Ung thư vú là loại ung thư phổ biến nhất ở phụ nữ trên toàn thế giới.
- Phát hiện sớm có thể giảm tỷ lệ tử vong đến 90%, nhưng điều kiện chẩn đoán sớm không đồng đều giữa các khu vực.
- Nhu cầu ứng dụng AI để hỗ trợ bác sĩ phân loại khối u từ ảnh hoặc dữ liệu lâm sàng ngày càng lớn.

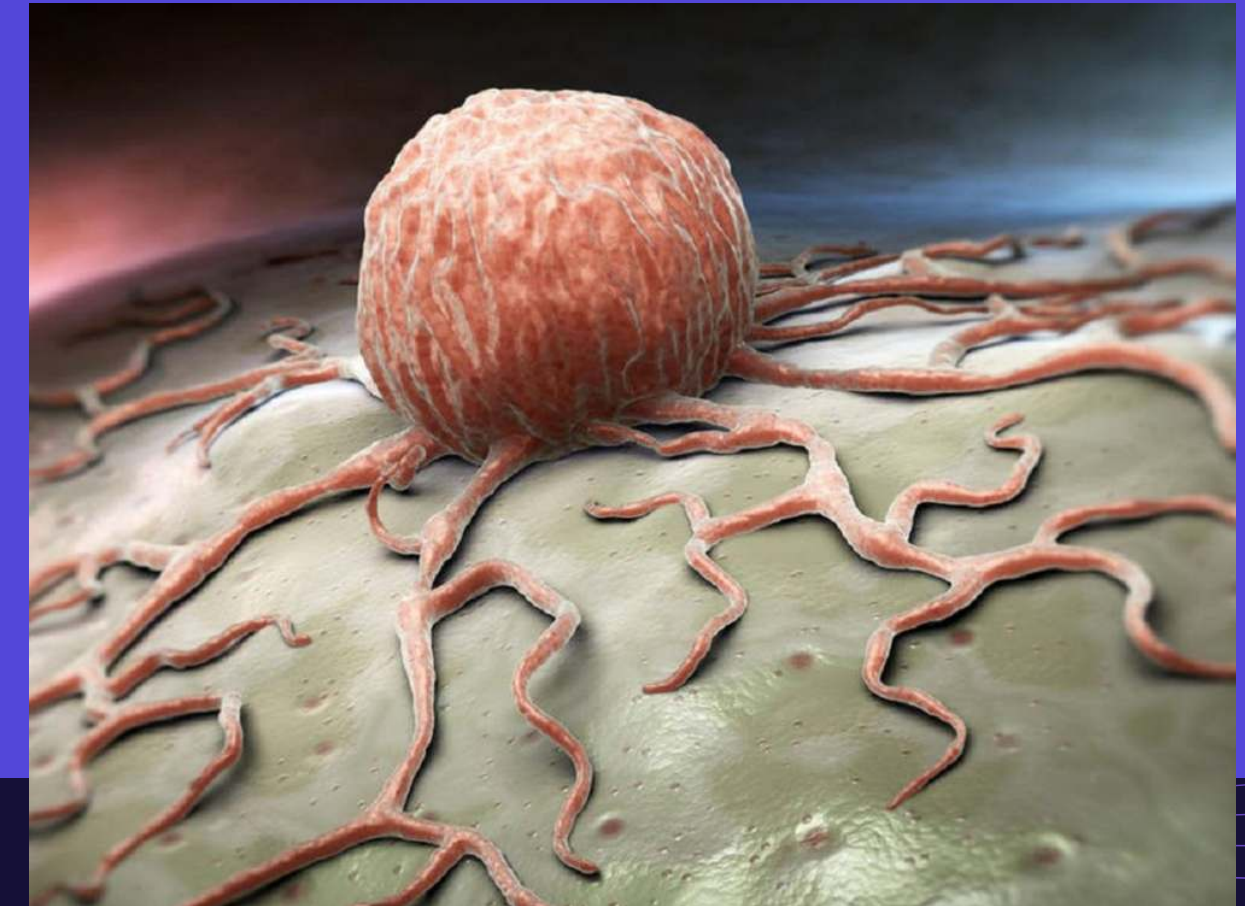
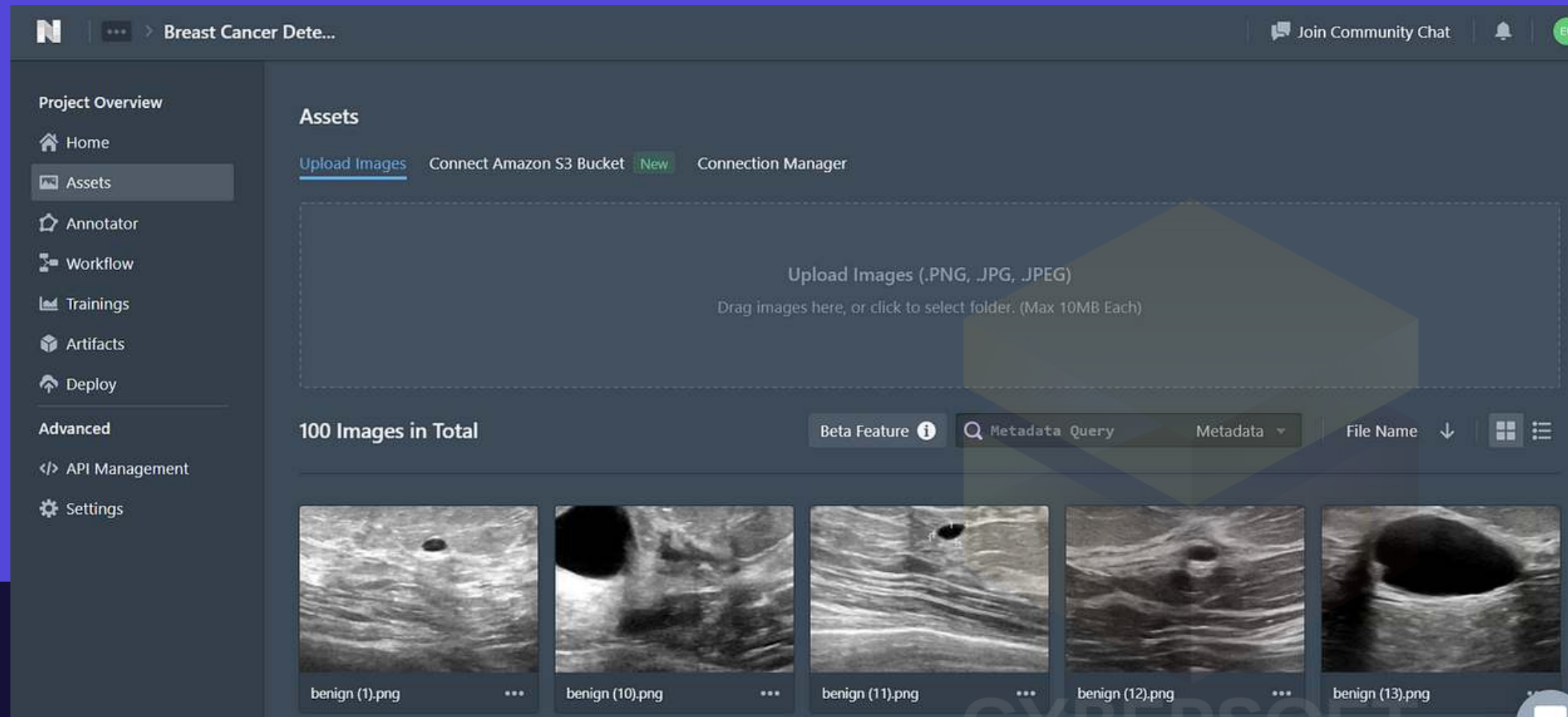


=> AI sẽ làm trợ lý chẩn đoán thông minh, giúp tăng tốc độ, độ chính xác, và phổ cập y tế chất lượng cao đến mọi nơi.

Case Study

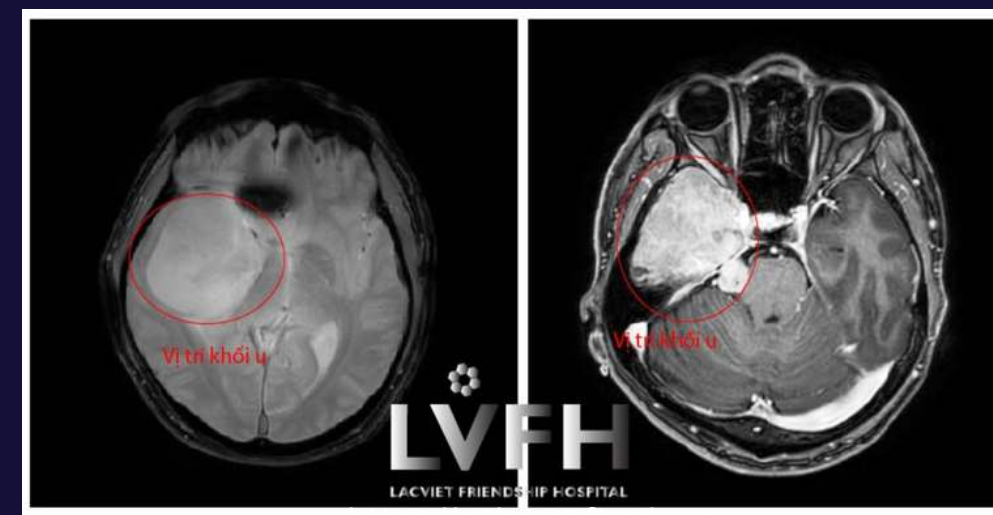
Breast Cancer Detector

👉 Trong bài toán phân loại khối u vú (lành tính ● hay ác tính ●):



🧪 Mục tiêu: Phân loại khối u là lành tính (Benign) hay ác tính (Malignant) từ các đặc trưng y học như:

- Bán kính trung bình
- Độ nhẵn bề mặt
- Độ đặc và kết cấu mô...



Breast Cancer Detetor

×

Input Method

Enter Values

BMI (kg/m2)

0.0000

-

+

Glucose (mg/dL)

0

-

+

Insulin (μU/mL)

0.0000

-

+

HOMA

0.0000

-

+

Resistin (ng/mL)

0.0000

-

+

Breast Cancer Prediction Using Artificial Intelligence 🤖

This web app uses machine learning to predict whether a person has breast cancer using some of their clinical data.

⚠ Not a diagnostic tool

This is just a demo application of machine learning.

🚧 Limitations

The dataset used to train the model is small and there might be better variables that could have been used.

Original dataset available here: [Breast Cancer Coimbra](#)

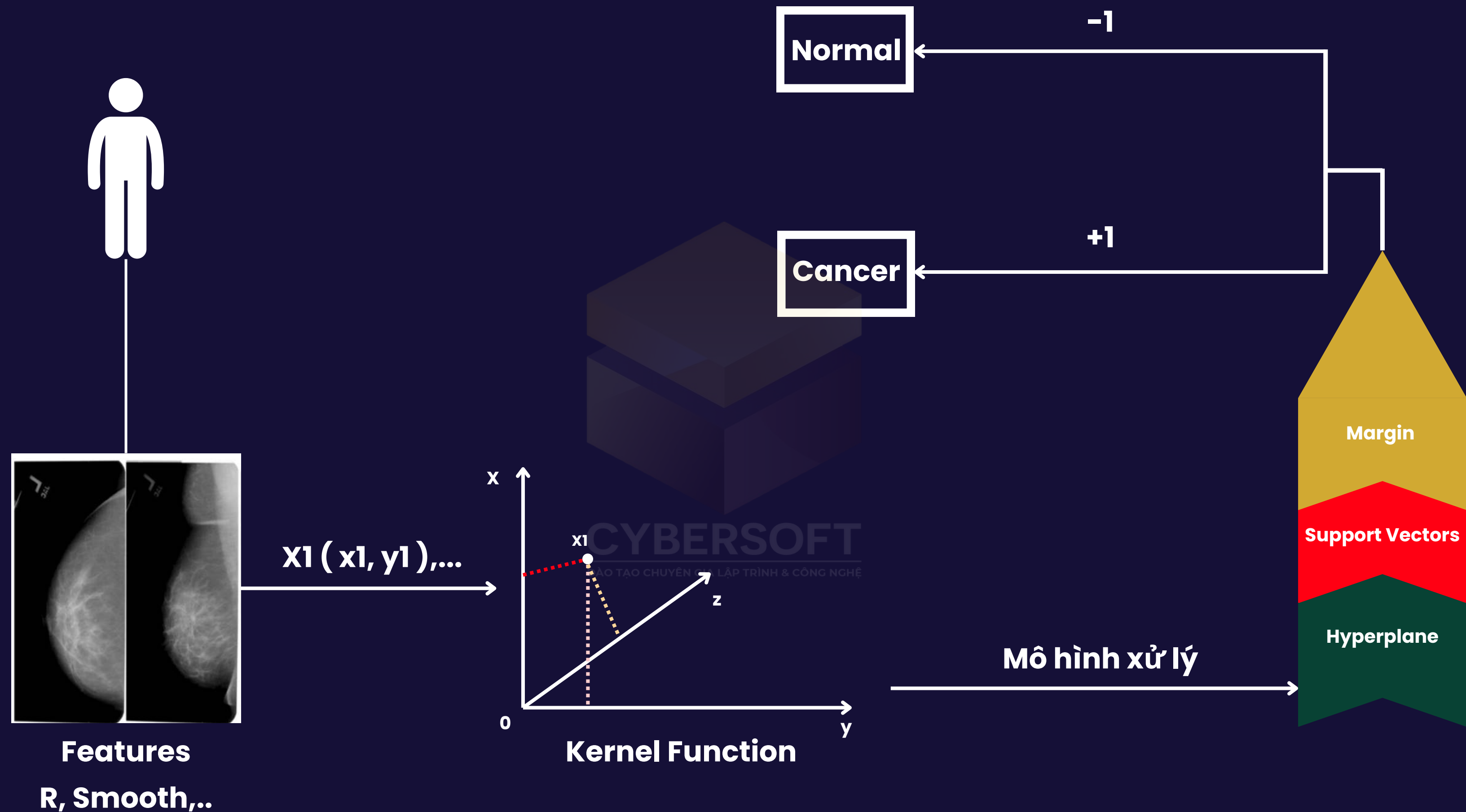
Details are described in: [Patrício, M., Pereira, J., Crisóstomo, J. et al. Using Resistin, glucose, age and BMI to predict the presence of breast cancer. BMC Cancer 18, 29 \(2018\)](#)

*click to show/hide

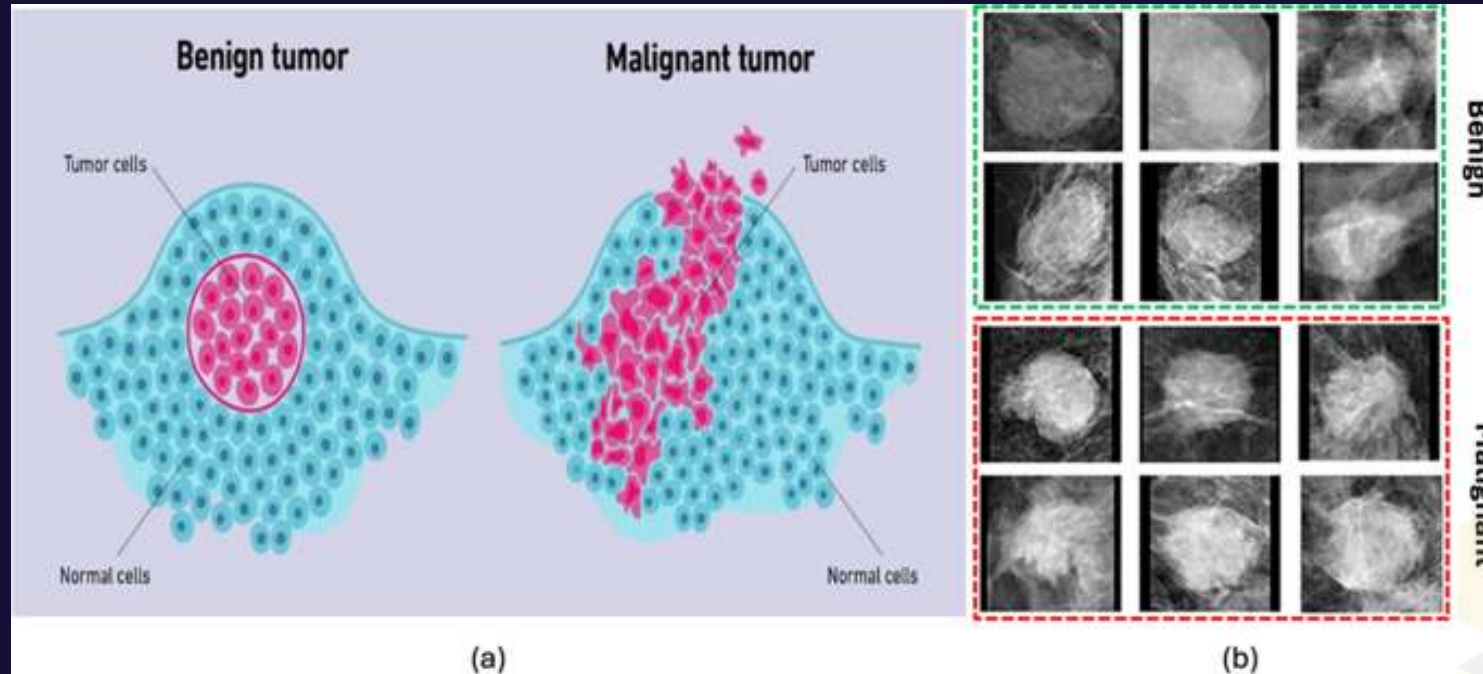
☐ Data

☐ Model Metrics

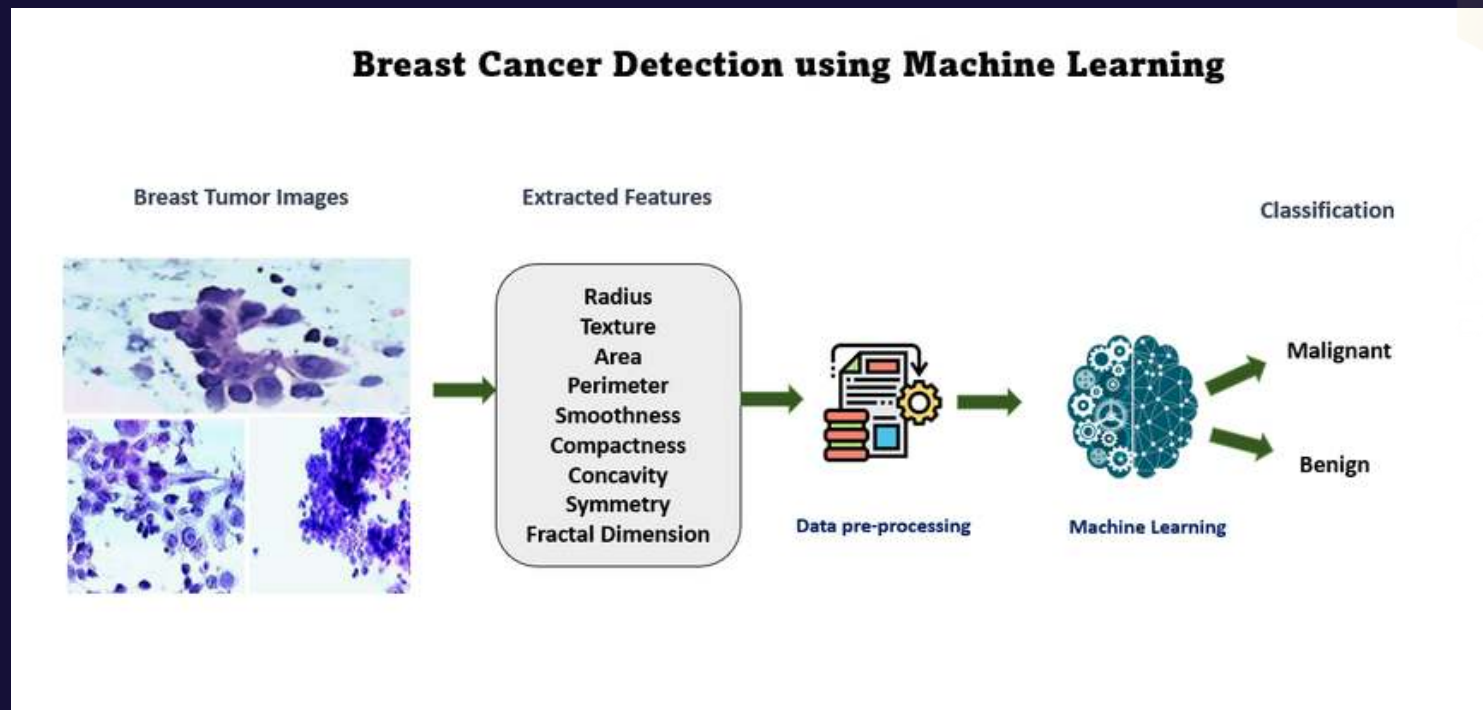
Breast Cancer Detetor



Chuẩn hóa Bài tập & Dự án

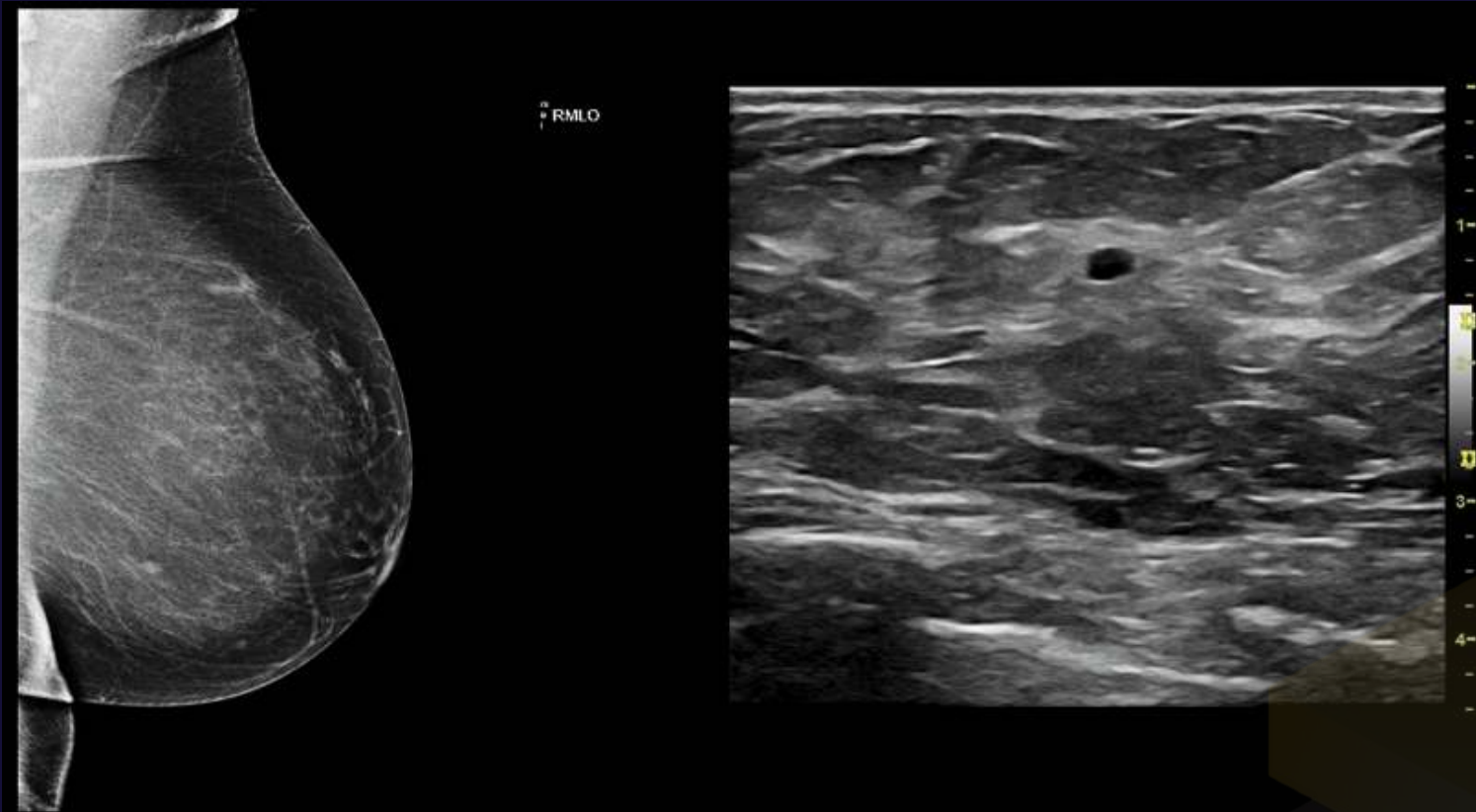


- Phân loại khối u là lành tính (Benign) hay ác tính (Malignant)

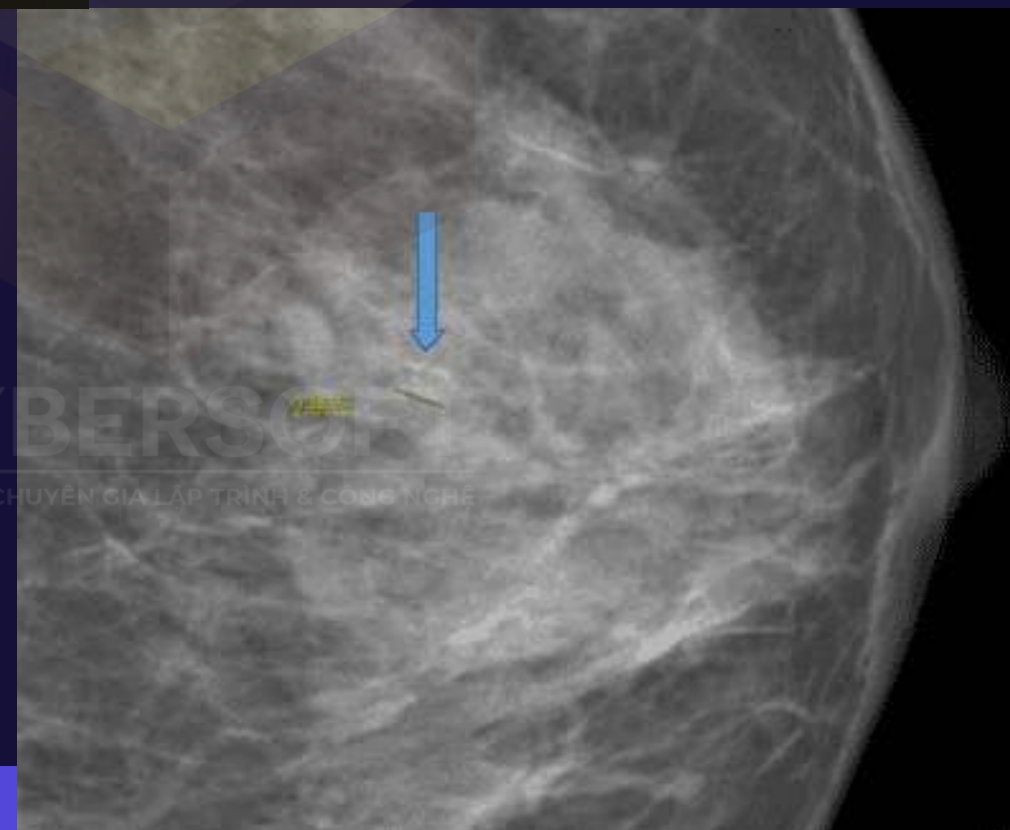
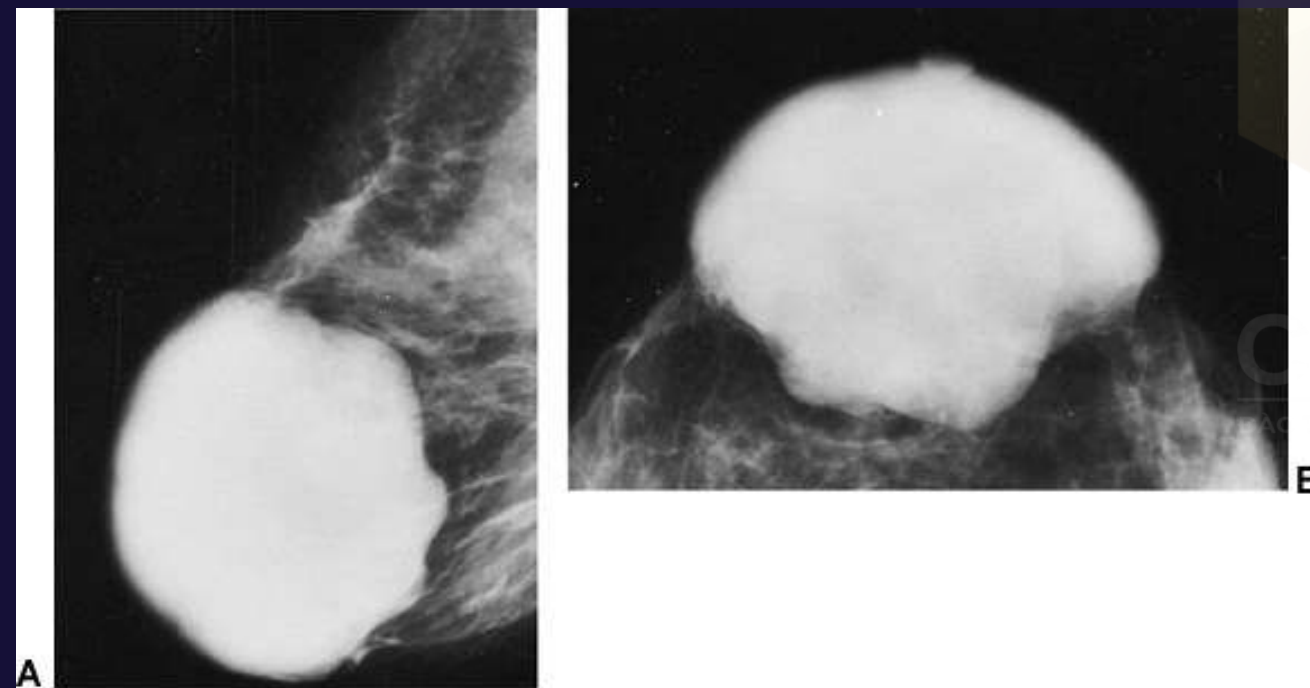


- Phân loại một khối u vú là:
 - Lành tính (Benign – 0) hoặc Ác tính (Malignant – 1)

Chuẩn hóa Bài tập & Dự án



Dùng Support Vector Machine (SVM) dựa vào các đặc trưng kích thước hạt nhân, độ mịn, đường viền,.

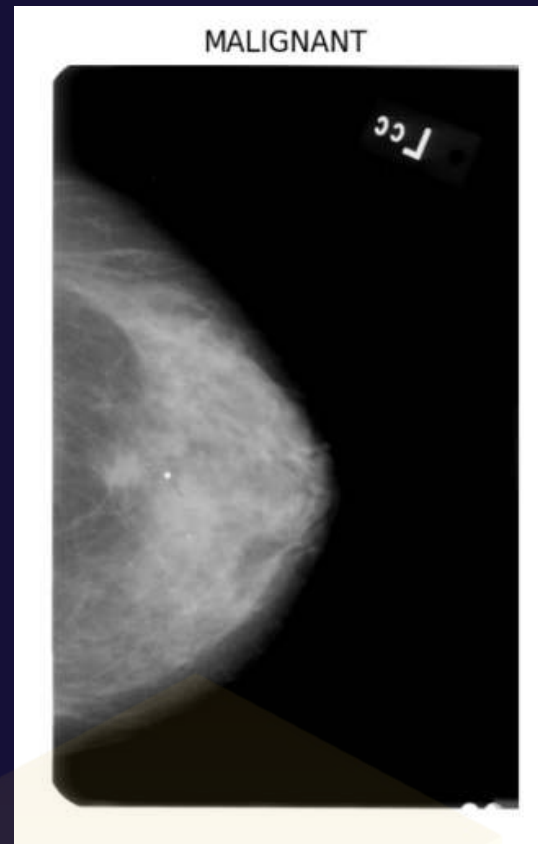


Flow dự án

Thu thập dữ liệu

Collect

kaggle



Cancer



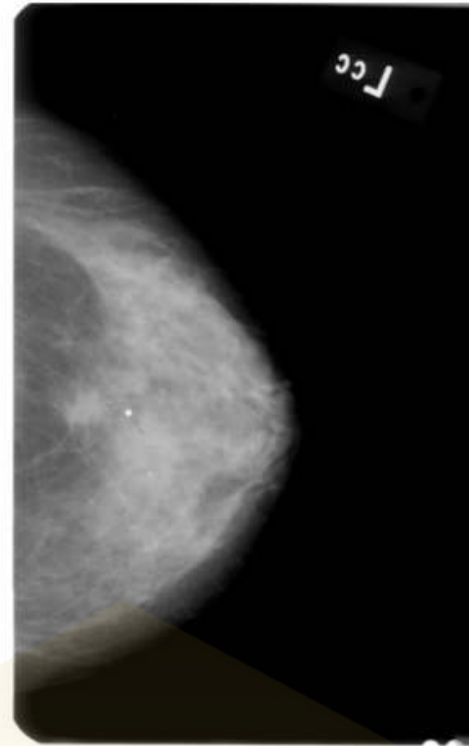
Normal

Flow dự án

Gán nhãn dữ liệu đầu vào cho dataset

Phân loại

MALIGNANT



$X = \text{Cancer}$

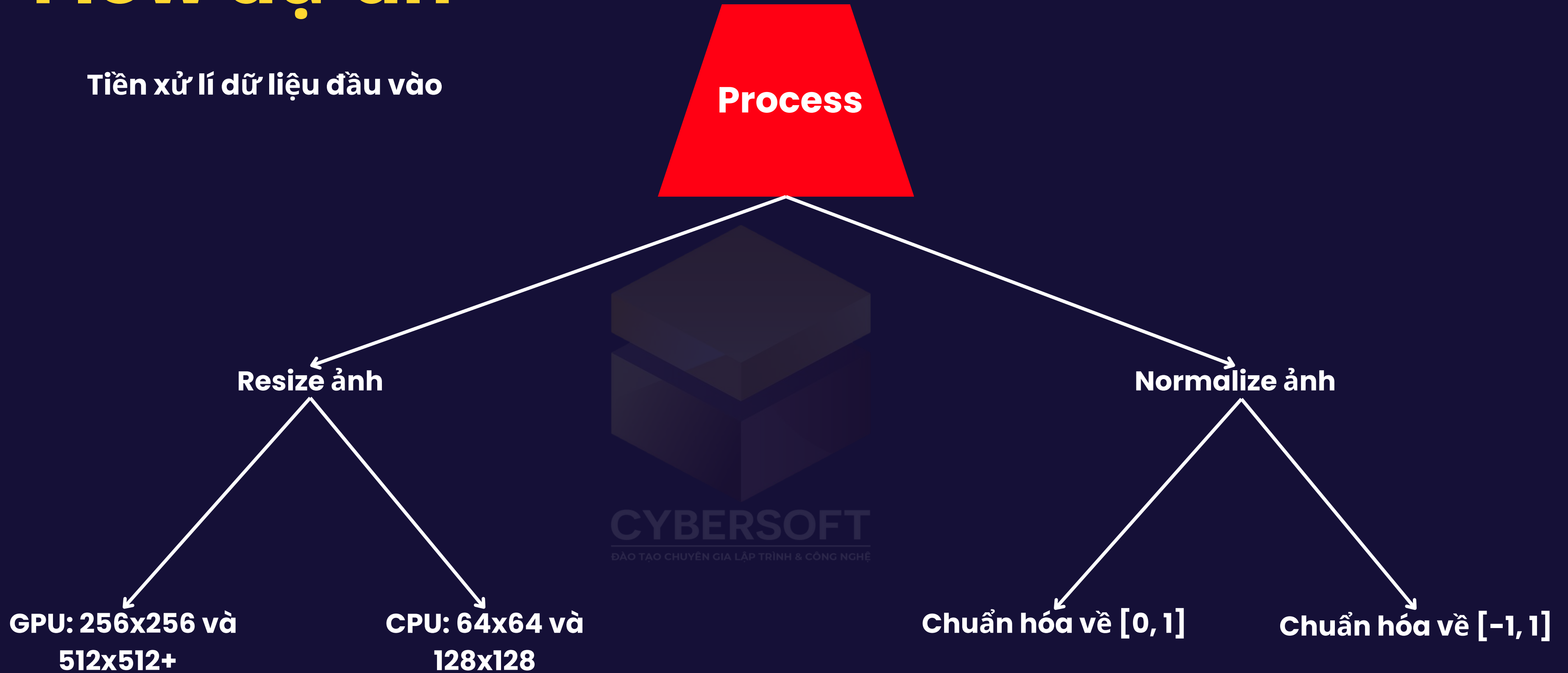
BENIGN



$Y = \text{Normal}$

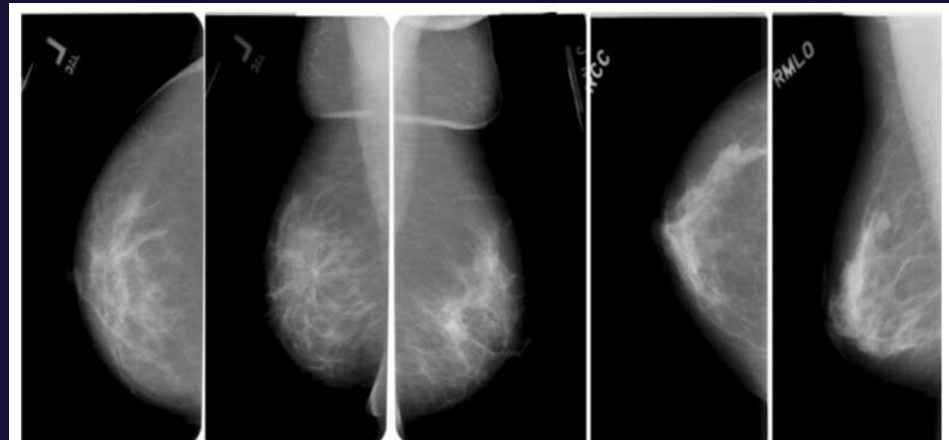
Flow dự án

Tiền xử lí dữ liệu đầu vào

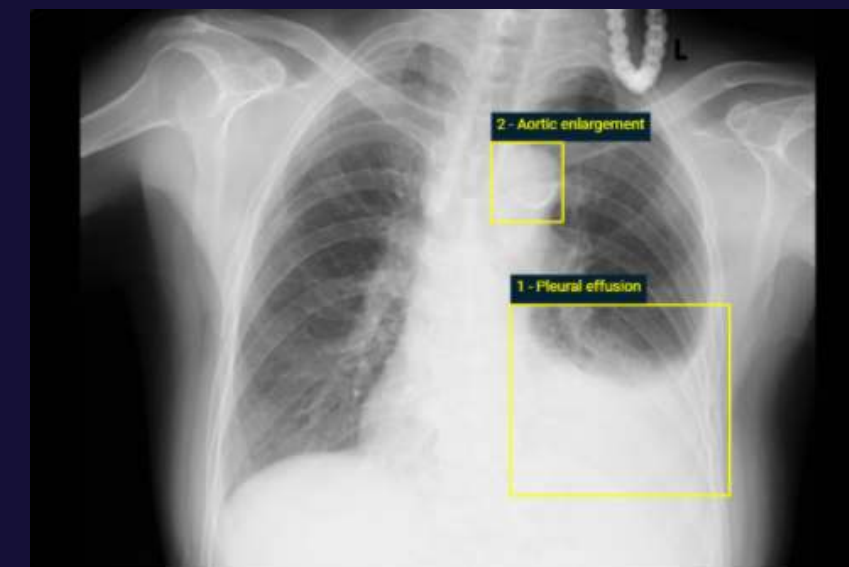


Flow dự án

Tách features từ dataset



Tách
features



Thư viện áp dụng



Pandas



Scikit – learn



Thuật toán áp dụng

Margin

Support Vector Machine (SVM)

Siêu phẳng phân chia

Kernel Trick (Kỹ thuật hạt nhân)

Các bước thực hiện



Bài tập thực hành: Breast Cancer Detetor

1. Cài đặt thư viện

```
pip install scipy==1.10.1 scikit-image seaborn numpy pandas scikit-learn
```



Bài tập thực hành: Breast Cancer Detector

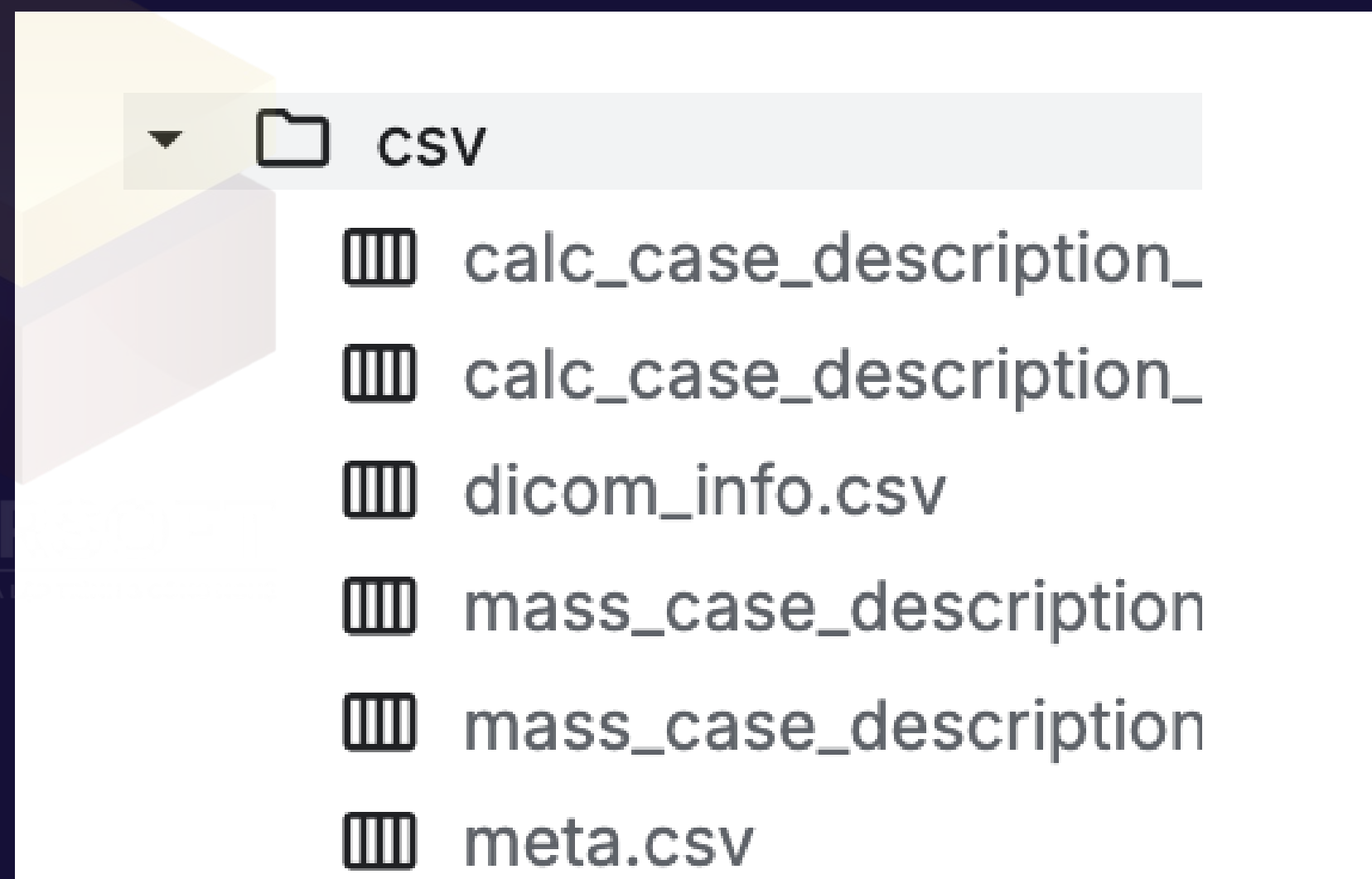
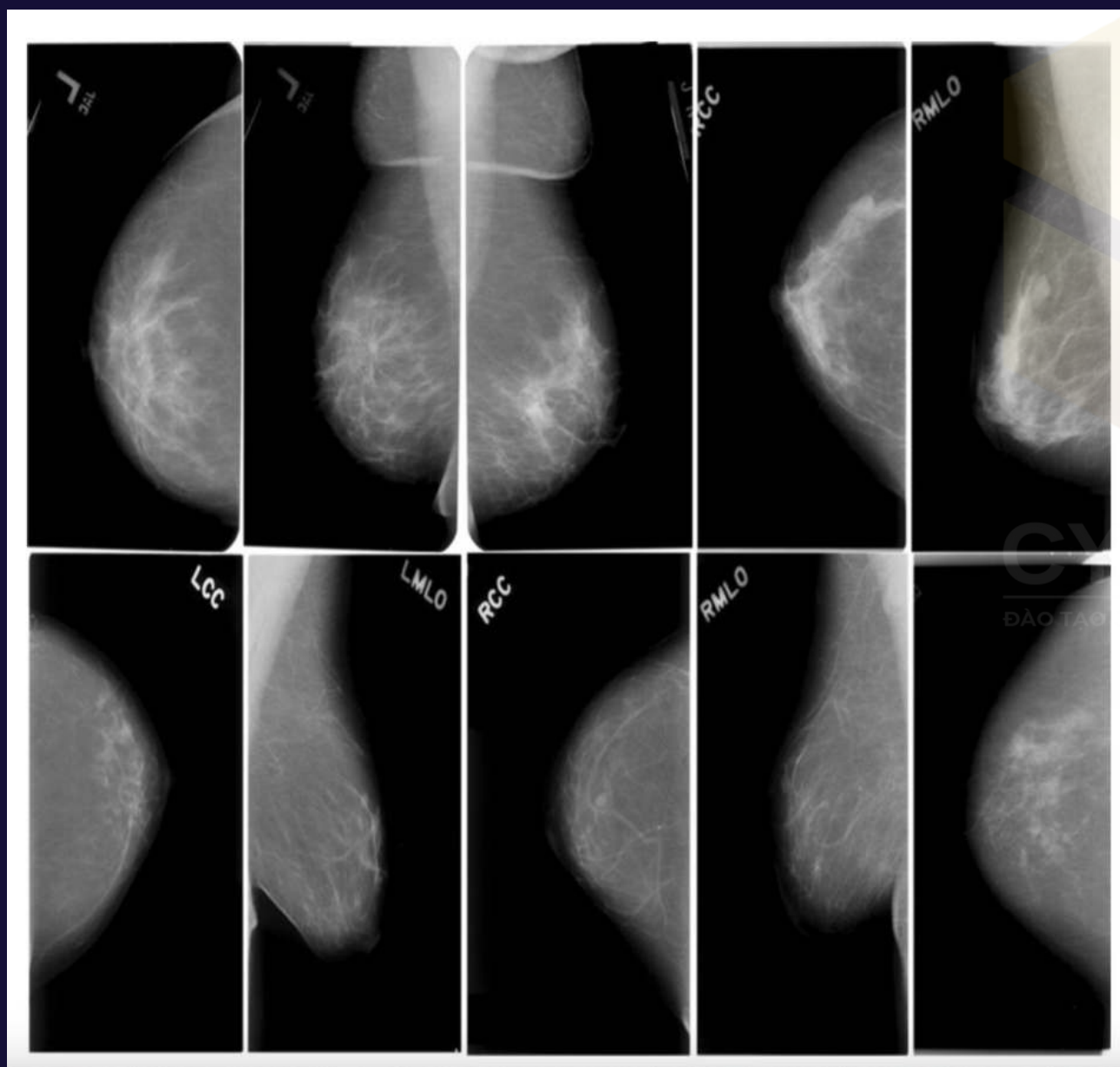
2. Import thư viện

```
# import libraries
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
import matplotlib.image as mpimg
from skimage.feature import hog
from skimage.transform import resize
import seaborn as sns
import os
```

Bài tập thực hành: Breast Cancer Detector

3. Import datasets

```
df_meta = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/meta.csv')
df_dicom = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/dicom_info.csv')
```

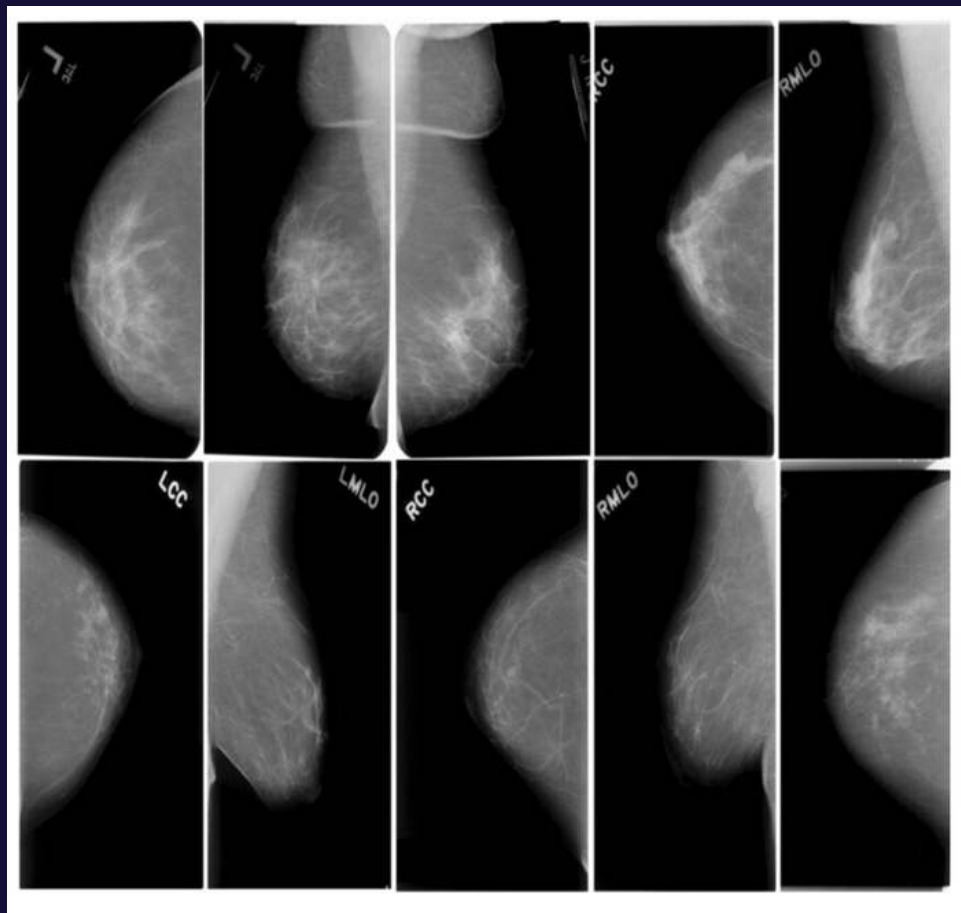


File CSV sẽ chứa các thông tin về các Features
File ảnh sẽ chứa ảnh để phân loại

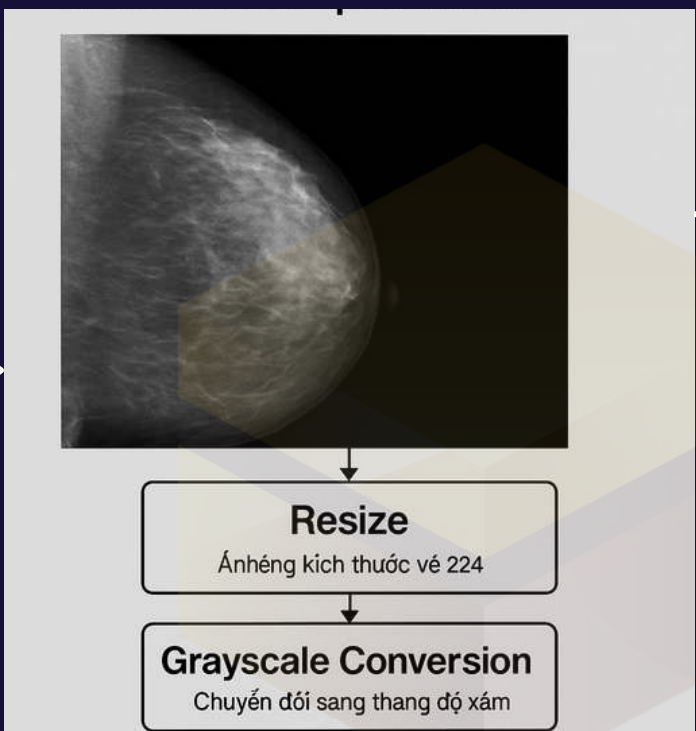
Bài tập thực hành: Breast Cancer Detetor

3. Import datasets

Load dataset



Chuẩn hóa hình ảnh



Tải và xử lý dữ liệu mô tả khối u

	patient_id	breast_density	left or right breast	image view	abnormality id	abnormality type	mass shape	mass margins	assessmer
0	P_00001	3	LEFT	CC	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	4
1	P_00001	3	LEFT	MLO	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	4
2	P_00004	3	LEFT	CC	1	mass	ARCHITECTURAL_DISTORTION	ILL_DEFINED	4
3	P_00004	3	LEFT	MLO	1	mass	ARCHITECTURAL_DISTORTION	ILL_DEFINED	4
4	P_00004	3	RIGHT	MLO	1	mass	OVAL	CIRCUMSCRIBED	4

Xử lý giá trị thiếu

patient_id	0
breast_density	0
left_or_right_breast	0
image_view	0
abnormality_id	0
abnormality_type	0
mass_shape	0

Tiền xử lý hình ảnh từ tập dữ liệu

	patient_id	breast_density	left_or_right_breast	image_view	abnormality_id	abnormality_type	mass_shape	mass_
0	P_00001	3	LEFT	CC	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICU
1	P_00001	3	LEFT	MLO	1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICU
2	P_00004	3	LEFT	CC	1	mass	ARCHITECTURAL_DISTORTION	ILL_D
3	P_00004	3	LEFT	MLO	1	mass	ARCHITECTURAL_DISTORTION	ILL_D
4	P_00004	3	RIGHT	MLO	1	mass	OVAL	CIRCU

Bài tập thực hành: Breast Cancer Detector

4. Tiền xử lý siêu dữ liệu hình ảnh

```
df_meta = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/meta.csv')
df_dicom = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/dicom_info.csv')

df_dicom.SeriesDescription.unique()
cropped_images = df_dicom[df_dicom.SeriesDescription=='cropped images'].image_path
full_mammo = df_dicom[df_dicom.SeriesDescription=='full mammogram images'].image_path
roi_img = df_dicom[df_dicom.SeriesDescription=='ROI mask images'].image_path
imdir = '../input/cbis-ddsm-breast-cancer-image-dataset/jpeg'
cropped_images = cropped_images.replace('CBIS-DDSM/jpeg', imdir, regex=True)
full_mammo = full_mammo.replace('CBIS-DDSM/jpeg', imdir, regex=True)
roi_img = roi_img.replace('CBIS-DDSM/jpeg', imdir, regex=True)

print('Cropped Images paths:\n')
print(cropped_images.iloc[0])
print('Full mammo Images paths:\n')
print(full_mammo.iloc[0])
print('ROI Mask Images paths:\n')
print(roi_img.iloc[0])
full_mammo_dict = dict()
cropped_images_dict = dict()
roi_img_dict = dict()

for dicom in full_mammo:
    key = dicom.split("/")[4]
    full_mammo_dict[key] = dicom
for dicom in cropped_images:
    key = dicom.split("/")[4]
    cropped_images_dict[key] = dicom
for dicom in roi_img:
    key = dicom.split("/")[4]
    roi_img[key] = dicom
next(iter((full_mammo_dict.items()))))
```

- Tải siêu dữ liệu và sắp xếp đường dẫn hình ảnh cho các loại ảnh (cắt xén, toàn bộ ảnh chụp X-quang vú, ảnh vùng quan tâm - ROI).
- Đọc meta.csv và dicom_info.csv để lấy thông tin siêu dữ liệu.
- Lọc df_dicom theo SeriesDescription để tách đường dẫn cho ảnh cắt xén, ảnh toàn bộ và ảnh ROI.
- Cập nhật đường dẫn ảnh phù hợp với cấu trúc thư mục cục bộ (imdir).
- Tạo từ điển (full_mammo_dict, cropped_images_dict, roi_img_dict) ánh xạ định danh ảnh (trích từ đường dẫn) tới đường dẫn file để dễ tra cứu.
- In mẫu đường dẫn để kiểm tra.

Bài tập thực hành: Breast Cancer Detetor

5. Tiền xử lý tập dữ liệu khối u

```
mass_train = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/mass_case_description_train_set.csv')
mass_test = pd.read_csv('/kaggle/input/cbis-ddsm-breast-cancer-image-dataset/csv/mass_case_description_test_set.csv')

mass_train.head()
def fix_image_path(data):
    """correct dicom paths to correct image paths"""
    for index, img in enumerate(data.values):
        img_name = img[11].split("/")[2]
        data.iloc[index,11] = full_mammo_dict[img_name]
        img_name = img[12].split("/")[2]
        data.iloc[index,12] = cropped_images_dict[img_name]

fix_image_path(mass_train)
fix_image_path(mass_test)
mass_train.pathology.unique()

mass_train = mass_train.rename(columns={'left or right breast': 'left_or_right_breast',
                                         'image view': 'image_view',
                                         'abnormality id': 'abnormality_id',
                                         'abnormality type': 'abnormality_type',
                                         'mass shape': 'mass_shape',
                                         'mass margins': 'mass_margins',
                                         'image file path': 'image_file_path',
                                         'cropped image file path': 'cropped_image_file_path',
                                         'ROI mask file path': 'ROI_mask_file_path'})

mass_train.head(5)
mass_train['mass_shape'] = mass_train['mass_shape'].bfill()
mass_train['mass_margins'] = mass_train['mass_margins'].bfill()
mass_train.isnull().sum()
mass_test['mass_margins'] = mass_test['mass_margins'].bfill()
mass_test.isnull().sum()

mass_test = mass_test.rename(columns={'left or right breast': 'left_or_right_breast',
                                         'image view': 'image_view',
                                         'abnormality id': 'abnormality_id',
                                         'abnormality type': 'abnormality_type',
                                         'mass shape': 'mass_shape',
                                         'mass margins': 'mass_margins',
                                         'image file path': 'image_file_path',
                                         'cropped image file path': 'cropped_image_file_path',
                                         'ROI mask file path': 'ROI_mask_file_path'})
```

Kiểm tra cho các trường hợp khối u, sửa đường dẫn và xử lý giá trị thiếu.

Bài tập thực hành: Breast Cancer Detetor

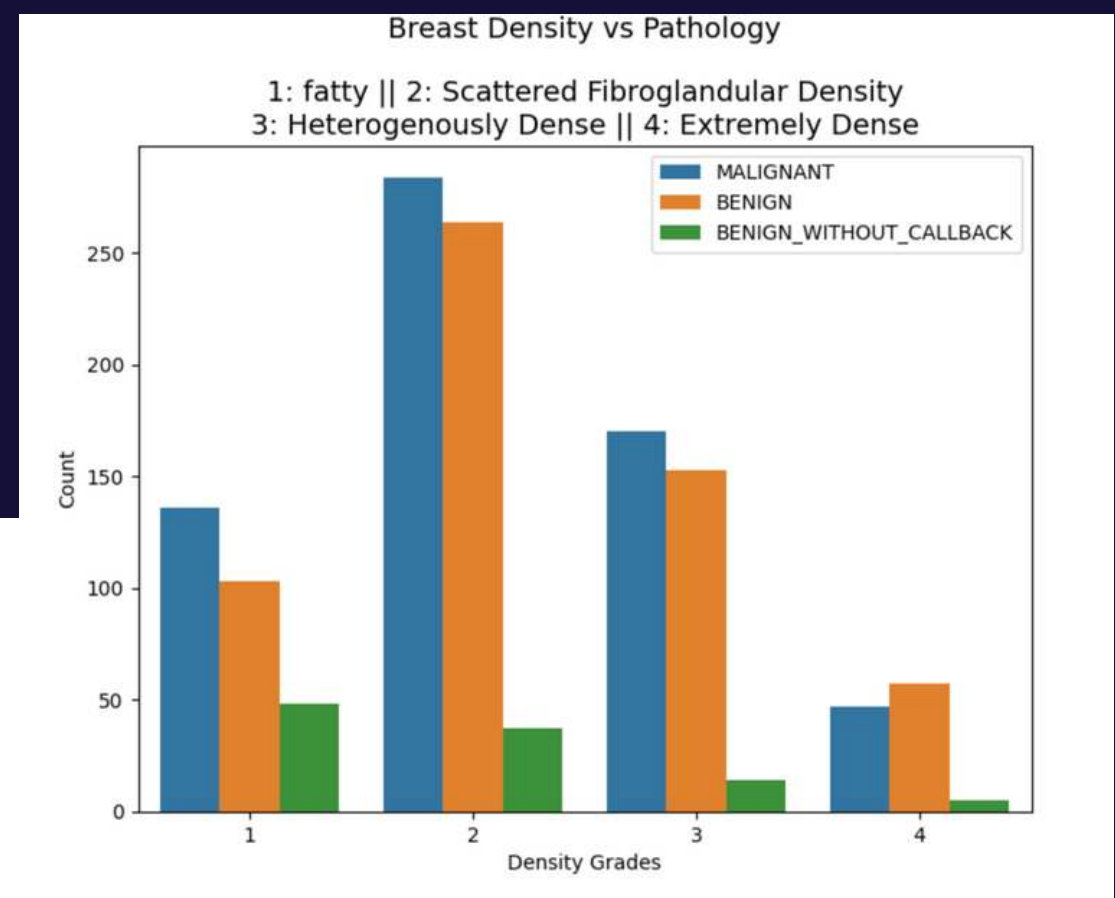
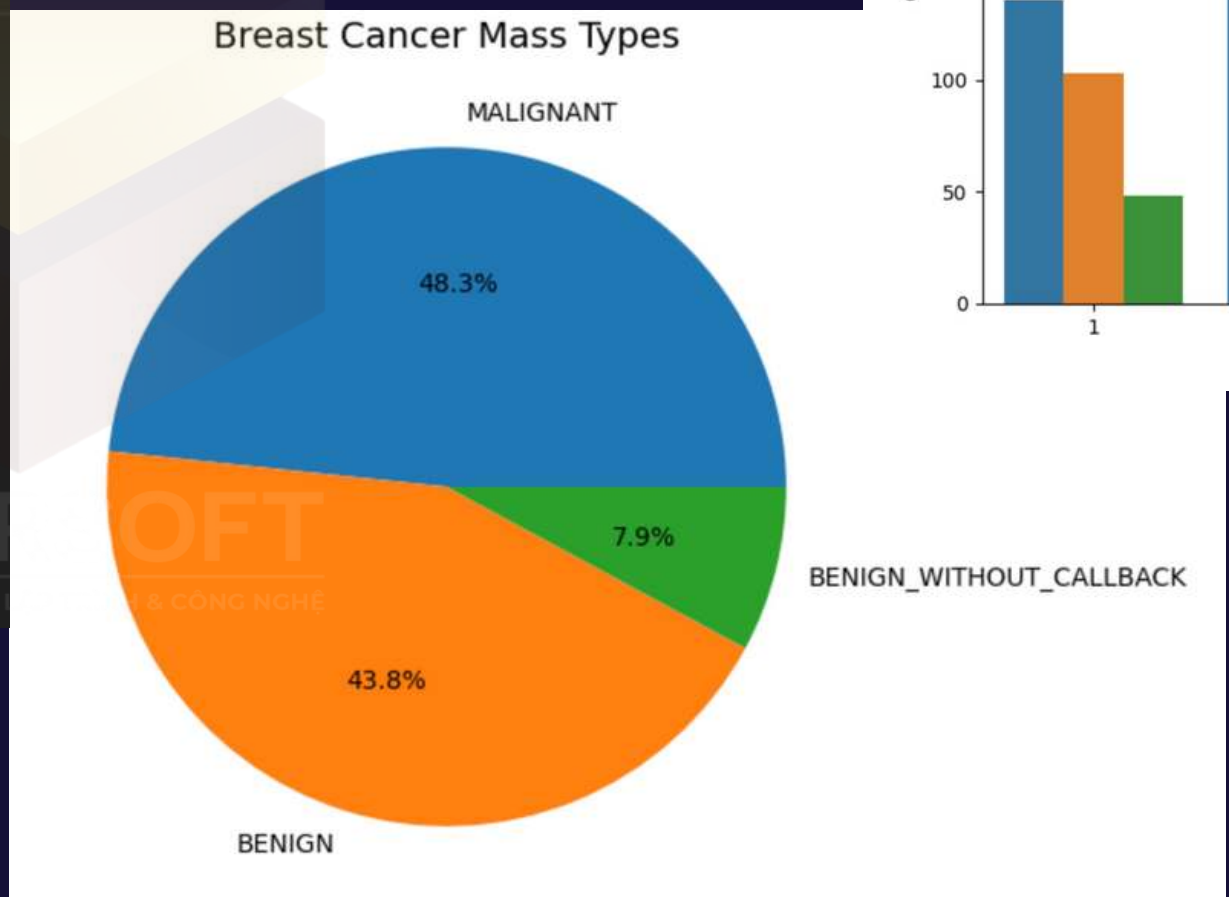
6. Trực quan hóa dữ liệu

```
# pathology distributions
value = mass_train['pathology'].value_counts()
plt.figure(figsize=(8,6))

plt.pie(value, labels=value.index, autopct='%1.1f%%')
plt.title('Breast Cancer Mass Types', fontsize=14)
plt.savefig('/kaggle/working/pathology_distributions_red.png')
plt.show()

# breast density against pathology
plt.figure(figsize=(8,6))

sns.countplot(mass_train, x='breast_density', hue='pathology')
plt.title('Breast Density vs Pathology\n\n1: fatty || 2: Scattered Fibroglandular ' \
'Density\n3: Heterogenously Dense || 4: Extremely Dense',
           fontsize=14)
plt.xlabel('Density Grades')
plt.ylabel('Count')
plt.legend()
plt.savefig('/kaggle/working/density_pathology_red.png')
plt.show()
```



Bài tập thực hành: Breast Cancer Detector

6. Trực quan hóa dữ liệu

```
# Display some images
import matplotlib.image as mpimg
# create function to display images
def display_images(column, number):
    # create figure and axes
    number_to_visualize = number
    rows = 1
    cols = number_to_visualize
    fig, axes = plt.subplots(rows, cols, figsize=(15, 5))

    # Loop through rows and display images
    for index, row in mass_train.head(number_to_visualize).iterrows():
        image_path = row[column]
        image = mpimg.imread(image_path)
        ax = axes[index]
        ax.imshow(image, cmap='gray')
        ax.set_title(f"{row['pathology']}")
        ax.axis('off')
    plt.tight_layout()
    plt.show()

print('Full Mammograms:\n')
display_images('image_file_path', 5)
print('Cropped Mammograms:\n')
display_images('cropped_image_file_path', 5)
```

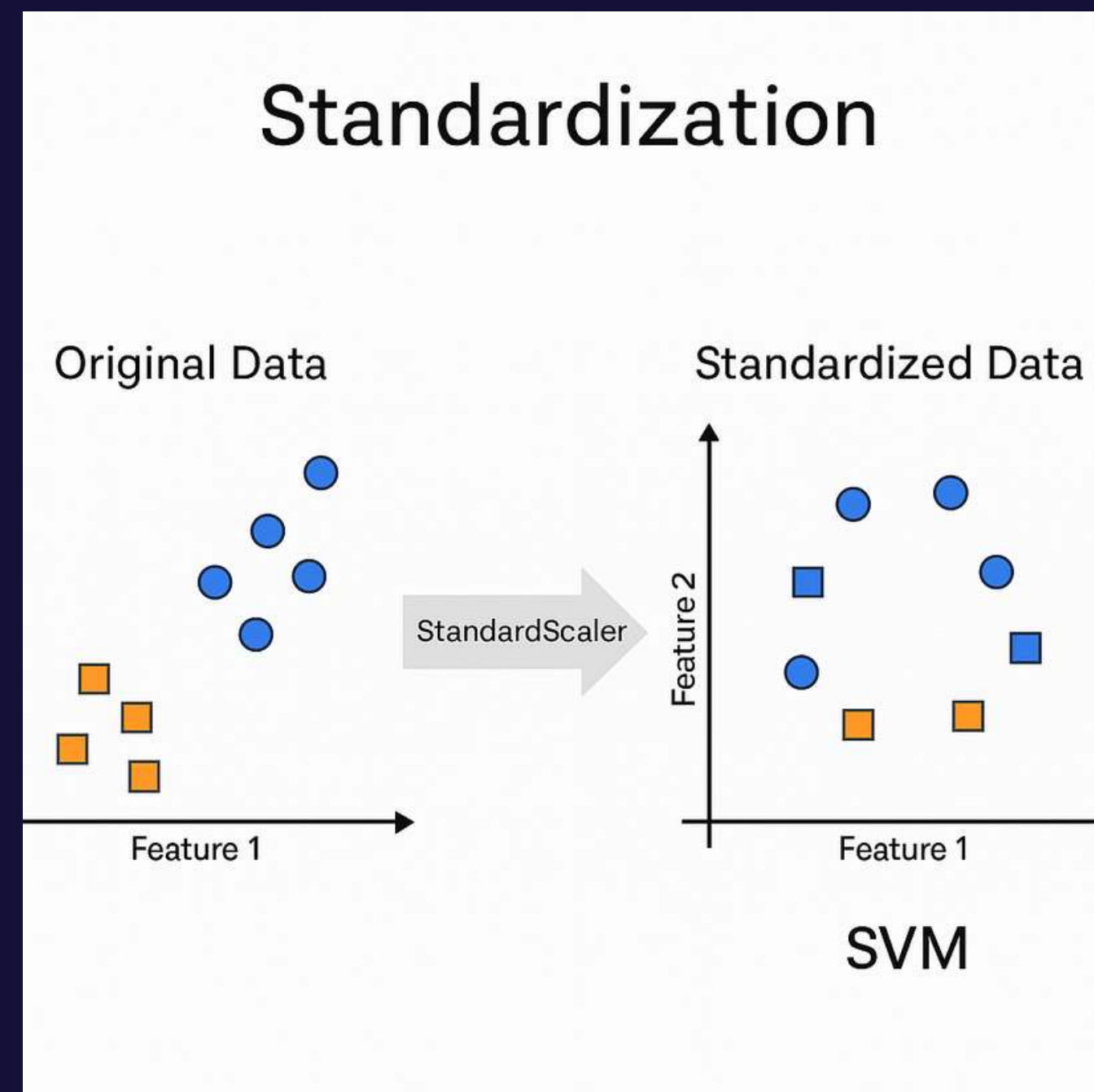
Bài tập thực hành: Breast Cancer Detetor

7. Huấn luyện mô hình

StandardScaler là một kỹ thuật chuẩn hóa đặc trưng phổ biến trong Machine Learning – đặc biệt quan trọng với các mô hình như SVM, vì SVM nhạy cảm với thang đo của dữ liệu.

```
# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

=> Tăng độ chính xác mô hình

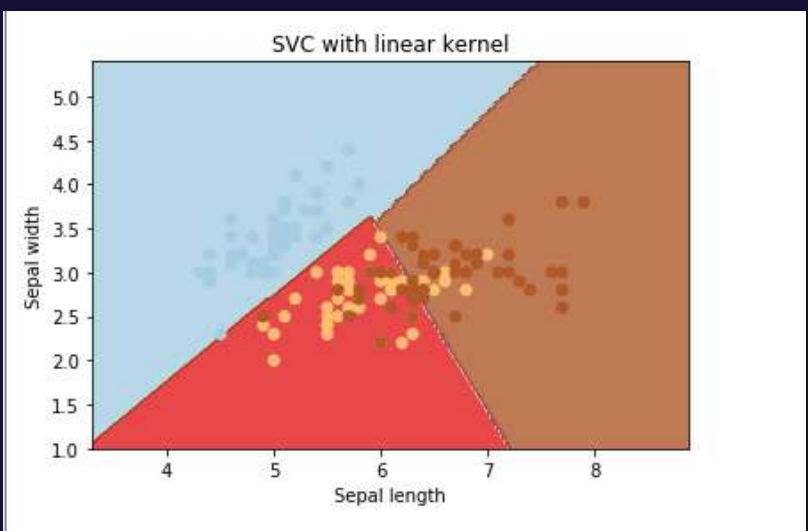


Bài tập thực hành: Breast Cancer Detetor

8. Lựa chọn Kernel để huấn luyện

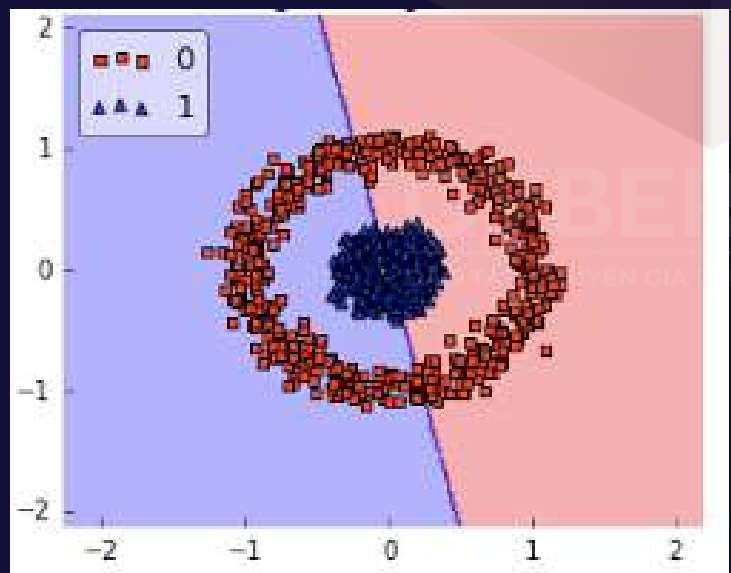
Kernel

Linear



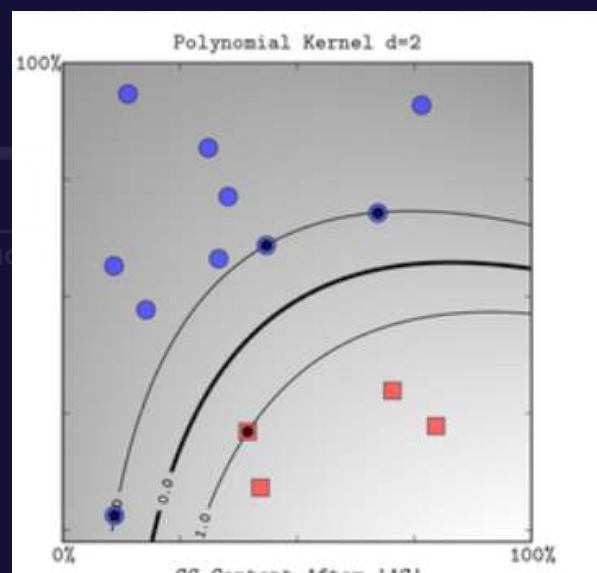
Dữ liệu tuyến tính

rbf



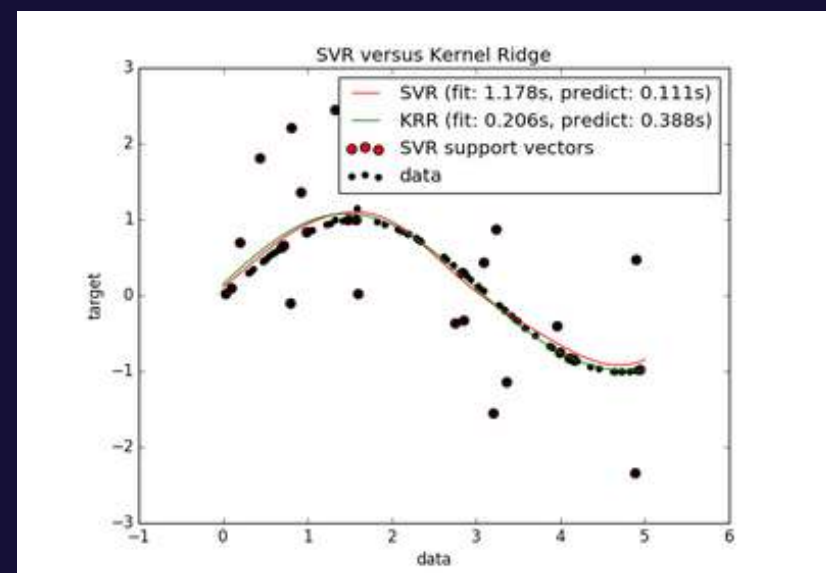
Có cụm rõ ràng,
ranh giới phức tạp

poly



Muốn thử dữ liệu cong
vừa phải

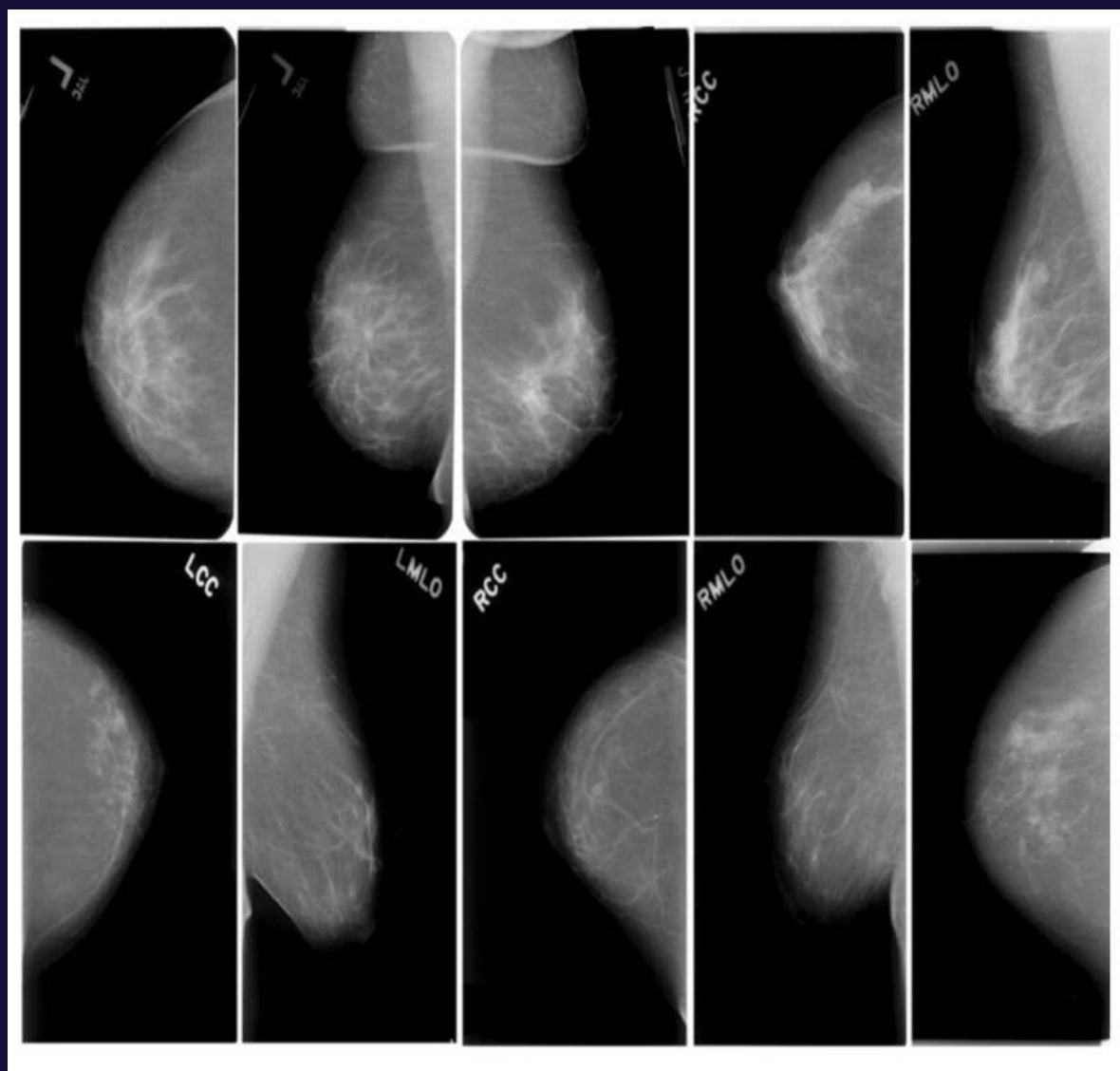
GridSearchCV



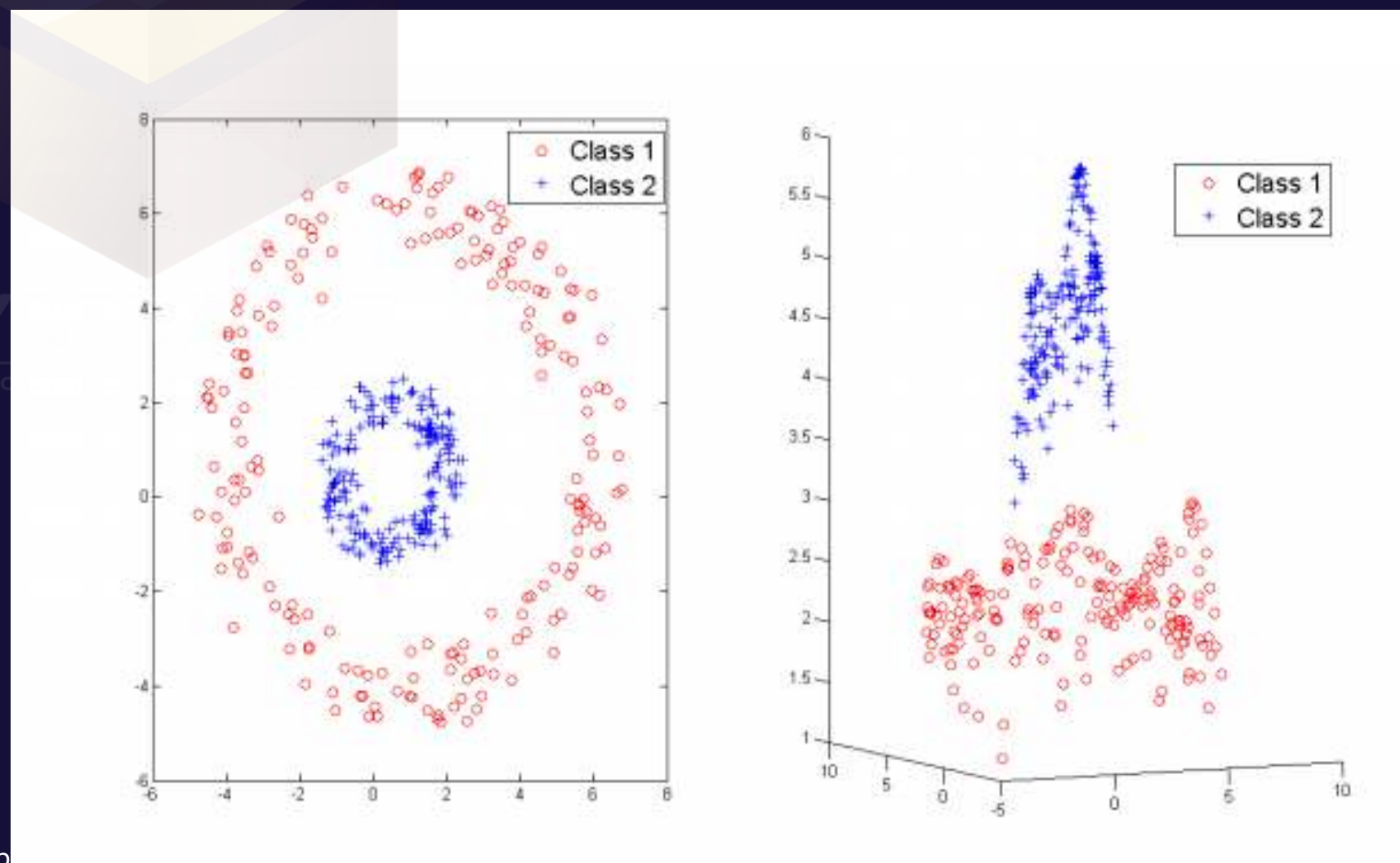
So sánh nhanh nhiều kernel

Bài tập thực hành: Breast Cancer Detector

8. Lựa chọn Kernel để huấn luyện



- Với dữ liệu đã được label và gán nhãn sẵn => **RBF kernel**
- Dữ liệu ảnh không tuyến tính
- StandardScaler đã chuẩn bị xong
- RBF là mặc định trong SVM sklearn

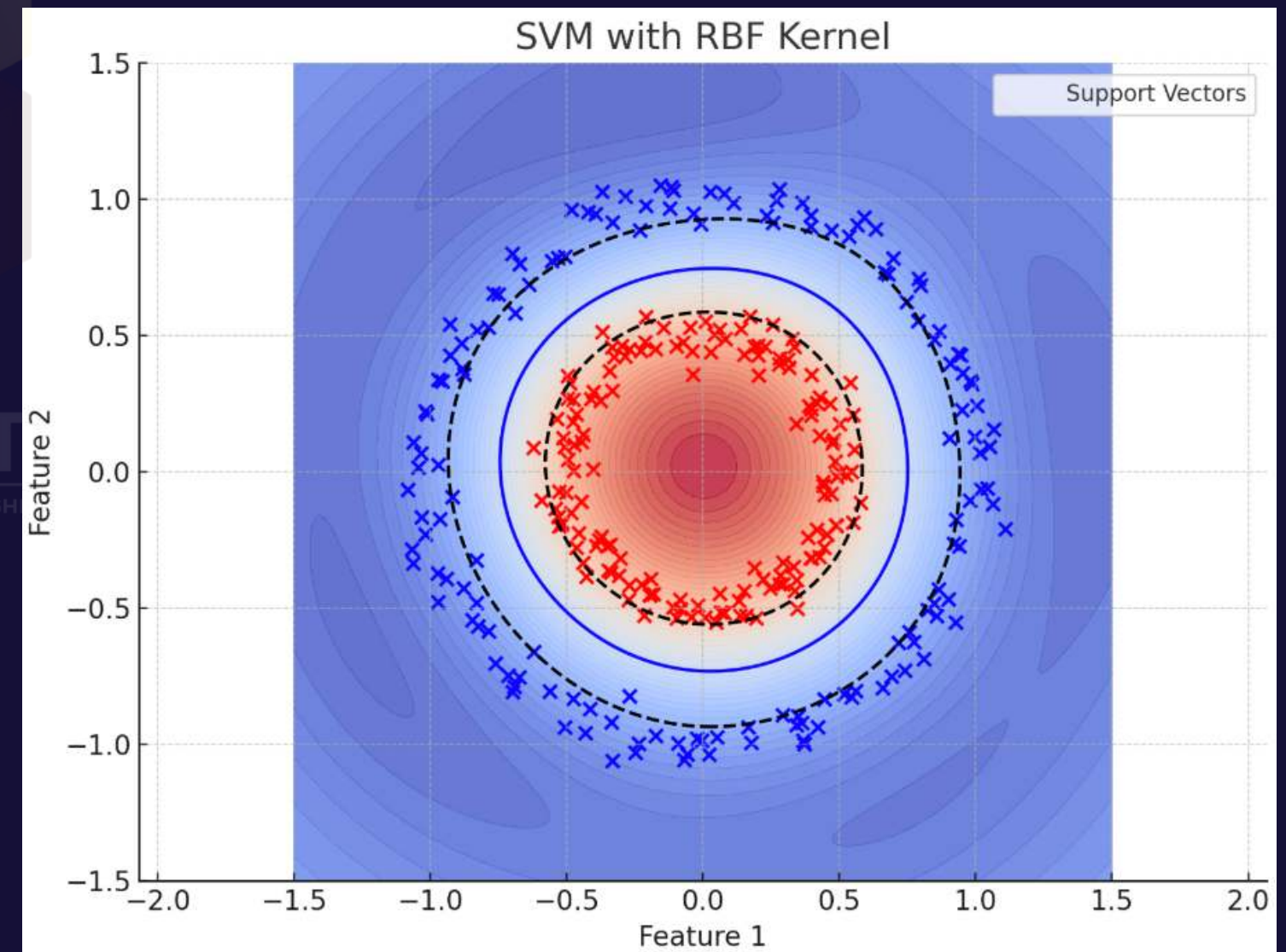


Bài tập thực hành: Breast Cancer Detetor

8. Lựa chọn Kernel để huấn luyện

```
# Train SVM model
print("Training SVM model...")
svm_model = SVC(kernel='rbf', random_state=42)
svm_model.fit(X_train_scaled, y_train)
```

- kernel='rbf': chọn kernel Gaussian Radial Basis Function
- C: kiểm soát mức độ cho phép sai số
- gamma: độ ảnh hưởng của một điểm dữ liệu; 'scale' là lựa chọn mặc định tốt



Bài tập thực hành: Breast Cancer Detetor

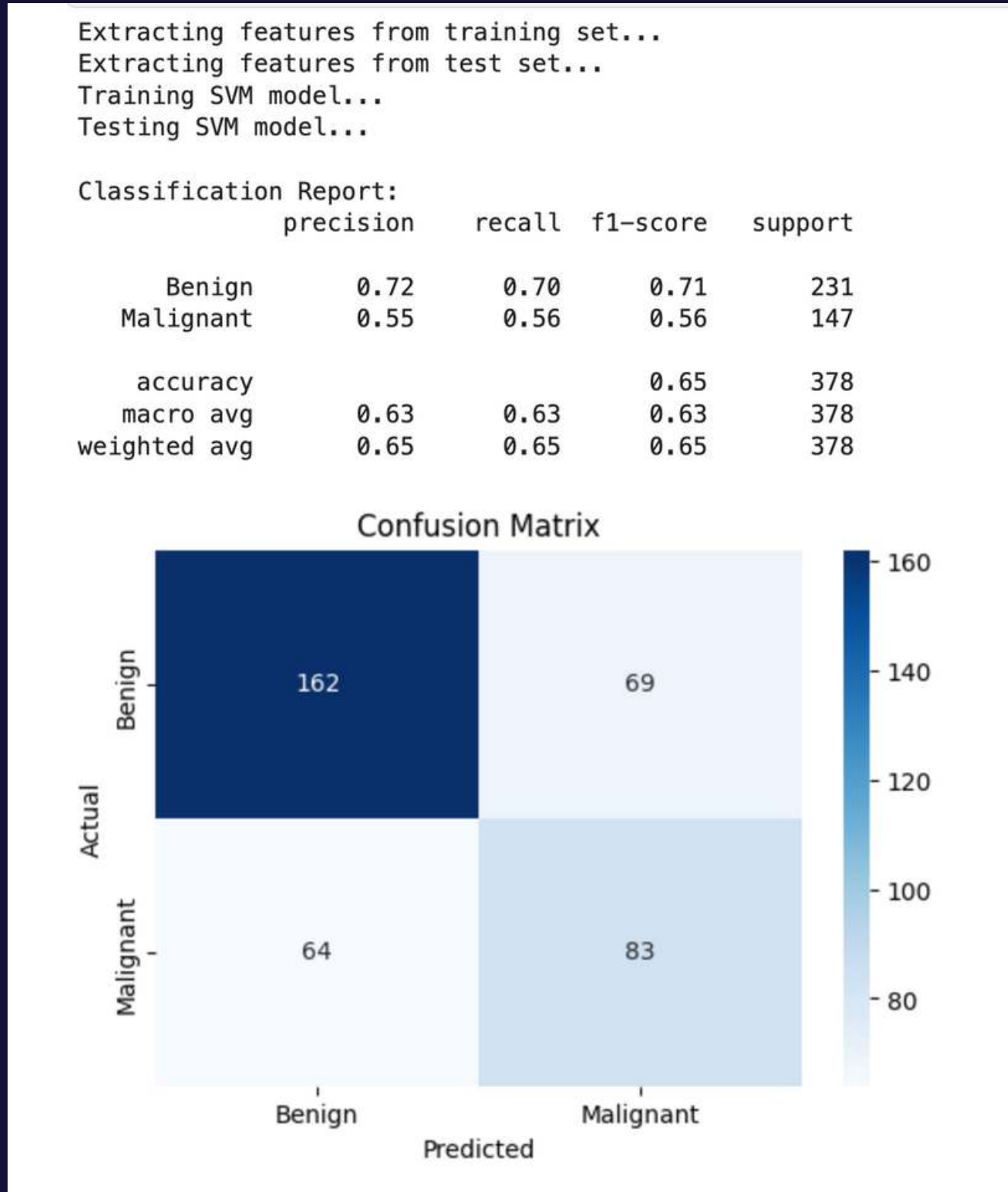
9. Đánh giá mô hình

```
# Test the model (overall performance)
print("Testing SVM model...")
y_pred = svm_model.predict(X_test_scaled)

# Evaluate the model
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Benign', 'Malignant']))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.savefig('/kaggle/working/confusion_matrix_svm.png')
plt.show()
```

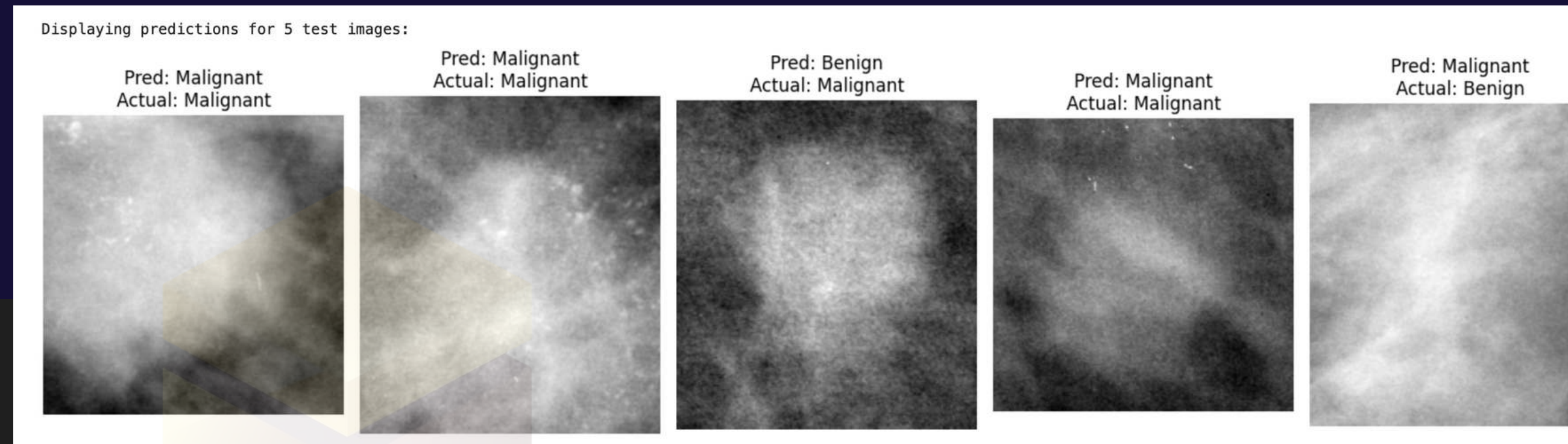
Trích xuất đặc trưng từ ảnh cắt xén cho tập huấn luyện (X_train) và kiểm tra (X_test).
Chuẩn hóa đặc trưng bằng StandardScaler.
Huấn luyện mô hình SVM với kernel RBF.
Dự đoán trên tập kiểm tra và đánh giá bằng báo cáo phân loại (độ chính xác, recall, F1-score) và ma trận nhầm lẫn.
Trực quan hóa ma trận nhầm lẫn dưới dạng heatmap.



Bài tập thực hành: Breast Cancer Detetor

10. Trực quan hóa kết quả dự đoán

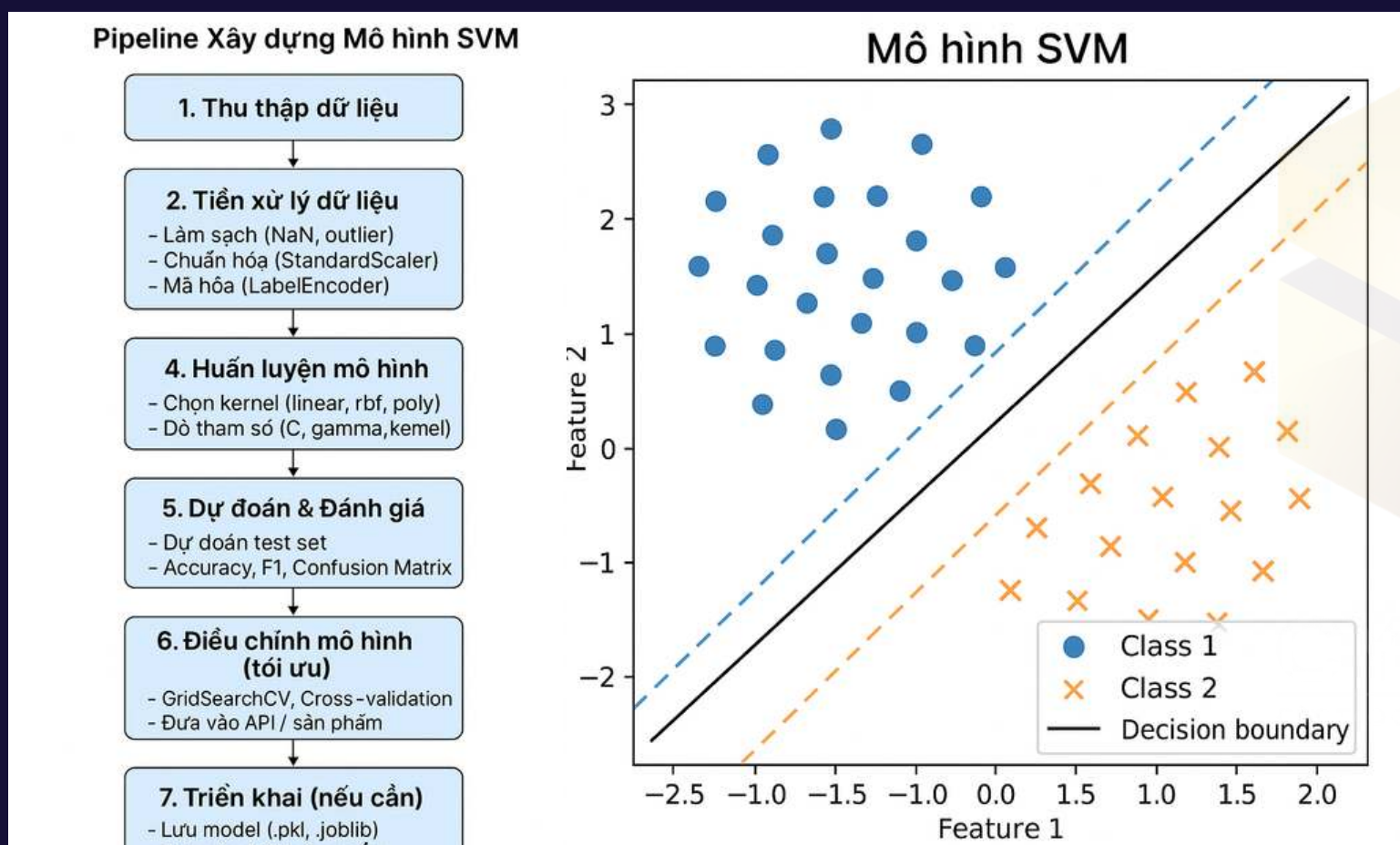
```
def test_and_display_images(model, scaler, image_paths, labels, number=5):  
    # Extract features for the selected images  
    features = extract_features(image_paths[:number])  
    if len(features) == 0:  
        print("No valid images to process.")  
        return  
  
    # Scale features  
    features_scaled = scaler.transform(features)  
  
    # Predict  
    predictions = model.predict(features_scaled)  
    label_map = {0: 'Benign', 1: 'Malignant'}  
  
    # Display images with predictions  
    fig, axes = plt.subplots(1, number, figsize=(15, 5))  
    for idx, (path, pred, actual) in enumerate(zip(image_paths[:number], predictions, labels[:number])):  
        img = mpimg.imread(path)  
        ax = axes[idx] if number > 1 else axes  
        ax.imshow(img, cmap='gray')  
        ax.set_title(f"Pred: {label_map[pred]}\nActual: {label_map[actual]}")  
        ax.axis('off')  
    plt.tight_layout()  
    plt.savefig('/kaggle/working/test_image_predictions.png')  
    plt.show()  
  
test_and_display_images(svm_model, scaler, mass_test['cropped_image_file_path'], mass_test['label'], number=5)
```



Nội dung

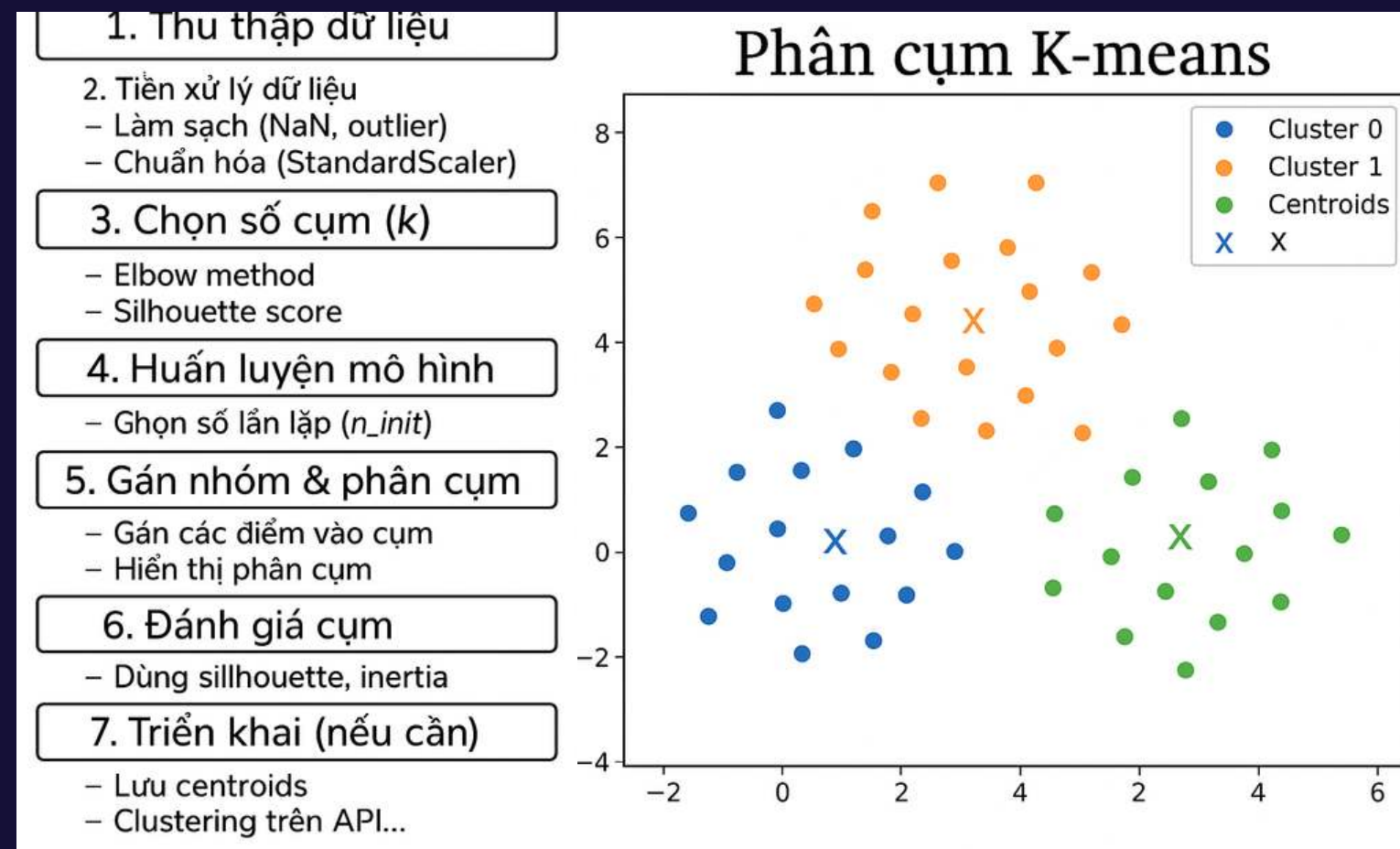
➤ Bài tập về nhà

- Biết cách xây dựng mô hình SVM và K-Means từ đầu
- So sánh trực quan hai mô hình



Mô hình SVM

Ảnh minh họa SVM phân loại hai lớp với siêu phẳng quyết định và hai biên margin tối ưu.



Mô hình K-Means

Ảnh trên minh họa K-means phân cụm dữ liệu thành 3 nhóm với tâm cụm (centroids) được đánh dấu rõ ràng.